

БИЛЕТ 1

1. Бинарный алгоритм.
2. Генератор нелинейных M - последовательностей.
3. Алгоритм цифровой подписи DSA.
4. Основные положения эллиптической криптографии. Эллиптические группы.

Бинарный алгоритм

Является усовершенствованным (развитым) алгоритмом Евклида.

Основан на использовании 4 правил:

1. Если числа a и b – **чётные**, то:
 $(a, b) = 2(a/2, b/2)$
2. Если a – **чётное**, a b - **нечётное**, то
 $(a, b) = (a, b/2)$
3. $(a, b) = (b, a-b)$
4. Если a и b – **нечётные**, то $a-b$ – чётное

Числа a и b взаимно простые, если $(a, b) = 1$;

Числа $a_1, a_2, \dots, a_n$ попарно простые, если любая пара этих чисел – взаимно простая

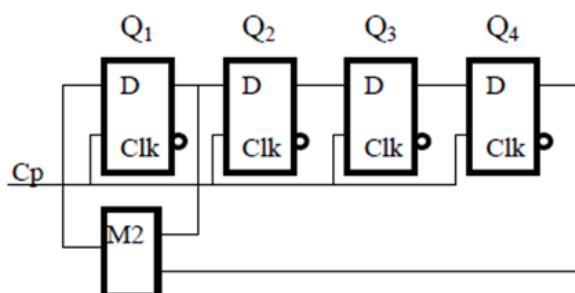
Генераторы нелинейных M -последовательностей

Являются одними из наиболее часто используемых типов генераторов псевдослучайных последовательностей.

Для их реализации используется сдвиговый регистр с линейной обратной связью (LFSR)

Подобный генератор часто используется как источник криптографического ключа для потоковых систем шифрования. Одна из основных причин широкого применения генераторов M последовательностей является сравнительно простая их реализация как программная, так и аппаратная. Кроме того, отмечаются высокие статистические свойства M последовательностей практически не отличающиеся от свойств случайных последовательностей. В тоже время генераторы M последовательностей подобно, как и любые другие генераторы псевдослучайных последовательностей обеспечивают воспроизводимость выходных последовательностей. Подобные генераторы, далее их будем обозначать LFSR, строятся на основании примитивных порождающих полиномов.

Структурная схема LFSR на основе порождающего полинома :



Составляющие схемы:

1. Триггеры типа D:

Q1, Q2, Q3, Q4 — это триггеры типа D, каждый из которых хранит один бит состояния.

На вход каждого триггера подается сигнал сдвига (Clk), который синхронизирует работу всех триггеров.

2. Сигнал тактирования (Cp):

Синхронизирует обновление состояния триггеров.

3. Модуль обратной связи (M2):

M2 обозначает операцию XOR (исключающее ИЛИ), которая выполняется над выходами Q1 и Q4. Результат подается на вход первого триггера (D вход Q1).

Состояния генератора LFSR

#	$Q_1Q_2Q_3Q_4$	#	$Q_1Q_2Q_3Q_4$
0	1 0 0 0	8	1 1 0 1
1	1 1 0 0	9	0 1 1 0
2	1 1 1 0	10	0 0 1 1
3	1 1 1 1	11	1 0 0 1
4	0 1 1 1	12	0 1 0 0
5	1 0 1 1	13	0 0 1 0
6	0 1 0 1	14	0 0 0 1
7	1 0 1 0	15	1 0 0 0

LFSR генерирует всевозможные четырехразрядные двоичные коды, 100 кроме нулевого кода (0000). Последовательность генерируемых кодов имеет случайный характер и не зависит от ненулевого начального состояния.

Существуют примитивные порождающие полиномы (для любой разрядности):

Примитивные порождающие полиномы

$m=\deg \varphi(x)$	$\varphi(x)$	$m=\deg \varphi(x)$	$\varphi(x)$
1	$1+x$	15	$1+x+x^{15}$
2	$1+x+x^2$	16	$1+x^2+x^3+x^5+x^{16}$
3	$1+x+x^3$	17	$1+x^3+x^{17}$
4	$1+x+x^4$	18	$1+x^7+x^{18}$
5	$1+x^2+x^5$	19	$1+x+x^2+x^5+x^{19}$
6	$1+x+x^6$	20	$1+x^3+x^{20}$
7	$1+x+x^7$	21	$1+x^2+x^{21}$
8	$1+x+x^5+x^6+x^8$	22	$1+x+x^{22}$
9	$1+x^4+x^9$	23	$1+x^5+x^{23}$
10	$1+x^3+x^{10}$	24	$1+x^3+x^4+x^{24}$
11	$1+x^2+x^{11}$	25	$1+x^3+x^{25}$
12	$1+x^3+x^4+x^7+x^{12}$	26	$1+x+x^2+x^6+x^{26}$
13	$1+x+x^3+x^4+x^{13}$	27	$1+x+x^2+x^5+x^{27}$
14	$1+x+x^{11}+x^{12}+x^{14}$	28	$1+x^3+x^{28}$

XOR - линейная операция

1. $a \oplus 0 = a$

2. $a \oplus a = 0$

3. $(a \oplus b) \oplus c = a \oplus (b \oplus c)$

Существует две альтернативные структуры реализации LFSR с внешними и внутренними сумматорами по модулю два.

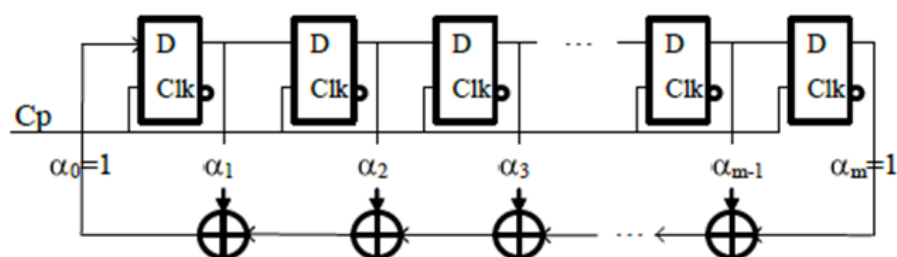


Рис.5.3. Структурные схемы LFSR с внешними сумматорами по модулю два

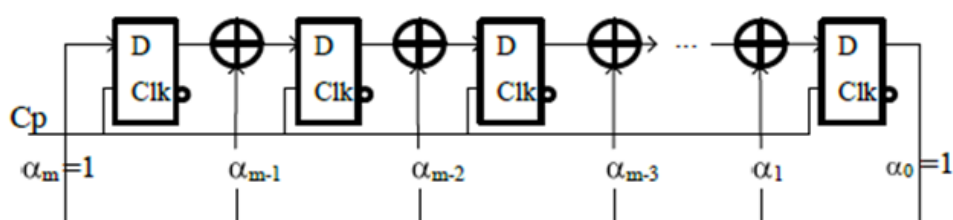
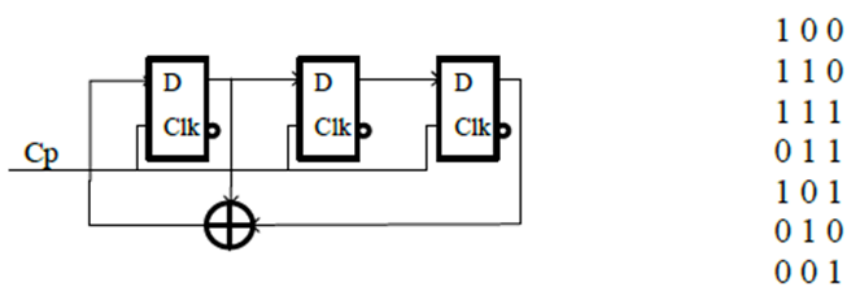
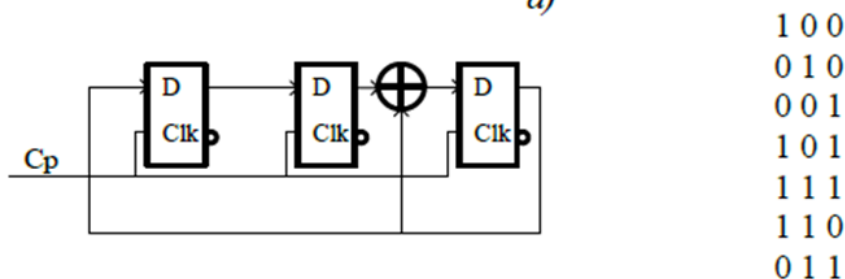


Рис.5.4. Структурные схемы LFSR с внутренними сумматорами по модулю два

Пример 5.1. В качестве примера LFSR с внешними и внутренними сумматорами рассмотрим случай порождающего полинома $\varphi(x)=1+x+x^3$ для которого ниже приведены соответствующие структуры и временные диаграммы формируемых последовательностей.



a)



Алгоритм цифровой подписи DSA

Алгоритм DSA (Digital Signature Algorithm – алгоритм цифровой подписи) был предложен Национальным институтом стандартов и технологий в августе 1991. Данный алгоритм вместе с криптографической хеш-функцией SHA-1 является частью DSS (Digital Signature Standard – стандарт цифровой подписи) – криптографического стандарта электронной цифровой подписи, используемой в США. DSA основан на трудности вычисления дискретных логарифмов и базируется на схеме, первоначально представленной Эль-Гамалем и Шнорром.

Схема цифровой подписи обычно состоит из трех этапов:

1. Этап подготовки, состоящий из различных параметров, необходимых для генерации подписи и проверки, таких как генерация закрытых и открытых ключей, выбор хеш-функции и так далее.
2. Алгоритм подписи (генерации подписи), который, учитывая сообщение, открытый ключ и подпись, либо принимает, либо отклоняет утверждение сообщения о подлинности.

Алгоритм цифровой подписи DSA состоит в следующем. Сначала необходимо получить секретный и открытый ключи, для этого выполнить следующие действия:

1. Выбрать большое простое число q .
2. Выбрать простое число p такое, что q является делителем $(p-1)$.
3. Подобрать число g такое, что для него верно $g = h(p-1)/q \bmod p$, где h – некоторое произвольное число из интервала $(1, p-1)$, и при этом $g > 1$. В

большинстве случаев значение $h = 2$ удовлетворяет этому требованию.

4. Закрытый ключ отправителя x выбирается случайно из интервала $(0, q)$.
5. Открытый ключ вычисляется из закрытого ключа по формуле:

$y = gx \bmod p$. (3.5). Вычислить y по известному x довольно просто (используя алгоритм быстрого возведения в степень). Однако, имея открытый ключ y , вычислительно невозможно определить x , который является дискретным логарифмом y по основанию g . Открытой информацией являются значения p , q и y , закрытой – x . При этом значения p и q могут быть общими для группы пользователей, а значение y и x – для каждого свое.

Подпись сообщения выполняется по следующему алгоритму:

1. Получаем хеш-образ исходного сообщения $h(M)$. При использовании формулы $H_i = (H_{i-1} + M_i)^2 \bmod n$, вычисления необходимо выполнять по модулю числа q .
2. Выбирается случайное число k из $(0, q)$, уникальное для каждой подписи.

3. Вычисляется значение r и s по формулам:

$$r = (g^k \bmod p) \bmod q,$$

$$s = k^{-1} \cdot (h(M) + x \cdot r) \bmod q.$$

4. Если одно из полученных значений r или s будет равно 0, то необходимо повторить вычисления для другого значения k . Иначе, подписью будет пара значений (r, s) . Таким образом сообщение с подписью будет иметь вид $\{M, r, s\}$.

Для того чтобы проверить подлинность подписи, сначала из полученного сообщения $\{M', r, s\}$ вычисляется хеш-образ $h(M')$, после чего находят значение v . Подпись признается подлинной, если $v = r$.

$$w = s^{-1} \bmod q,$$

$$u_1 = h(M) \cdot w \bmod q,$$

$$u_2 = r \cdot w \bmod q,$$

$$v = (g^{u_1} \cdot y^{u_2} \bmod p) \bmod q.$$

Цифровая подпись – это свойство, частное для пользователя или процесса, которое используется для подписи сообщения. Пусть B – получатель сообщения M , подписанного A . Тогда подпись A должна удовлетворять этим требованиям:

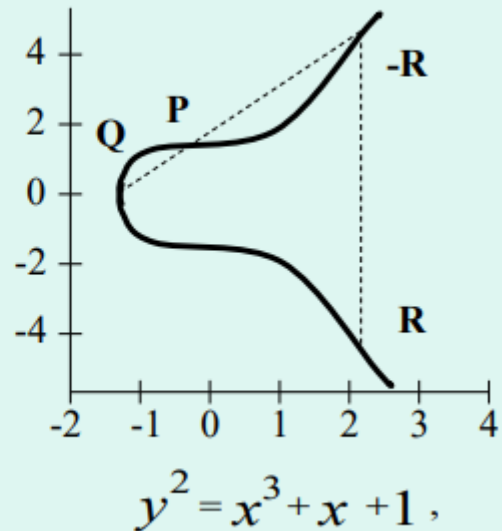
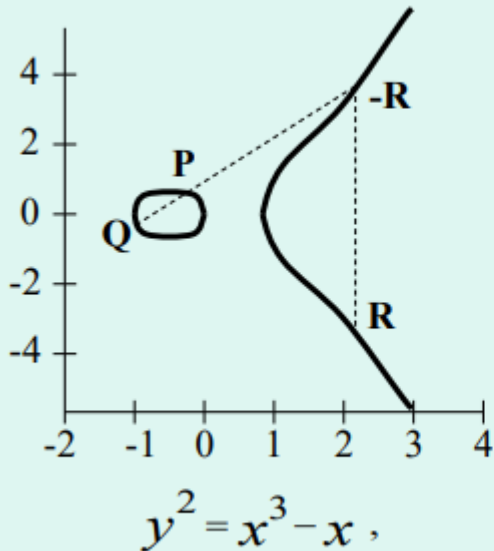
- 1. B должен быть в состоянии подтвердить подпись A на M .*
- 2. Должно быть невозможно, чтобы кто-либо, включая B , подделал подпись A .*
- 3. В случае, если A должен отказаться от подписания сообщения M , у судьи или третьей стороны должна быть возможность разрешить спор, возникающий между A и B .*

Таким образом, цифровая подпись устанавливает подлинность отправителя. Она аналогична обычной письменной подписи. А также устанавливает подлинность данных.

Основные положения эллиптической криптографии. Эллиптические группы

Эллиптическая кривая не означает эллипс. Это всего лишь кривая, описываемая кубическими уравнениями, которые выглядят так, как будто уравнения использовались для описания эллипсов.

$$y^2 + a x y + b y = x^3 + c x^2 + d x + e ,$$



Согласно определению эллиптической кривой, существует следующее свойство: если три точки эллиптической кривой находятся на прямой, то их сумма равна О. Это свойство позволяет определить правила сложения точек на эллиптической кривой.

1. Объект О является нулевым аддитивным элементом, для которого существует равенство $O = -O$, и для любого Р на эллиптической кривой мы имеем $P + O = P$.
2. Прямая, проходящая через Р и $-P$, является вертикальной линией, которая не пересекает эллиптическую кривую в третьей точке; таким образом, точки Р и $-P$ не могут быть добавлены, как ранее. Именно по этой причине группа эллиптических кривых включает точку на бесконечности О. По определению, $P + (-P) = O$ в группе эллиптических кривых.
3. Добавление различных точек Р и Q. Предположим, что Р и Q – это две различные точки на эллиптической кривой, и Р не является $-Q$. Чтобы сложить точки Р и Q, через эти две точки проводится прямая. Эта линия пересечет эллиптическую кривую ровно в одной точке, называемой $-R$. Точка $-R$ отражается на оси x как точка R. Закон сложения в группе эллиптических кривых таков: $P + Q = R$
4. Чтобы добавить точку Р к самой себе, в точке Р проводится касательная к кривой. Если y_P не равно 0, то касательная пересекает эллиптическую кривую ровно в одной точке R. Эта операция называется удвоением точки Р; закон удвоения точки в группе эллиптических кривых определяется следующим образом: $P + P = 2P = R$.
5. Умножение точки Р на целое положительное число k определяется как сумма k копий Р, таким образом, $kP = P + P + P + \dots + P$.

1. Если две точки $P = (x_P, y_P)$ и $Q = (x_Q, y_Q)$ различны и $P \neq -Q$, то $P+Q=R$, где $R=(x_R, y_R)$, а

$$\begin{aligned} s &= (y_P - y_Q) / (x_P - x_Q); \\ x_R &= s^2 - x_P - x_Q; \\ y_R &= -y_P + s(x_P - x_R). \end{aligned}$$

2. Удвоение точки P дает результат $2P = R$

$$\begin{aligned} s &= (3x_P^2 + a) / (2y_P); \\ x_R &= s^2 - 2x_P; \\ y_R &= -y_P + s(x_P - x_R). \end{aligned}$$

БИЛЕТ 2

1. Введение. Криптографическое преобразование информации.
2. Модулярная арифметика.
3. Графическая стеганография. LSB стеганография.
4. Эллиптические кривые. Основные свойства.

Введение. Криптографические преобразования информации.

Целью криптографии является обеспечение конфиденциальности (секретности) и аутентичности (подлинности) передаваемых сообщений.

Криптоанализ — это область знаний, которая занимается исследованием возможностей расшифровывания информации без знания криптографических ключей.

В качестве информации, подлежащей шифрованию и дешифрованию, будут рассматриваться исходные тексты (открытые тексты) и зашифрованные тексты (шифротексты), построенные с использованием некоторых алфавитов.

Криптографическое преобразование — это преобразование информации, основанное на некотором алгоритме, зависящем от изменяемого параметра (обычно называемого секретным ключом), и обладающее свойством невозможности восстановления исходной информации по преобразованной, без знания действующего ключа, с трудоемкостью меньше заранее заданной.

Под этими терминами в дальнейшем будем понимать следующее.

1. Алфавит - конечное множество элементов (знаков) используемых для представления текстов (messages).
2. Исходный текст (Plaintext) - упорядоченный набор из элементов алфавита исходных текстов.
3. Зашифрованный текст (Ciphertext) - упорядоченный набор из элементов алфавита зашифрованных текстов.
4. Шифр – это секретный метод записи, в соответствии с которым исходный текст преобразуется в зашифрованный текст (шифротекст).
5. Шифрование (Enciphering) – это процесс преобразования исходного текста в шифротекст.
6. Дешифрование (Deciphering) – это обратный процесс по отношению к шифрованию, в результате которого шифротекст преобразуется в исходный текст.

Шифрование и дешифрование зависят от криптографического ключа или ключей, количество которых зависит от класса используемой криптосистемы. Различают два основных класса криптосистем:

- симметричные одноключевые криптосистемы (с секретным ключом);
- асимметричные двухключевые криптосистемы (с открытым ключом).

Криптология разделяется на два направления - *криптографию* и *криптоанализ*. Цели этих направлений прямо противоположные. *Криптография* – занимается поиском и исследованием методов преобразования информации с целью ее шифрования. В криптографии различают два основных преобразования — это непосредственно *шифрование* и обратный ему процесс *дешифрование*.



Рис.1.1. Процедура шифрования и дешифрования

Модулярная арифметика

Два целых числа a и b *сравнимы (конгруэнтны) по модулю* натурального числа m , если при делении на m они дают одинаковые остатки. Другими словами, a и b *сравнимы по модулю m* , если их разность $a - b$ делится на m . Например, 32 и 39 сравнимы по модулю 7, так как $32 = 7 \cdot 4 + 4$, $39 = 7 \cdot 5 + 4$. Утверждение a и b сравнимы по модулю m записывается в виде:

$$a \equiv b \pmod{m}.$$

Последнее выражение называется *сравнением (конгруэнцией)*.

Пример 3.11. Следующие равенства являются сравнениями $32 \equiv 5 \pmod{9}$; $48 \equiv 12 \pmod{9}$; $17 \equiv 7 \pmod{5}$. Числа a и b *несравнимы по модулю m* , если их разность $a - b$ не делится на m . Этот факт обозначается неравенством $a \not\equiv b \pmod{m}$. Очень часто для представления сравнения $a \equiv b \pmod{m}$ используют символ равенства ($=$), тогда $a = b \pmod{m}$ означает, что a и b сравнимы по модулю m , а $a \neq b \pmod{m}$, что не сравнимы по модулю m . В случае, когда в сравнении $a = b \pmod{m}$ величина $b < m$, b называется *вычетом a по модулю m* . Для сравнений существует несколько полезных лемм:

Лемма 3.1. Если $a \equiv b \pmod{m}$, тогда для любого целого k будет справедливо $ka \equiv kb \pmod{m}$.

Лемма 3.2. Если $ka \equiv kb \pmod{m}$, и $(k, m) = 1$, то $a \equiv b \pmod{m}$.

Лемма 3.3. Если $ka \equiv kb \pmod{km}$, где k и m любые целые числа, то $a \equiv b \pmod{m}$.

Лемма 3.4. Если $a \equiv b \pmod{m}$, и $c \equiv d \pmod{m}$, то $a + c \equiv b + d \pmod{m}$.

Лемма 3.5. Если $a_1 \equiv b_1 \pmod{m}$, и $a_2 \equiv b_2 \pmod{m}, \dots, a_n \equiv b_n \pmod{m}$, то $a_1 + a_2 + a_3 + \dots + a_n \equiv b_1 + b_2 + b_3 + \dots + b_n \pmod{m}$.

Лемма 3.6. Если $a \equiv b \pmod{m}$, и $c \equiv d \pmod{m}$, то $a \cdot c \equiv b \cdot d \pmod{m}$.

Лемма 3.7. Если $a_1 \equiv b_1 \pmod{m}$, и $a_2 \equiv b_2 \pmod{m}, \dots, a_n \equiv b_n \pmod{m}$, то $a_1 \cdot a_2 \cdot a_3 \cdot \dots \cdot a_n \equiv b_1 \cdot b_2 \cdot b_3 \cdot \dots \cdot b_n \pmod{m}$.

Лемма 3.8. Если $a \equiv b \pmod{m}$, то для любого целого $k > 0$ будет справедливо $ak \equiv bk \pmod{m}$. Теория сравнений эффективно используется для выполнения различных вычислений при реализации криптографических алгоритмов.

Модулярная арифметика.

Основой для реализации вычислительных процедур является так называемая *модулярная арифметика*. Модулярная арифметика основана на следующей классической лемме, используемой во многих технических приложениях. 50

Лемма 3.9. $(a * b) \pmod{m} = [(a \pmod{m}) * (b \pmod{m})] \pmod{m}$, где $*$ это любая из следующих операций “+”, “-” или “x”.

Приведенная лемма показывает, что вычисление $(a * b) \pmod{m}$ в модульной арифметике даёт тот же результат, что и в случае обычной целочисленной арифметике, однако существенно упрощает вычисление результата $(a * b) \pmod{m}$.

Пример 3.12. $7 \cdot 9 \pmod{5} = [(7 \pmod{5}) \cdot (9 \pmod{5})] \pmod{5} = 3$.

Заметим, что принцип модульной арифметики также применяется и для возведения в степень, так как операция возведения в степень равносильна повторяющемуся выполнению операции умножения.

Графическая стеганография. LSB стеганография

Стеганография - двойная криптография, в которой факт того, что информация передаётся тоже засекречен.

Стеганография — метод сокрытия информации, при котором ее помещают в некий объект так, чтобы третьи лица не могли ее обнаружить. В отличие от криптографии, стеганография направлена не на защиту данных от прочтения и изменения, а на то, чтобы скрыть сам факт их существования.

Графическая стеганография - скрытая передача информации через графические файлы путём сохранения в тайне самого факта передачи. LSB (наименьший значащий бит) — суть этого метода заключается в замене последних значащих битов в контейнере (изображении) на биты скрываемого сообщения. Разбивается байт секретного сообщения на 4 двухбитовые части, и заменяются полученными фрагментами младшие биты изображения. Такая замена в общем случае не заметна человеческому глазу. Более того, многие старые устройства вывода, даже не смогут отобразить такие незначительные перемены.

Для формата RGB каждый пиксель должен быть представлен 3 байтами (красный, зеленый, синий) 24 бит. Эксперименты показывают до 50% биты имеют случайный характер, и могут быть заменены.

Почитать - <https://habr.com/ru/post/422593/>

Эллиптические кривые. Основные свойства

Эллиптические кривые (Elliptic Curve) в общем случае описываются кубическими уравнениями вида $y^2+axy+by=x^3+cx^2+dx+e$, где a, b, c, d и e представляют собой действительные числа, которые удовлетворяют достаточно простым требованиям.

1.Элемент O является нулевым аддитивным элементом, для которого выполняется равенство $O=-O$. Тогда соответственно для любой точки P эллиптической кривой получим $P+O=P$.

2.Прямая линия, проведенная через точки P и $-P$ является вертикальной линией. Тогда согласно определению $P+(-P)=O$.

3.Для P, Q и $P - Q$ результатом сложения двух точек будет являться третья точка, принадлежащая эллиптической кривой, то есть $P+Q=R$.

4.В случае, когда $P = Q$, то есть когда точка P складывается сама с собой и $P(x,0)$, в точке P проводится касательная линия к эллиптической кривой. Эта линия пересечет эллиптическую кривую только в одной точке $-R$. Тогда результатом удвоения точки P будет точка R симметричная точке $-R$. Таким образом $P+P=2P=R$. 133

5.Для случая, когда $P = (x,0)$, касательная к эллиптической кривой в этой точке будет представлять собой вертикальную линию. Тогда, согласно определению, для такой точки $2P = O$. Для случая вычисления $3P$ будем иметь $2P + P$. Выполнив подстановку вместо $2P$ нулевого элемента O , окончательно получим $P + O = P$. Таким образом, $3P = P$. Для $P = (x,0)$ будут выполняться равенства $3P = P, 4P = O, 5P = P, 6P = O, 7P = P$, и так далее.

6.Операция умножения точки P на положительное целое число k определяется как сумма k точек P , так что $kP=P+P+P+\dots+P$. Несмотря на то, что геометрическая интерпретация операции сложения точек эллиптических кривых является достаточно наглядной иллюстрацией арифметики эллиптических кривых, он не может быть использован для практической реализации вычислений

Основополагающим свойством эллиптических кривых является следующее свойство. Секущая прямая линия всегда пересекает эллиптическую кривую только в трех точках. Исключением является случай вертикальной секущей прямой, для которой третьей точкой является бесконечный элемент O .

1.Если P и Q являются различными точками эллиптической кривой и P не равняется $-Q$, тогда результатом операции сложения $P+Q=R$ является точка $R=(x_R, y_R)$, координаты которой вычисляются по следующим соотношениям $s=(y_P-y_Q)/(x_P-x_Q)$; $x_R=s^2-x_P-x_Q$; $y_R=-y_P+s(x_P-x_R)$.

2. Удвоение точки P будет осуществляться подобным образом $s=(3x_P^2+a)/(2y_P)$; $x_R=s^2-2x_P$; $y_R=-y_P+s(x_P-x_R)$.

Величина a , используемая в последнем выражении является одним из параметров эллиптической кривой описываемой уравнением $y^2=x^3+ax+b$.

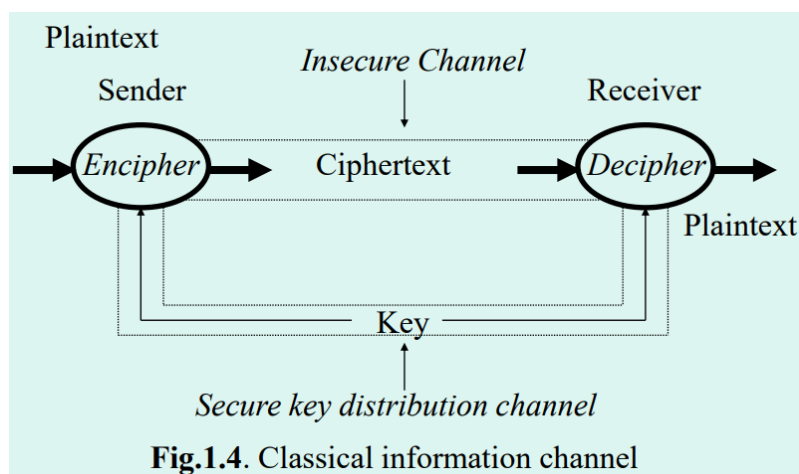
БИЛЕТ 3

1. Классический информационный канал. Формальное описание
2. ВPCS стеганография
3. Комбинированные генераторы ключа. Генератор Stop and Go
4. Потокковая криптосистема RC 4

Классический информационный канал. Формальное описание.

Современная криптография защищает данные, переданные по сверхскоростным электронным линиям или хранящимся в компьютерных системах. Существует две основные цели, которые должны достигаться в результате использования криптографических систем:

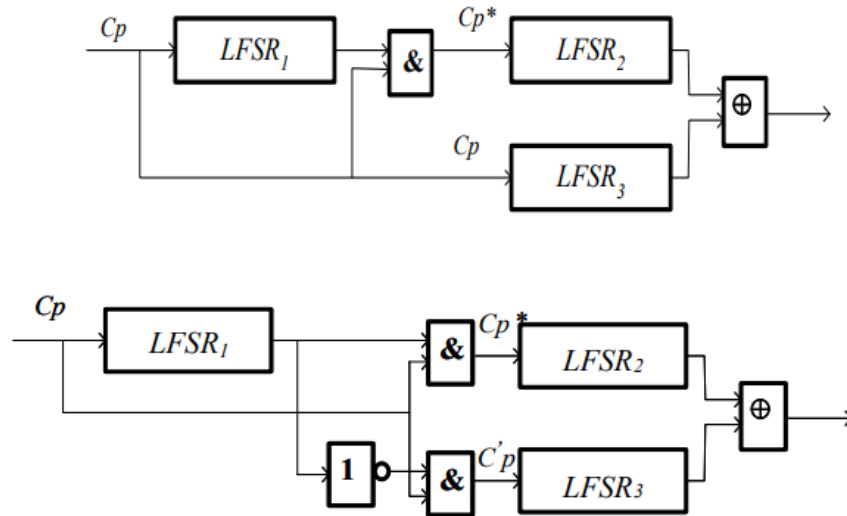
- Скрытность (или секретность) – предотвращение несанкционированного доступа к данным
- Достоверность (или целостность) – предотвращение несанкционированного изменения данных.



Информация, передаваемая по открытому каналу, подвержена пассивному перехвату, что угрожает секретности, и активному перехвату, что угрожает достоверности. Пассивный перехват приводит к перехвату сообщений, обычно без их изменения, активный перехват приводит к умышленному изменению потока данных.

ВPCS стеганография

Комбинированные генераторы ключа. Генератор Stop and Go



Работает синхронно.

Про комбинированные генераторы ключа хз(

Потоковая криптосистема RC 4

Использование S-массива, состоящего из 256 байт (все значения уникальны)

Два индекса, которые указывают на 2 элемента S-массива

Алгоритм инициализации S-массива

1. Значение элемента массива соответствует номеру индекса элемента.
2. Используется ключ (40-256 бит)

$i, j = 0 \Rightarrow j = [j + S[i] + \text{key}[i \bmod \text{Length}(\text{key})]] \bmod 256$

3. Поменять местами $S[i]$ и $S[j]$
4. Увеличить i
5. Начиная с п2 цикл на 256 итераций

Псевдокод

```

for i from 0 to 255
    S[i] := i
end for
j := 0
for i from 0 to 255
    j := [j + S[i] + key[i mod keylength]] mod 256
    swap values of S[i] and S[j]
end for
    
```

Псевдорандомное генерирование ключа, алгоритм:

1. $i, j = 0$
2. Вычисление новых значений i и j

$$i = (i + 1) \bmod 256$$

$$j = (j + S[j]) \bmod 256$$
3. Меняем местами $S[i]$ и $S[j]$
4. Вычисление того бита, который подойдет в ключ (индекс в S-массиве).

$$(S[i] + S[j]) \bmod 256$$

Псевдокод:

algorithm:

$i := 0$

$j := 0$

while Generating Output:

$i := (i + 1) \bmod 256$

$j := (j + S[i]) \bmod 256$

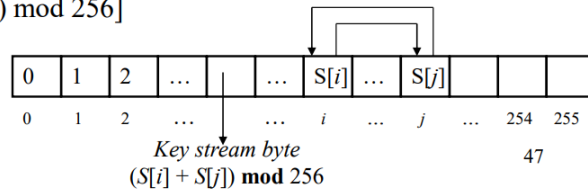
swap values of $S[i]$ and $S[j]$

$K := S[(S[i] + S[j]) \bmod 256]$

Output K

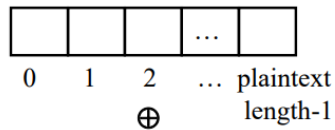
end while

S

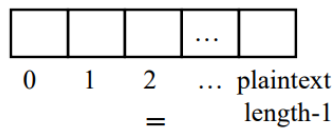


Как только был сгенерирован поток ключей алгоритм (де)шифрования состоит из операции XOR каждого символа исходного текста с потоком ключей.

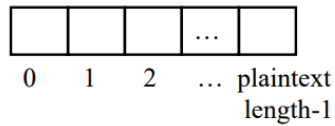
Plaintext



Key stream



Ciphertext



БИЛЕТ 4

1. Требования к криптографической системе. Требования секретности и достоверности. Классификация криптосистемы.
2. Лексическая стеганография
3. Комбинированные генераторы ключа. Каскадный генератор Gollmann
4. Потокоская криптосистема A5/1

Требования к криптографической системе. Требования секретности и достоверности. Классификация криптосистемы.

Любая криптосистема описывается 5 компонентами:

1. Пространство исходных текстов M (как правило, алфавиты)
2. Пространство шифротекстов C (как правило, алфавитом шифров)
3. Пространство ключей k (мощность ключа == количество)
4. Алгоритм шифрования $E_k: M \rightarrow C$
5. Алгоритм дешифрования $D_k: C \rightarrow M$

Криптосистемы должны удовлетворять следующим **требованиям**:

1. Легко использовать криптосистему (и реализовывать ее: по критериям времени, ограничения и пространственным реалиям, удобство пользования)
2. Процедуры (де)шифрования должны быть эффективны для всех допустимых ключей из алфавита
3. Секретность криптосистемы зависит только от секретности ключа и не зависит от секретности алгоритма (де)шифрования.

Требования для **секретности**:

1. Для криптоаналитика должно быть невозможным определение алгоритма дешифрования на основании шифра C даже в том случае, если исходный текст M известен
2. Для криптоаналитика должно быть аналитически невозможным получить на основании зашифрованного текста C исходный текст M

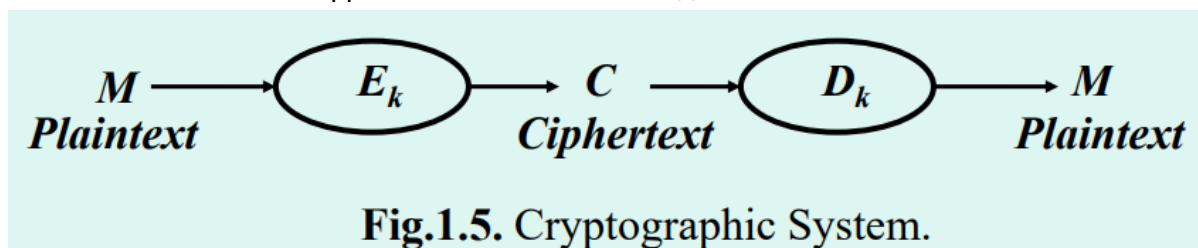


Fig.1.5. Cryptographic System.

Требования для **достоверности**:

1. Для криптоаналитика должно быть невозможно получение алгоритма E_k из шифротекста C , если известен исходный текст M
2. Для криптоаналитика аналитически невозможно нахождение шифротекста C' такого, что его дешифрование будет правильным для исходного текста M

Классификация Симонса

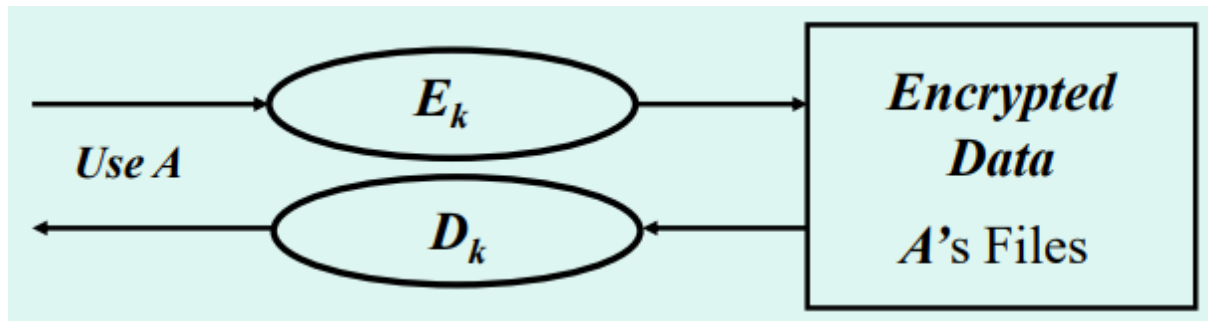
1. Симметричные криптосистемы (криптосистемы с одним ключом)
2. Ассиметричные криптосистемы (двухключевые)

Симметричные криптосистемы

В симметричных криптосистемах или криптосистемах с одним ключом ключ шифрования и ключ дешифрования одинаковы (или легко определяются друг из друга).

Это означает, что преобразования E и D также легко выводятся друг из друга. До недавнего времени все криптосистемы были системами только с одним ключом. Существуют также системы, которые обычно называют традиционными (или классическими) системами.

Системы с одним ключом обеспечивают превосходный способ шифрования личных файлов пользователя. У каждого пользователя A есть личные преобразования E и D для шифрования и расшифровки файлов.



Ассиметричные криптосистемы.

Ассиметричная криптография с открытым ключом/двумя ключами предполагает использование двух ключей:

- открытый ключ, который может быть известен любому и может использоваться для шифрования сообщений и проверки подписей.
- закрытый ключ, известный только получателю, используемый для расшифровки сообщений и подписания (создания) подписей.

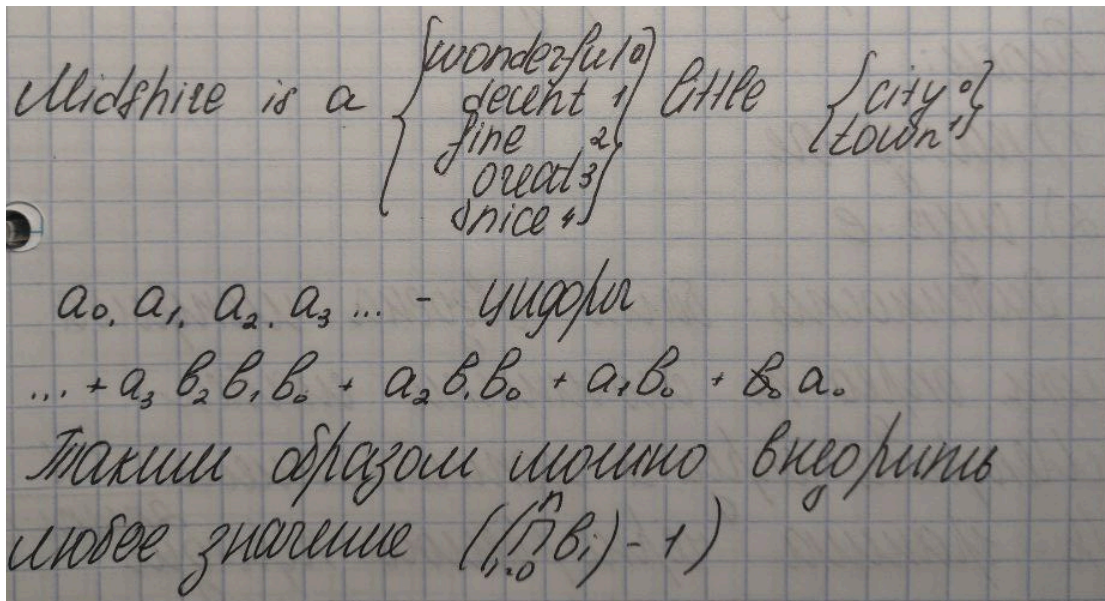
Такая криптосистема является ассиметричной, поскольку те, кто шифрует сообщения или проверяет подписи, не могут расшифровать сообщения или создать подписи.

Ключи математически связаны, один из самых популярных алгоритмов генерации таких ключей – RSA.

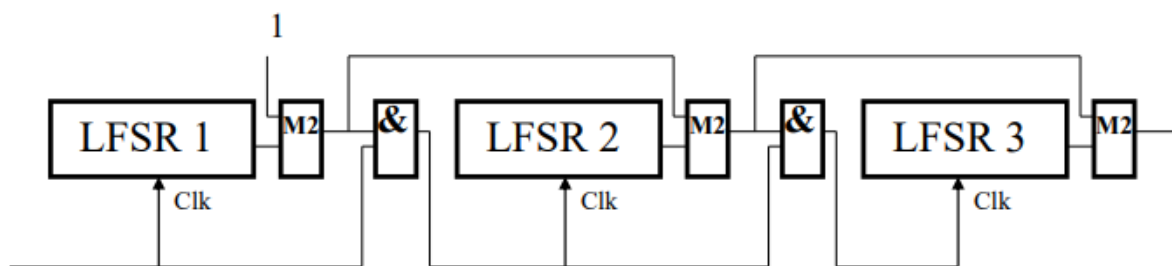
Лексическая стеганография

Основана на использовании синонимов (“propensity”, “predilection”, “penchant”, “proclivity”).

Чаще всего для реализации лексических стеганографических приложений используется метод, основанный на применении систем счисления со смешанными весами.



Комбинированные генераторы ключа. Каскадный генератор Gollmann



Длина последовательности: $(2^{m_1} - 1)(2^{m_2} - 1)(2^{m_3} - 1)$

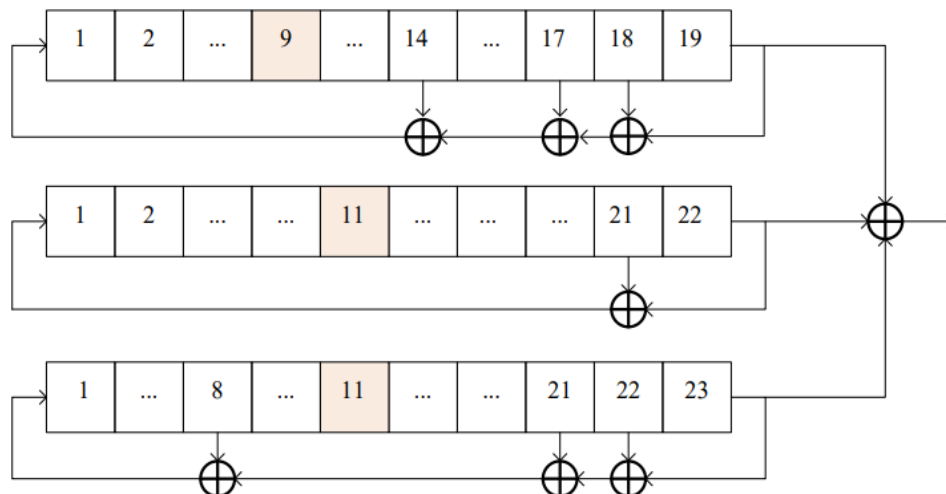
Потоковая криптосистема A5/1

Основа – 3 LFSR-а

Порождающий полином: $\varphi(x) = 1 + x^{21} + x^{22}$

LFSR number	Length in bits	Feedback polynomial	Clocking bit	Tapped bits
1	19	$x^{19} + x^{18} + x^{17} + x^{14} + 1$	9	14, 17, 18, 19
2	22	$x^{22} + x^{21} + 1$	11	21, 22
3	23	$x^{23} + x^{22} + x^{21} + x^8 + 1$	11	8, 21, 22, 23

Синхронизация регистров осуществляется в режиме stop/go с использованием правила большинства. Каждому регистру соответствует бит синхронизации. В каждом цикле проверяется бит синхронизации всех трех регистров и определяется бит большинства. Регистр синхронизируется, если бит синхронизации совпадает с основным битом. Следовательно, на каждом шаге синхронизируются два или три регистра, и каждый регистр останавливается с вероятностью 3/4.



Из конспекта.

9 и 11 участвуют в реализации Stop and Go

g	n	n	g
0	0	0	1 2 3
0	0	1	1 2
0	1	0	1 3
0	1	1	2 3
1	0	0	2 3
1	0	1	1 3
1	1	0	1 2
1	1	1	1 2 3

Ключ сеансовый.

БИЛЕТ 5

1. Криптосистемы с открытым ключом. Характеристика подобных систем.
2. Алгоритм цифровой подписи ElGamal
3. Комбинированные генераторы ключа. Пороговый генератор.
4. Открытая система распределения ключей (Алгоритм Diffie, Hellman)

Криптосистемы с открытым ключом. Характеристика подобных систем

От классических симметричных алгоритмов они отличаются тем, что для шифрования данных используется один ключ, обычно его называют открытым или публичный ключ, а для дешифрования – другой, секретный или закрытый, ключ.

Диффи и Хеллман, которые впервые предложили и описали криптосистему с открытым ключом, выявляют следующие требования к криптосистемам:

1. Вычислительно легко создавать пару открытый ключ (K_o), закрытый ключ (K_c).
2. Вычислительно легко, имея открытый ключ и незашифрованное сообщение M , создать соответствующее зашифрованное сообщение: $C = E_{K_o}[M]$.
3. Вычислительно легко дешифровать сообщение, используя закрытый ключ: $M = D_{K_c}[C] = D_{K_c}[E_{K_o}[M]]$.
4. Вычислительно невозможно, зная открытый ключ K_o , определить закрытый ключ K_c .
5. Вычислительно невозможно, зная открытый ключ K_o и зашифрованное сообщение C , восстановить исходное сообщение M .

Можно добавить шестое требование, хотя оно не выполняется для всех алгоритмов с открытым ключом:

6. Шифрующие и дешифрующие функции могут применяться в любом порядке, т.е. $M = E_{K_o}[D_{K_c}[M]] = D_{K_c}[E_{K_o}[M]]$.

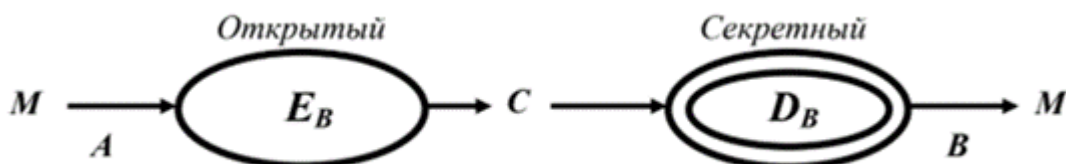
Таким образом, данные, зашифрованные открытым ключом, можно расшифровать только секретным ключом. Следовательно, открытый ключ может распространяться через обычные коммуникационные сети и другие открытые каналы, что устраняет главный недостаток стандартных криптографических алгоритмов: необходимость использовать специальные каналы связи для распределения ключей.

В системах с открытым ключом каждый пользователь (A) имеет открытое кодирующее преобразование E_A , которое может храниться в открытой базе данных, и секретное декодирующее преобразование D_A , известное только этому пользователю. Секретное преобразование D_A описывается секретным ключом, а открытое преобразование E_A – открытым ключом, полученным из секретного преобразования D_A при помощи односторонних вычислений. Определяющим требованием к вычислению E_A из D_A является требование вычислительной невозможности определения D_A из E_A или преобразования эквивалентного D_A .

В таких системах секретность и достоверность обеспечиваются независимыми преобразованиями. Предположим, пользователь A хочет послать сообщение M другому пользователю B . Если A знает открытое преобразование E_B пользователя B ,

то А может секретно передать М пользователю В путём пересылки шифротекста $C=EB(M)$. При получении шифротекста С пользователь В декодирует С используя своё секретное преобразование DB и в результате получает $DB(C)=DB(EB(M))=M$.

Передаваемая информация является секретной, однако при полном отсутствии достоверности (целостности). В качестве пользователя А может выступать абсолютно любой пользователь в том числе и злоумышленник, который знает открытый ключ пользователя В.



Для достижения достоверности, М должно быть преобразовано пользователем А с помощью собственного секретного преобразования DA . Игнорируя секретность, А посылает шифрограмму $C=DA(M)$ пользователю В. По получении шифрограммы, пользователь В использует открытое преобразование EA отправителя А и в результате преобразования получает $EA(C)=EA(DA(M))=M$. В данном случае достигается достоверность (целостность) информации при полном отсутствии ее секретности. Любой из пользователей, в том числе и злоумышленник, может дешифровать и прочитать передаваемое сообщение.

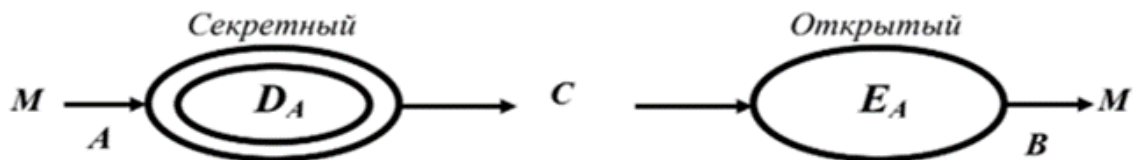


Рис.1.9. Достоверность в системах с открытым ключом

Чтобы достичь одновременно и секретности, и достоверности, отправитель и получатель должны применить два этапа преобразования. Отправитель А генерирует шифротекст $C=EB(DA(M))$, а В восстанавливает М согласно правилу: $EA(DB(C))=EA(DB(EB(DA(M))))=EA(DA(M))=M$.

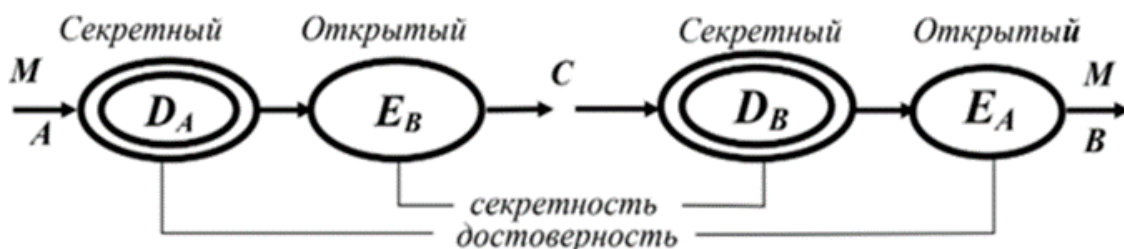


Рис.1.10. Секретность и достоверность в системах с открытым ключом

Алгоритм цифровой подписи ElGamal

Цифровая подпись ЭльГамала - подпись, основанная на сложности вычисления дискретного логарифма. Была описана в 1984 году Taher ElGamal.

1. Стадия подготовки:

1. Выбирается хеш-функция $H(M)$ для сжатия сообщения.
2. Выбирается большое простое число P такое, что невозможно вычислить дискретный логарифм по модулю P .
3. Случайно выбирается $a < P$, где a - примитивный элемент (генератор) мультипликативной группы целых чисел по модулю P . Все эти параметры распространяются среди участников подписи.
4. Выбирается случайное число $1 < X < P - 1$ - секретный ключ.
5. Вычисляется $Y = a^x \bmod P$.

Тогда открытым ключом является тройка значений (P, a, Y) .
И секретным ключом - Число X .

2. Генерация подписи:

Чтобы подписать сообщение M необходимо выполнить следующие шаги:

1. Вычислить хеш значение $m = H(M)$.
2. Выбрать случайное K такое, что $0 < k < P - 1$ и $(K, P - 1) = 1$.
3. Посчитать $W = a^K \bmod P$.
4. Посчитать $KV = (m - XW) \bmod (P - 1)$. Если $V = 0$, начать всё сначала. P.s. K находится с помощью расширенного алгоритма Евклида. [MAXimal :: algo :: Расширенный алгоритм Евклида \(e-maxx.ru\)](#) (хороший сайт) Пара (W, V) будет являться цифровой подписью сообщения M . Для каждой подписи необходимо повторить вышеперечисленные шаги.

3. Верификация подписи

Чтобы проверить подлинность подписи, получатель берет исходное сообщение M и саму подпись (W, V) . Подпись верифицируется следующим образом:

1. Вычисляется $m = H(X)$.
2. Сравниваются значения $0 < W < P$ и $0 < V < P - 1$
3. Проверить равенство

Комбинированные генераторы ключа. Пороговый генератор

Большое количество *LFSR* использует так называемый *пороговый генератор* (*Threshold generator*), приведенный на рис.5.13. Количество (m) *LFSR* выбирается нечётным, тогда нелинейная функция F формирует выходное значение 0 или 1 в зависимости от соотношения единичных и нулевых значений на выходах генераторов M последовательностей. Если количество единичных значений больше нулевых на выходе генератора формируется единичное значение, в противном случае выходное значение равняется нулю.

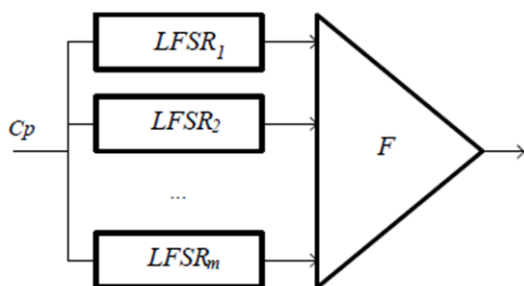


Рис.5.13. Пороговый генератор

Подобно, как и в предыдущих случаях, периоды M последовательностей должны быть взаимно простыми, а степени порождающих их полиномов иметь сравнимые величины и, как правило, больше 10.

Открытая система распределения ключей (Алгоритм Diffie, Hellman)

Открытая *система распределения ключей* (*public key distribution algorithm*), предложенная *W.Diffie* и *M.Hellman*. Авторы предложили алгоритм, основанный на возведении в степень по модулю q , где q является простым числом. Стойкость предложенного алгоритма основывается на сложности вычисления дискретного логарифма над полем Галуа $GF(q)$ с $q-1$ элементами $\{1, 2, \dots, q-1\}$. Рассмотрим пару взаимно инверсных функций: где $0 < M$, $C < q$, соответственно, q простое целое число, а примитивный элемент (корень) $GF(q)$. Вычисление C из M осуществляется простым возведением в степень, что вычислительно не составляет труда. В тоже время вычисление дискретного логарифма над $GF(q)$ процедура намного более сложная и требует для своего решения больших вычислительных затрат. Для больших значений q эта задача вычислительно неразрешима. На этом факте и основан рассматриваемый алгоритм, как, впрочем, и большинство из известных криптосистем с открытым ключом. Сущность открытой системы распределения ключей состоит в том, что два пользователя используя открытый канал, обмениваются криптографическим ключом, который будет известен только им, двоим. Злоумышленник при попытке получения значения ключа столкнется с проблемой дискретного логарифма над полем $GF(q)$. Предварительно оба пользователя A и B обмениваются параметрами $(, q)$ конечного поля, где является примитивным элементом поля, а модуль q представляет собой большое простое число. Эта информация является открытой для всех. Далее последовательно выполняются следующие действия.

1. Каждый из двух пользователей A и B генерирует независимое случайное число. Пользователь A число MA , а пользователь B число MB равновероятно выбранное из набора целых чисел $\{1, 2, \dots, q-1\}$, и каждый из них хранит свое число в секрете.

2. Оба пользователя осуществляют следующие вычисления: A вычисляет, а B вычисляет.

3. Затем по открытому каналу A посылает в адрес B результат своего вычисления CA , а B значение CB . Отметим невозможность восстановления величин MA , и MB на основании доступных для злоумышленника значений CA , и CB даже если параметры конечного поля $(, q)$ ему известны.

4. Пользователь A вычисляет KA , получая CB из открытого файла и выполняя вычисления.

$$K_A = C_B^{M_A} \bmod q = (\alpha^{M_B})^{M_A} \bmod q = \alpha^{M_B M_A} \bmod q$$

пользователь B вычисляет общий ключ K_B в такой же форме

$$K_B = C_A^{M_B} \bmod q = (\alpha^{M_A})^{M_B} \bmod q = \alpha^{M_A M_B} \bmod q$$

Таким образом, оба пользователя A и B обменялись секретным ключом $K_A=K_B$, что следует из приведенных выше соотношений. Отметим, что значения M_A , и M_B являются аналогами секретного ключа пользователей A и B , а C_A , и C_B аналогами их открытых ключей. Как результат приведенных действий с секретными и открытыми ключами пользователей оба они получают криптографический ключ $K=K_A=K_B$, который может быть использован ими для секретного общения используя, например DES.

БИЛЕТ 6

1. Алгоритмы шифрования: Перестановочные шифры.
2. Способы решения линейных сравнений: 1 случай (нет решений).
3. Самосинхронизирующиеся потоковые криптосистемы.
4. Эллиптические группы. Порядок точки эллиптической группы.

Алгоритмы шифрования. Перестановочные шифры

Простейшими примерами шифров с транспозицией являются:

- Объединение слов (удаление пробелов)
- Разделение символов на блоки
- Обратное написание слов (this -> siht)

Перестановочный шифр переставляет символы в соответствии с некоторой схемой. Классически такая перестановка осуществлялась с помощью какой-либо геометрической фигуры. Процедура шифрования состоит из двух этапов и описывается диаграммой:

Исходный текст ==> Геометрическая фигура =====> Шифротекст

Правило записи

Правило чтения

1. Сначала исходный текст записывается в виде фигуры в соответствии с каким-либо правилом записи
2. Затем шифротекст извлекается из фигуры согласно какому-либо правилу чтения

Возьмем пример с прямоугольником 3x4 и словом CRYPTOGRAPHY

<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
C	R	Y	P
T	O	G	R
A	P	H	Y

Если снимать в порядке 3-1-4-2, то будет: YGHCTAPRYROP

Как правило все современные системы строятся по блочному принципу – деление на блоки фиксированной размерности d . Ключ для шифра задается парой $K=(d,f)$, где d - количество символов в блоке, а f функция перестановки. Таким образом, сообщение с открытым текстом:

$$M = m_1, \dots, m_d, m_{d+1}, \dots, m_{2d}, \dots$$

Шифруется как

$$E_K(M) = m_{f(1)}, \dots, m_{f(d)}, m_{f(d+1)}, \dots, m_{f(2d)}, \dots$$

Дешифрация происходит в обратном порядке

Пример:

i	1	2	3	4
$f(i)$	3	1	4	2

M=CRYPTOGRAPHY
 $E_k(M)$ =YCPRGTHAYP

Сеансовый ключ – ключевое слово, которое совместно с долгосрочным ключом формирует алгоритмы шифрования.

Используя ключевое слово, например CONVENIENCE присваиваем каждой букве в слове номер в соответствии с алфавитным порядком и если одна и та же буква встречается дважды, то по позиции в слове.

Пример:

Исходный текст – M=HERE IS A SECRET MESSAGE ENCIPHERED BY TRANSPOSITION

C	O	N	V	E	N	I	E	N	C	E
1	10	7	11	3	8	6	4	9	2	5
H	E	R	E	I	S	A	S	E	C	R
E	T	M	E	S	S	A	G	E	E	N
C	I	P	H	E	R	E	D	B	Y	T
R	A	N	S	P	O	S	I	T	I	O
N										

Шифротекст – C=HECRN CEYI ISEP SGDI RNTD AAES RMPN SSRO EEBT ETIA EEHS

Способы решения линейных сравнений: 1 случай (нет решений)

Под линейным сравнением (линейной конгруэнцией) понимают выражение вида:

$$ax = b \pmod n, b < n$$

где a , b и n есть целые числа, и $b < n$, а x ($x < n$) неизвестное целое число, удовлетворяющее линейному сравнению.

Существует три возможных случая для линейного сравнения относительно его решения x , а именно линейное сравнение может:

- не иметь решений;
- иметь одно решение;
- иметь множество решений, удовлетворяющих этому линейному сравнению.

Теорема: Если $(a, n) = d$ ($\text{НОД} = d$) и d не является делителем b , то не существует решений линейного сравнения $ax = b \pmod n$.

Доказательство: Предположим, что существует решение x_0 , удовлетворяющее линейному сравнению $ax_0 = b \pmod n$. По условию теоремы, d является делителем a и n , что означает, что d должно быть делителем ax_0 и nq , также как $ax_0 - nq = b$. Из этого следует противоречие – по условию теоремы d не является делителем b , а в случае наличия решения x_0 d должно является делителем b , значит, линейное сравнение $ax = b \pmod n$ не имеет решений, если $d = (a, n)$ не является делителем b .

Пример: Линейное сравнение $2x = 1 \pmod 4$ не имеет решения, так как наибольший общий делитель $d=2$ чисел $a=2$ и $n=4$ ($d = (a, n)$) не является делителем $b=1$.

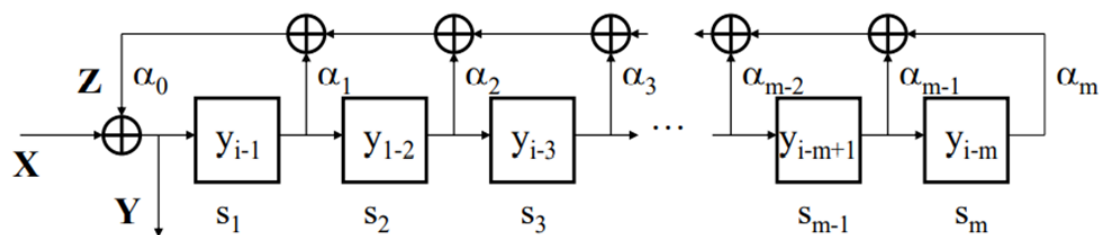
Самосинхронизирующиеся потоковые криптосистемы

Различают два типа потоковых шифров синхронные (synchronous) и самосинхронизирующиеся (self-synchronizing). В первом случае последовательность значений ключа является независимой от открытого текста и шифротекста. В самосинхронизирующихся смысл в том, что значение очередного бита ключа будет определяться m битами шифротекста или исходного текста.

Подразделяются на шифры с автоключом для шифротекста и шифры с автоключом для открытого текста.

Первый случай – на базе шифротекста шифратор и дешифратор:

Схема шифратора:



$$\varphi(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \dots + \alpha_{m-1} x^{m-1} + \alpha_m x^m; \quad \alpha_m = \alpha_0 = 1; \quad \alpha_i \in \{0, 1\}; \quad Z -$$

ключевой поток

X – исходный текст

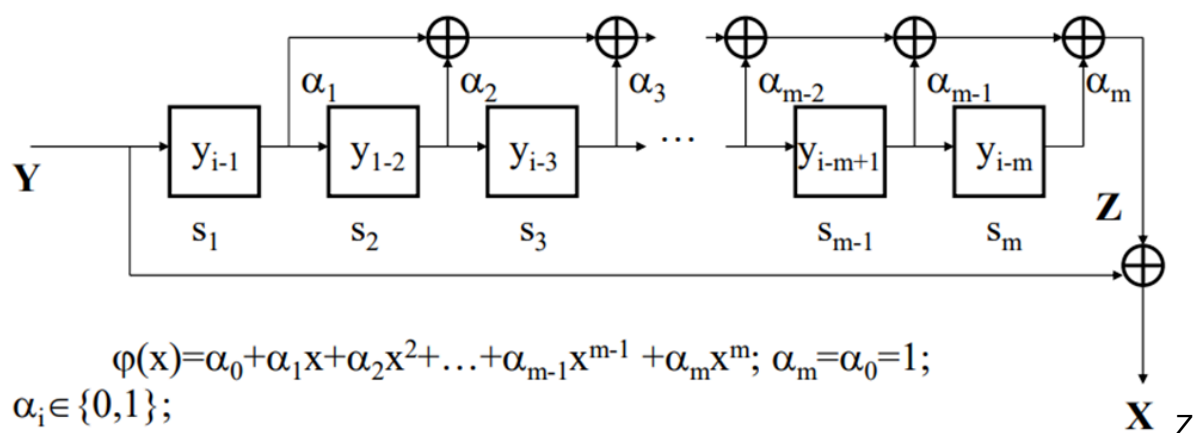
Y – шифротекст

S – начальное состояние на основании ключа

Аналитическая схема такого шифратора задается:

$$y_i = \begin{cases} x_i + \sum_{j=1}^i \alpha_j y_{i-j} + \sum_{j=i+1}^m \alpha_j s_{j-i}; & 0 \leq i \leq m-1; \\ x_i + \sum_{j=1}^m \alpha_j y_{i-j}; & i \geq m. \end{cases}$$

Схема дешифратора:



$$\varphi(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \dots + \alpha_{m-1} x^{m-1} + \alpha_m x^m; \alpha_m = \alpha_0 = 1; \alpha_i \in \{0, 1\};$$

– ключевой поток

X – исходный текст

Y – шифротекст

S – начальное состояние на основании ключа

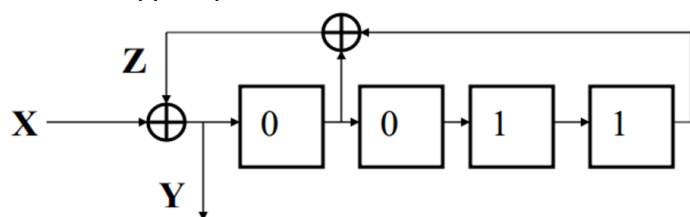
Аналитическая схема такого дешифратора задается:

$$x_i = \begin{cases} y_i + \sum_{j=1}^i \alpha_j y_{i-j} + \sum_{j=i+1}^m \alpha_j s_{j-i}; & 0 \leq i \leq m-1; \\ y_i + \sum_{j=1}^m \alpha_j y_{i-j}; & i \geq m. \end{cases}$$

Пример шифрации:

$$\varphi(x) = 1 + x + x^4; X = 11101001; S = (s_1 s_2 s_3 s_4) = 0011.$$

Схема шифратора

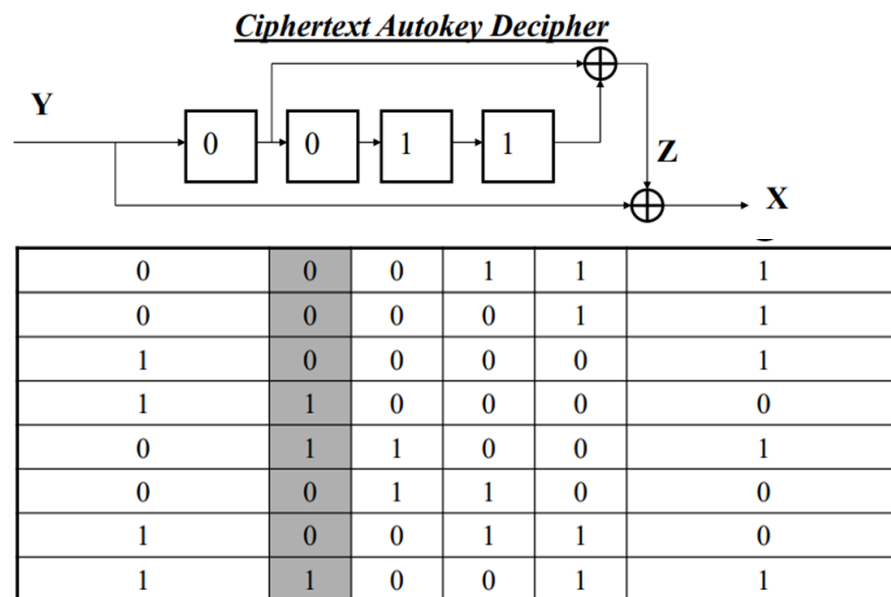


1	0	0	0	0	1
1	0	0	0	0	0
1	1	1	0	0	0
0	1	1	1	0	0
1	0	0	1	1	0
0	0	0	0	1	1
0	1	1	0	0	1
1	1	1	1	0	0

Пример дешифрации:

$$\varphi(x)=1+x+x^4; Y=00110011; S=(s_1s_2s_3s_4)=0011.$$

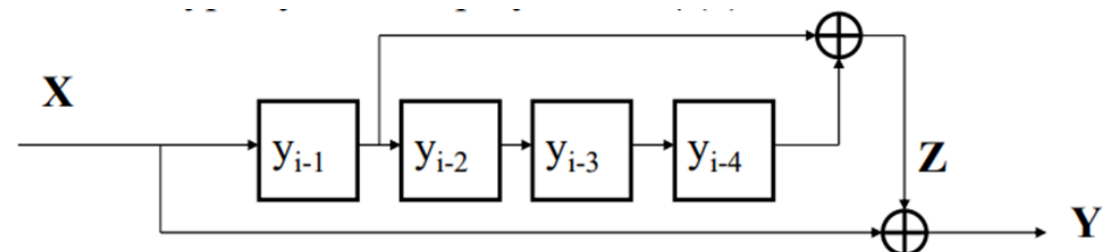
Схема дешифратора



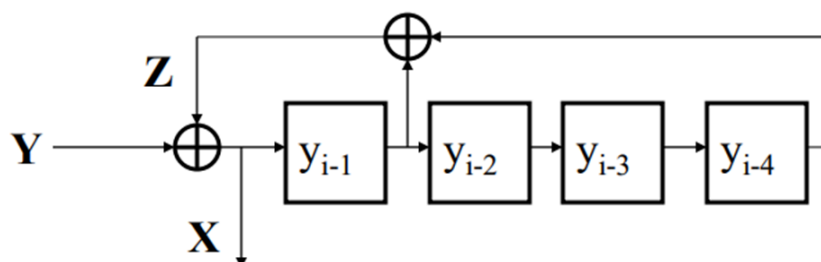
Второй случай – самосинхронизирующимися шифры с исходным текстом. У них исходный текст входит в сдвиговый регистр. Пример для полинома:

$$\varphi(x)=1+x+x^4$$

Самосинхронизирующийся шифр с зависимостью от открытого текста:



Дешифратор с обратной связью по исходному тексту



Эллиптические группы. Порядок точки эллиптической группы

Если a и b целые положительные числа и выполняется равенство:

$$4a^3 + 27b^2 \not\equiv 0 \pmod{M},$$

где M – простое число и $a < M$ и $b < M$, тогда Э. К. позволяет сформировать эллиптическую группу:

Эллиптическая группа $E_M(a, b)$ состоит из множества пар (x, y) целых положительных чисел, меньших M , которые удовлетворяют соотношению:

$$y^2 = x^3 + ax + b \pmod{M}$$

Правило генерации всех элементов эллиптической группы состоит из следующих шагов:

1. Для всех значений x , $0 \leq x < M$, необходимо вычислить значение $x^3 + ax + b \pmod{M}$.
2. Для каждого значения из предыдущего шага необходимо определить квадратный корень по модулю M . В случае отрицательного ответа не существует элемента внутри эллиптической группы $E_M(a, b)$. Если корень существует, то существует два значения y и два элемента в группе.

Пример: $M = 5$ и $y^2 = x^3 + 4 \pmod{5}$, что может быть описано как $E_5(0, 4)$. В этом случае $a = 0$, $b = 4$ и $4 \cdot 0^3 + 27 \cdot 4^2 \pmod{5} = 1 \not\equiv 0$, $\Rightarrow$ мы можем построить эллиптическую группу $E_5(0, 4)$. Начнем с $x = 0$; $y^2 = 0^3 + 4 \pmod{5} = 4$; $y = +2$ и $y = -2$. Тогда есть две точки $(0, 2)$ и $(0, -2) = (0, 3)$ в эллиптической группе $E_5(0, 4)$. Все множество точек в группе:

$$E_5(0, 4) = \{O, (0, 2), (0, 3), (1, 0), (3, 1), (3, 4)\}.$$

Количество точек в эллиптической группе оценивается неравенством Хоссего:

$$M + 1 - 2M^{0.5} \leq E_M(a, b) \leq M + 1 + 2M^{0.5}$$

Пример:

$M = 5$ и $y^2 = x^3 + 1 \pmod{5}$, $\Rightarrow E_5(0, 1)$. В этом случае $a = 0$, $b = 1$ и $4 \cdot 0^3 + 27 \cdot 1^2 \pmod{5} = 2 \not\equiv 0$ $\Rightarrow$ можно построить эллиптическую группу с 6 точками:

$$E_5(0, 1) = \{O, (0, 1), (0, 4), (2, 2), (2, 3), (4, 0)\}.$$

$$5 + 1 - 2 \cdot 5^{0.5} = 2 \leq E_5(0, 1) = 6 \leq 5 + 1 + 2 \cdot 5^{0.5} = 10$$

Для эллиптических групп также выполняются операции сложения и умножения, которые задаются данными выражениями:

Сложение различных точек P и Q . Если $P \neq Q$ и $P \neq -Q$, тогда $P + Q = R$, где:

$$\begin{aligned} s &= (y_P - y_Q) / (x_P - x_Q) \pmod{M}; \\ x_R &= s^2 - x_P - x_Q \pmod{M}; \\ y_R &= -y_P + s(x_P - x_R) \pmod{M}; \end{aligned}$$

Удваивание точки P . Представлено, что y_P не равен 0, $2P = R$, где:

$$s = (3x_P^2 + a) / (2y_P) \bmod M;$$

$$x_R = s^2 - 2x_P \bmod M;$$

$$y_R = -y_P + s(x_P - x_R) \bmod M;$$

a – один из параметров, выбираемых с Э. К.

Пример:

Рассмотрим сложение двух точек $P = (0, 1)$ и $Q = (2, 2)$, соответствующих эллиптической группе $E_5(0, 1)$. В силу того, что $(0, 1) \neq (2, 2)$ и $(2, 2) \neq -(0, 1)$, используем первый случай для сложения. Тогда $s = (y_P - y_Q) / (x_P - x_Q) \bmod M = (1 - 2) / (0 - 2) \bmod 5$. Как результат $2s = 1 \bmod 5$, $s = 3$. Результативная точка $R = (x_R, y_R)$ имеет координаты:

$$x_R = s^2 - x_P - x_Q \bmod M = (3^2 - 0 - 2) \bmod 5 = 2$$

$$y_R = -y_P + s(x_P - x_R) \bmod M = -1 + 3 * (0 - 2) \bmod 5 = -7 \bmod 5 = 3$$

Окончательно получим точку $R = (x_R, y_R) = (2, 3)$.

Рассмотрим удвоение точки $P = (2, 2)$, принадлежащей $E_5(0, 1)$. Т.к $(2,2) = (2,2)$, то следует выбрать второй случай для операции сложения. Тогда $S = (3x_P^2 + a) / (2y_P) \bmod M = (3 * 2^2 + 0) / (2 * 2) \bmod 5 = 3$. Результативная точка $2P = R = (x_R, y_R)$:

$$x_R = s^2 - 2x_P \bmod M = (3^2 - 2 * 2) \bmod 5 = 0$$

$$y_R = -y_P + S * (x_P - x_R) \bmod M = -2 + 3 * (2 - 0) \bmod 5 = 4.$$

$R = (0, 4)$

Рассмотрим сложение двух точек $P = -P$. $P = (2,2)$ и $-P = (2, 3)$, входящие в эллиптическую группу $E_5(0, 1)$. В силу того, что $(2, 2) = -(2,3)$ результатом будет точка O . Формально это значение получается при вычислении значения $s = (y_P - y_Q) / (x_P - x_Q) \bmod M = (2 - 3) / (2 - 2) \bmod 5 = \infty$.

Операция сложения точек $E_5(0,1)$

+	O	$(0,1)$	$(0,4)$	$(2,2)$	$(2,3)$	$(4,0)$
O	O	$(0,1)$	$(0,4)$	$(2,2)$	$(2,3)$	$(4,0)$
$(0,1)$	$(0,1)$	$(0,4)$	O	$(2,3)$	$(4,0)$	$(2,2)$
$(0,4)$	$(0,4)$	O	$(0,1)$	$(4,0)$	$(2,2)$	$(2,3)$
$(2,2)$	$(2,2)$	$(2,3)$	$(4,0)$	O	$(0,1)$	$(0,4)$
$(2,3)$	$(2,3)$	$(4,0)$	$(2,2)$	O	$(0,1)$	$(0,4)$
$(4,0)$	$(4,0)$	$(2,2)$	$(2,3)$	$(0,1)$	$(0,4)$	O

Если продолжать вычисления $P+P+P+\dots$ достаточно долго, то, поскольку число точек кривой конечное, в конце концов мы должны получить результат O . Отсюда следует, что $kP=O$. Наименьшее k , для которого это верно, называется порядком точки.

Результат выполнения операции $P+P+P+\dots$ для $E_5(0,1)$

+	$(0,1)$	$(0,4)$	$(2,2)$	$(2,3)$	$(4,0)$
P	$(0,1)$	$(0,4)$	$(2,2)$	$(2,3)$	$(4,0)$
$2P$	$(0,4)$	$(0,1)$	$(0,4)$	$(0,1)$	O
$3P$	O	O	$(4,0)$	$(4,0)$	$(4,0)$
$4P$	$(0,1)$	$(0,4)$	$(0,1)$	$(0,4)$	O
$5P$	$(0,4)$	$(0,1)$	$(2,3)$	$(2,2)$	$(0,4)$
$6P$	O	O	O	O	O

Как видно из приведенной таблицы у каждой точки различный порядок, так точки $(0,1)$ и $(0,4)$ имеют порядок $s=3$, точки $(2,2)$ и $(2,3)$ порядок $s=6$, а точка $(4,0)$ имеет $s=2$.

БИЛЕТ 7

1. Алгоритмы шифрования. Алгоритм вращающихся квадратов.
2. Способы решения линейных сравнений: 2 случай (одно решение)
3. Функция Эйлера
4. Эллиптическая криптосистема

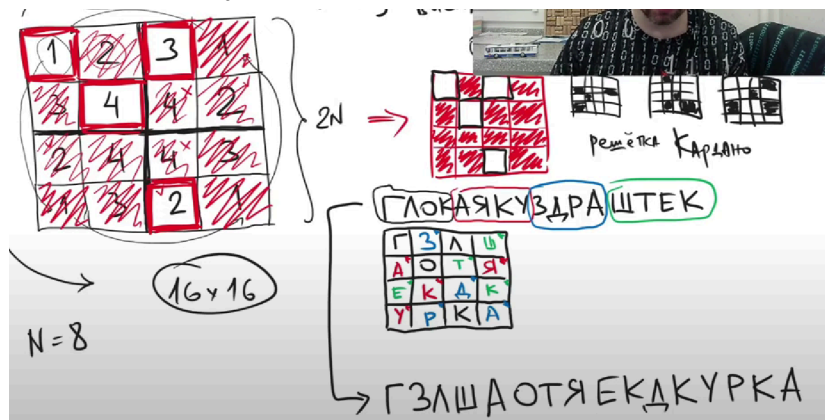
Алгоритмы шифрования. Алгоритм вращающихся квадратов.

Алгоритм вращающихся квадратов относится к перестановочным алгоритмам.

Ключ представлен квадратной матрицей, высота/ширина которой является числом четным.

Алгоритм.

1. Матрица делится на квадраты меньшего размера (пример матрица 8x8 делим на матрицы 4x4)
2. В меньшем квадрате случайным образом нумеруются клетки
3. Основная матрица заполняется числовыми значениями путем поворота маленького квадрата на 90 каждый раз.
4. Случайным образом выбираются отверстия (желательно, чтобы соприкосновения отверстий не было чтобы буквы в шифротекст не шли по порядку как в исходном тексте).
5. Поворачиваем матрицу. Записываем в отверстия символы исходного текста.



Количество вариантов перестановок: $Q = 4^{\frac{(2n)^2}{4}}$

Способы решения линейных сравнений: 2 случай (одно решение)

Дано. Линейное уравнение вида

$$ax = b \bmod n, b > n$$

Есть 3 возможней для решения линейного сравнения

1. Сравнение не имеет решений
2. **Сравнение имеет только одно решение**
3. Сравнение имеет множество решений

Теорема: Если a и n взаимно простые ($(a, n) = 1$), то линейное уравнение имеет только одно решение

Доказательство: Возьмем полную систему вычетов $\{0, 1, 2, \dots, n-1\}$. Так как a и n взаимно простые, то целые числа

$\{0-a, 1-a, 2-a, \dots, (n-1)-a\}$

Создают полную систему вычетов по модулю n . Среди всех чисел есть только одно и только одно ax с остатком b .

Функция Эйлера

Определена над целыми числами, определяет количество целых чисел, меньших n , и взаимно простых с n .

$$\psi(1) = 0$$

$$\psi(2) = 1$$

$$\psi(3) = 2$$

$$\psi(4) = 2$$

$$\psi(5) = 4$$

$$\psi(6) = 2$$

$$\psi(8) = 4$$

$$\psi(9) = 6$$

$$\psi(10) = 4$$

Теорема: Если $n = pq$, где p и q простые числа, тогда

$$\psi(n) = \psi(p)\psi(q) = (p - 1)(q - 1)$$

Эллиптическая криптосистема

БИЛЕТ 8

1. Алгоритмы шифрования. Подстановочные шифры. Алгоритм Цезаря
2. Способы решения линейных сравнений: 3 случай (множество решений)
3. Экспоненциальный шифратор (Pholing & Hellman)
4. Распределение ключей (симметричные криптосистемы)

Алгоритмы шифрования. Подстановочные шифры. Алгоритм Цезаря

Основой подстановочных шифров является таблица символов, которые ставятся в соответствие символы исходного и зашифрованного текста.

Криптографические системы, основанные на методе подстановки, можно разделить на четыре основных класса:

- Одноалфавитный шифр подстановки (шифр простой замены) – шифр, при котором каждый символ открытого текста заменяется некоторым фиксированным при данном ключе символом того же алфавита. Примером может служить шифр Цезаря.
- Однозвучный шифр подстановки (омофонный) похож на одноалфавитный за исключением того, что символ открытого текста может быть заменен одним из нескольких возможных символов. К данным шифрам относят шифр Билла.
- Полиграммный шифр подстановки заменяет не один символ, а целую группу символов. Примерами таких шифров являются шифр Плейфейера, шифр Хилла.
- Многоалфавитный шифр подстановки состоит из нескольких шифров простой замены. Например, шифр Виженера, одноразовый блокнот.

Основная формула алгоритма:

$$c = (a + k) \bmod n, \text{ где}$$

c - символ шифротекста

a - индекс символа исходного текста в алфавите

k – смещение индекса

Существуют более сложные методы подстановки. Шифраторы, основанные на умножении номера каждого символа исходного текста на значение ключа k (метод децимации)

$$c = ka \bmod n$$

Из конспекта:

$$a + k = c \bmod n$$

$$a = c - k \bmod n$$

$$a = c + (n + k) \bmod n$$

$$n - k = k'$$

k' – ключ дешифрации

Способы решения линейных сравнений: 3 случай (множество решений)

Дано. Линейное уравнение вида

$$ax = b \bmod n, b > n$$

Есть 3 возможных для решения линейного сравнения

4. Сравнение не имеет решений
5. Сравнение имеет только одно решение
- 6. Сравнение имеет множество решений**

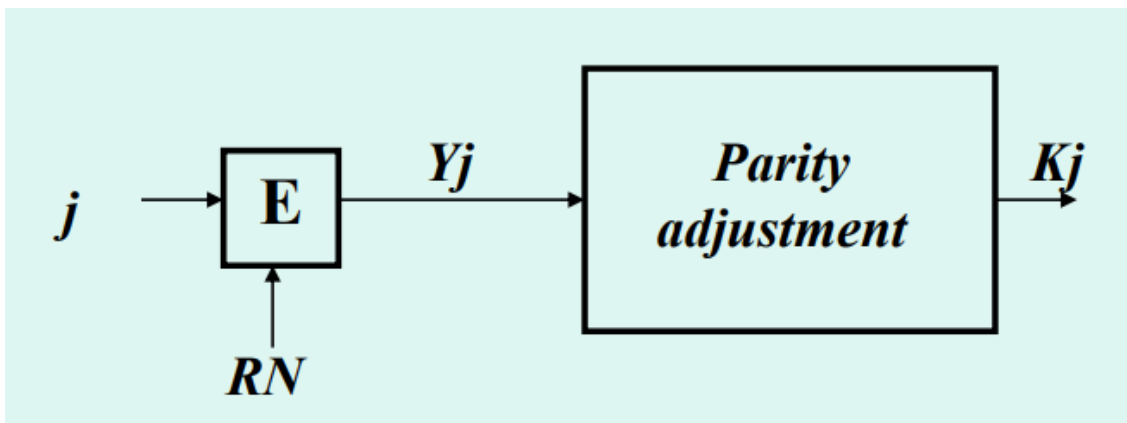
Экспоненциальный шифратор (Pholing & Hellman)

Распределение ключей (симметричные криптосистемы)

Предисловие про иерархию ключей в криптосистемах.

1. Главный ключ (как правило один, требования для него самые серьезные и качественные, хранение и доступ к главному ключу самые сложные)
2. Ключи для шифрования (для пересылки ключей)
3. Сеансовые ключи

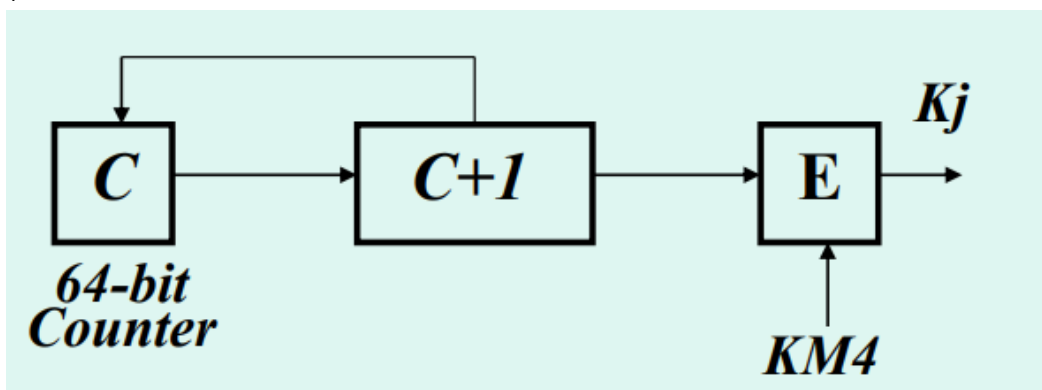
Генерирование ключей

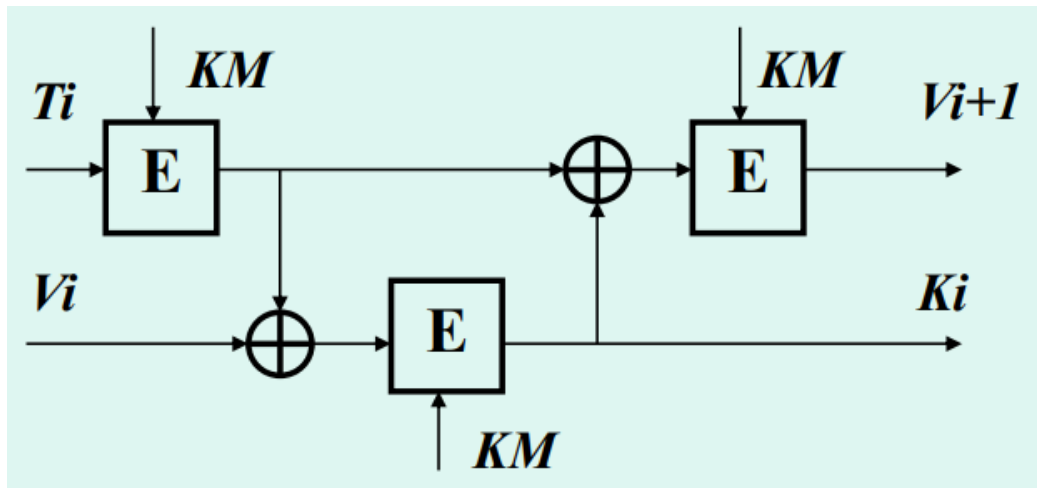


Основная идея – использование криптографических систем (генерирование через DES)

j = двоичный счетчик

RN = псевдо-случайное число либо главный ключ от RN (от него зависят остальные ключи)





Где

E — Алгоритм DES

V_0 — секретное 64-битное число

KM — главный ключ

T_i — время/момент времени

Распределение ключей для симметричной системы

Криптографические ключи генерируются/распределяются по прецеденту.

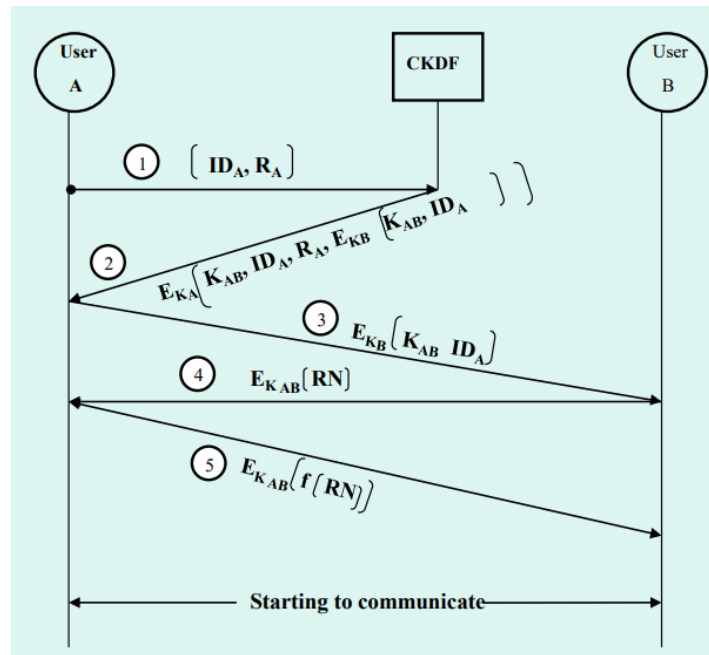
Предварительно все пользователи при регистрации должны получить один секретный ключ (он уникальный).

Этот протокол может быть применен к группе из n пользователей, чтобы устранить проблему генерации и управления

$$\binom{n}{2} = n(n-1)/2$$

возможных пары ключей, что было бы огромным количеством. Для реализации этого протокола необходим централизованный ключ.

Для организации связи между пользователями А и В необходимо использовать средство распространения (СКДФ). Все пользователи должны иметь ключи для связи с СКДФ. Пользователь А использует ключ K_A и пользователь В - K_B



Алгоритм

1. Пользователь A отправляет соответствующий запрос R_A с ID_A в $CKDF$ в открытой форме.
2. $CKDF$ отправляет пользователю A следующую криптограмму:

$$E_{K_A}(K_{AB}, ID_A, R_A, E_{K_B}(K_{AB}, ID_A)), \text{ где -}$$

- a. передача данных, защищенная междоменным ключом, между пользователями A и B ;
 - b. K_A и K_B - вторичные ключи связи между A и $CKDF$ и между B и $CKDF$, соответственно;
 - c. $E_{K_A}(K_{AB}, ID_A)$ - это криптограмма K_{AB}, ID_A , полученная с помощью секретного ключа пользователя B (имя ключа K_S).
3. Получив криптограмму, A расшифровывает ее и проверяет, совпадают ли идентификатор ID_A и запрос R_A с оригиналом. Если да, A пересылает сообщение $E_{K_B}(K_{AB}, ID_A)$ пользователю B .
 4. B расшифровывает полученную криптограмму под ключом K_B и восстанавливает понятную форму как K_{AB} , так и ID_A . В результате пользователи A и B могут иметь один и тот же ключ K_{AB} , он инициирует процедуру аутентификации, чтобы исключить любую возможность ложной криптограммы. Таким образом, B направляет криптограмму $E_{K_{AB}}(RN)$, т.е. шифрование случайного числа RN с помощью K_{AB} , пользователю A .
 5. A расшифровывает полученную криптограмму, чтобы получить RN , и модифицирует ее определенным образом, чтобы получить $f(RN) = (RN)''$, которое зависит от времени. Последовательность $f(RN)$ зашифровывается с помощью K_{AB} и возвращает $E_{K_{AB}}(f(RN))$ пользователю B .

6. Если A подтвердит получение законного RN и использование K_{AB} с помощью подтверждения связи, A может начать передачу сообщения B .

Билет 9.

9.1. Алгоритмы шифрования: Алгоритм децимаций. Аффинное преобразование.

Существуют более сложные методы подстановки. Шифраторы, основанные на умножении номера каждого символа исходного текста на значение ключа k (метод децимации), описываются следующим отношением 1.3:

$$Ek(i) = (i * k) \bmod n, \quad (1.3)$$

где n и k должны быть взаимно простыми числами, т.е. у них не должно быть общих делителей кроме 1.

Аффинное преобразование - это частный случай более общего моноалфавитного **шифра подстановки**. Поскольку аффинный **шифр** легко дешифровать - он обладает слабыми **криптографическими** свойствами. Комбинацией методов цезаря и децимации является аффинное преобразование, при котором уже используются два ключа:

$$Ek(i) = (i * k1 + k2) \bmod n. \quad (1.4)$$

Рассмотренные выше шифры простой замены легко взламываются с помощью криптографических атак, основанных на анализе частот появления символов в шифротексте. Так в естественном языке частота встречи букв разная и некоторые из них встречаются чаще, чем другие, и это является ключом для криптоаналитика. Проанализировав частоту встречи символов в шифротексте, можно сделать вывод о соответствии им символов исходного алфавита.

9.2. Симметричные криптосистемы. Lucifer. DES.

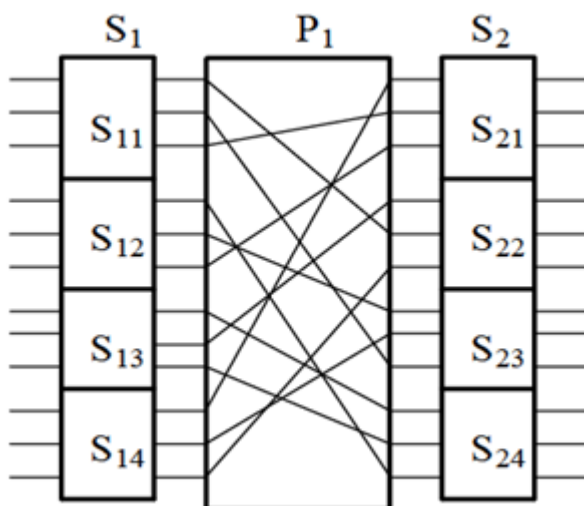
В **симметричной криптосистеме шифрования** используется один и тот же ключ для шифрования и расшифровки сообщения. Это означает, что любой, кто имеет доступ к ключу шифрования, может расшифровать сообщение. Именно поэтому симметричные криптосистемы называют криптосистемами с секретным ключом - ключ шифрования должен быть доступен только тем, кому предназначено сообщение.

Симметричные криптосистемы делятся следующих на два типа:

- 1. блочные криптосистемы** - разбивают информацию (файл / сообщение / документ) на отдельные блоки, а после этого преобразовывают данные блоки, используя секретный ключ;
- 2. поточные криптосистемы** - на основе секретного ключа создают определенную последовательность (выходная гамма), накладываемую на исходную информацию. Поэтому преобразование данных выполняется в потоке по мере создания выходной гаммы и данный тип криптосистем в основном применяется для шифрования в средствах связи.

Задача обеспечения конфиденциальности передачи электронных документов с помощью симметричной криптосистемы сводится к обеспечению конфиденциальности ключа шифрования. Обычно ключ шифрования представляет собой файл или массив данных и хранится на персональном ключевом носителе, например на дискете или смарт-карте; обязательно принятие мер, обеспечивающих недоступность персонального ключевого носителя кому-либо, кроме его владельца.

Lucifer.



Впервые этот подход реализован в виде шифра LUCIFER, созданного на фирме IBM Фейстлом (Feistl). LUCIFER использует преобразование исходного сообщения, переменнo применяя подстановки и перестановки.

Процедура подстановки шифруемого блока S_i из 12 бит состоит из 4 операций подстановки $S_{i1}, \dots, S_{i4}$, меньшей размерности, каждая из которых управляет трехбитным блоком данных. Это необходимо чтобы уменьшить сложность операций подстановки, которая реализуется с помощью таблиц подстановки. Перестановка P_i переставляет (перемешивает) все двенадцать бит, полученные как результат подстановки, в некоторой случайной последовательности. Например, для подстановки P_1 первый символ переставляется на пятую позицию, второй, на девятую, третий на вторую и так далее. Таким образом, последовательно выполняется несколько итераций для достижения хорошего качества шифрования.

DES.

Data Encryption Standard (DES), кодирует 64-битные блоки исходного текста, используя комбинацию процедур подстановки и перестановки.

В 1977 Национальное Бюро Стандартов США представило стандарт шифрования данных (Data Encryption Standard - DES) для использования в несекретных приложениях правительства США, а также различными неправительственными организациями. Алгоритм шифрования был разработан на фирме IBM и был производным от криптосистемы LUCIFER.

DES кодирует 64-битные блоки данных с использованием 56-битного ключа. Существует мнение, что 56-битный ключ являлся достаточно сильным на ранних стадиях использования стандарта, а в настоящее время его размерность недостаточна.

Алгоритм стандарта DES, который используется и для кодирования и для декодирования, в обобщенном виде представлен на рисунке 4.2. Входной блок $T = t_1t_2, \dots, t_{64}$ состоящий из 64 бит, предварительно перемешивается согласно исходной перестановке IP, $T_0 = IP(T)$. После прохождения через 16 итераций функции F, он перемешивается согласно обратной перестановке IP-1 и формирует итоговый результат.

Перестановки IP и IP-1 заданы в таблице 4.1.

Как правило, реализация DES предполагает аппаратное решение в виде микросхемы с целью обеспечения высокого быстродействия, хотя возможна и программная реализация, позволяющая вносить модификации в данный алгоритм.

Выделяют два основных недостатка DES. 1. Размер ключа равный 56 бит может не обеспечить достаточную криптостойкость алгоритма. 2. Функция F подстановки, являющаяся одним из основных элементов, защищающих DES от взлома, может содержать в себе скрытые слабости.

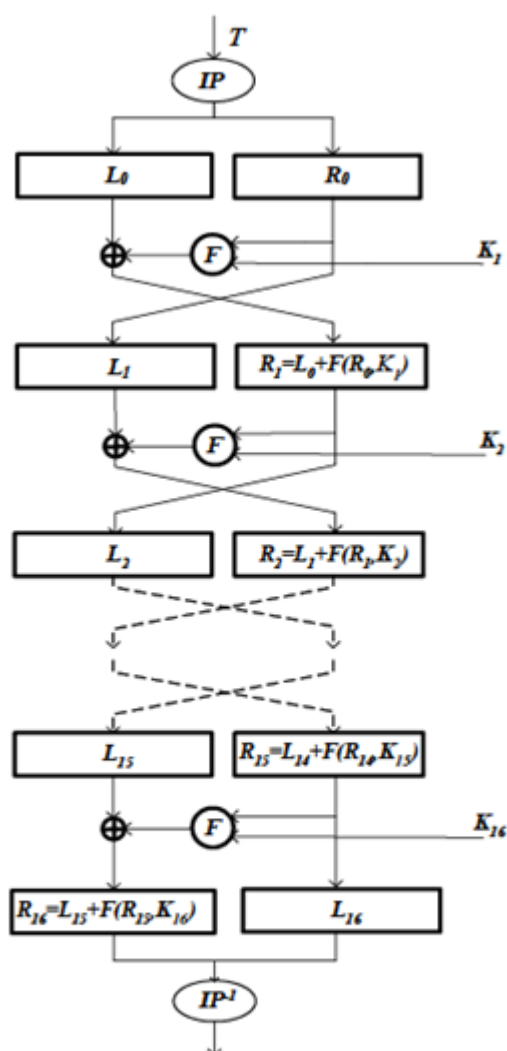


Рис. 4.2. Диаграмма **DES**

Система шифрования Виженера (*Vigenere Cipher*) впервые была опубликована в 1586 году и является одной из старейших и наиболее известных много алфавитных систем шифрования.

Шифратор Виженера, подобно как и шифр Porta, по своей сути использует развитие идеи Цезаря. Отличается только тем, что число реальных таблиц подстановки зависит от длины криптографического ключа и используемого алфавита. Важной особенностью шифратора Виженера является чередование использования таблиц подстановки в зависимости от последовательности символов используемого ключа. Этот шифр много алфавитной подстановки можно описать таблицей шифрования называемой **таблицей (квадратом) Виженера**. Ниже приведен пример таблицы Виженера.

В таблице Виженера каждая строка представляет собой циклически сдвинутую на один символ предыдущую строку таблицы таким образом, что каждая строка по своей сути является таблицей подстановки шифратора Цезаря для конкретного значения ключа.

Таблица Виженера используется для шифрования и дешифрирования. Верхняя строка таблицы Виженера используется для задания символов исходных текстов, а левый столбец для задания символов криптографического ключа.

При шифровании исходного сообщения его записывают в строку, а под ним ключевое слово либо фразу. Если ключ оказался короче исходного текста, то его циклически повторяют необходимое число раз. На каждом шаге шифрования в верхней строке таблицы Виженера находят очередную букву исходного текста, а в левом столбце - очередное значение символа ключа. В результате очередная буква шифротекста находится на пересечении столбца определенного символом исходного текста и строки, соответствующей строке символа ключа.

При шифровании слова $M=CRYPTOGRAPHY$ по методу Виженера для

ключа RAND предварительно исходный текст и ключевое слова запишем в виде двух строк.

```

C R Y P T O G R A P H Y
R A N D R A N D R A N D

```

Тогда первая буква исходного текста *C* определяет третий столбец таблицы Виженера а буква *R* ключа семнадцатую строку таблицы на пересечении которых находится символ шифротекста *T*. Окончательный результат шифрования имеет вид *C=TRLSKOTURPUB*.

Различают три возможных варианта использования криптографического ключа: прямое использование (*Straight Keyword*); прогрессивный ключ (*Progressive Key*); самогенерирующийся ключ (*Auto Key*). Рассмотрим использование всех трех вариантов для исходного сообщения "*Wish you were here*".

Пример 2.17. Для шифрования сообщения "*Wish you were here*", используем ключ SIAMESE. Тогда для первого случая прямого использования ключа получим:

```

M= W I S H Y O U W E R E H E R E
K= S I A M E S E S I A M E S E S
C= O Q S T C G Y O M R Q L W V W

```

Идея использования прогрессивного ключа заключается в циклическом сдвиге символов ключа на одну позицию в упорядоченном алфавите символов при повторном применении ключа. Тогда для ключа SIAMESE при повторном его использовании по прогрессивной схеме имеем TJBNFTF, а при третьем UKCOGUG, и так далее.

Пример 2.18. Для шифрования сообщения "*Wish you were here*", используем ключ SIAMESE и прогрессивную схему его применения получим.

```

M= W I S H Y O U W E R E H E R E
K= S I A M E S E T J B N F T F U
C= O Q S T C G Y P N S R M X W Y

```

В случае самогенерирующегося ключа в качестве его последующих символов используется исходный текст.

Пример 2.19. Для шифрования сообщения "Wish you were here", используем самогенерирующийся ключ SIAMESE. В результате имеем.

$M =$ W I S H Y O U W E R E H E R E
 $K =$ S I A M E S E W I S H Y O U W

37

$C =$ O Q S T C G Y S M J L F S L W

Модификациями метода Вижинера могут быть алгоритмы, в которых используются другие алфавиты, в том числе и различные алфавиты для исходных текстов и криптографических ключей, а также другой принцип генерирования строк шифрующей таблицы.

9.4. Алгоритм обмена ключами по открытому каналу с использованием эллиптических групп.

Эллиптическая открытая система распределения ключей (*public key distribution algorithm*). Стойкость основана на сложн. вычисл. дискретного логарифма над эллиптической группой $EM(a,b)$. Рассм. пару взаимно инверсных преобр-ний: $Q=kG; k=\log GQ \pmod{M}$ над группой $EM(a,b)$, где k – простое целое число, а G – генерирующая точка эллиптической группы $EM(a,b)$.

Алгоритм:

1. Пользователь A случайным образом выбирает целое число n_A , такое, что $n_A < M$, которое является аналогом секретного ключа пользователя A . Значение числа n_A должен знать только пользователь A . A генерирует свой открытый параметр (ключ) как результат скалярного произведения n_A на G , то есть выполняет вычисление точки $PA = n_A G$, которая является точкой эллиптической группы $EM(a,b)$.
2. Пользователь B повторяет аналогичные действия и в результате получает свой секретный параметр n_B и соответствующий ему открытый параметр PB .
3. По открытому каналу юзер A высылает в адрес юзера B свой открытый параметр PA , а юзер B – свой параметр PB в адрес юзера A .
4. Пользователь A получает значение секретного ключа K как рез-т скалярного произведения $K = n_A PB$. Аналогично пользователь B генерирует значение общего с пользователем A секретного ключа K как рез-т вычисления $K = n_B PA$. В рез-те оба юзера получают одно и то же

знач. ключа K из-за того, что для скалярного произведения целого числа на точку эллип. гр. выполняется след. рав-во $nAPB = nA(nBG) = nB(nAG) = nBPA$.

Билет 10.

10.1. Шифраторы, использующие различные алфавиты. Шифр церковного двора.

Все ранее рассмотренные примеры простейших подстановочных алгоритмов шифрования основывались на использовании одного и того же алфавита для исходных текстов и шифротекстов.

Чаще всего подстановочные элементы шифротекста кодируются цифровыми эквивалентами. Примером подобного подхода может быть метод, основанный на использовании ключа в виде некоторого предложения либо словосочетания в котором используются все буквы алфавита исходных текстов. Например, фраза ключевого словосочетания, имеющего определенный смысл, *“The quick brown fox jumps over the lazy dog”* содержит все символы английского алфавита. Отметим, что повторение символов исходного текста еще больше усложняет шифр, так как в данном случае одной букве исходного текста будет соответствовать несколько подстановочных элементов. При построении ключевой фразы необходимо

22

придерживаться ограничений на длину выбираемых слов, которая не должна превышать девяти символов.

THE	QUICK	BROWN	FOX	JUMPS	OVER	THE	LAZY	DOG
1	2	3	4	5	6	7	8	9

Рис.2.5. Ключевая фраза, определяющая таблицу подстановки

Так, в соответствии с приведенной таблицей символу исходного текста *T* соответствующие подстановочные элементы 11 и 71.

Пример 2.11. Для исходного текста *M*=JOHNSON IS SPY получим

<i>J</i>	O	H	N	S	O	N	I	S	S	P	Y
51	33	12	35	55	33	35	23	55	55	54	84

Тогда шифротекст принимает вид *C*=513312355533352355555484.

Весьма оригинальным шифром, использующим различные алфавиты для исходных текстов и шифротекстов, является шифр *церковного двора* (*The Churchyard cipher*), который был использован для шифрования текста на надгробном камне.

В соответствии с шифром церковного двора, на надгробной плите на одном из кладбищ Нью-Йорка в 1794 был выгравирован текст

**	*	**	*	**	*	*	**
	*	*	*		*		

Рис.2.6. Криптограмма шифра церковный двор

Для того, что бы прочитать зашифрованный текст необходимо знать соответствующий криптографический ключ, который в данном случае представляется в виде таблицы подстановки следующего вида:

A*	B*	C*	K**	L**	M**	T	U	V
D*	E*	F*	N**	O**	P**	W	X	Y
G*	H*	I-J*	Q**	R**	S**	Z		

Данный пример показывает многообразие возможных реализаций простейших подстановочных шифраторов.

10.2. Цифровая подпись. Цифровая подпись, основанная на симметричных алгоритмах (ЭЦП & Diffie & Lamport).

Пусть у Алисы есть 256-битная криптографическая хеш-функция и криптографически стойкий генератор псевдослучайных чисел. Она хочет создать и использовать пару ключей Лэмпорта — секретный ключ и соответствующий ему открытый ключ.

Создание пары ключей

Чтобы создать секретный ключ, Алиса использует генератор случайных чисел для получения 256 пар случайных чисел (всего 2×256 чисел). Каждое число состоит из 256 бит, то есть общий размер равен $2 \times 256 \times 256$ бит = 16 [КиБ](#). Эти числа будут секретным ключом Алисы, и она будет хранить их в секретном месте, чтобы использовать в дальнейшем.

Чтобы создать открытый ключ, Алиса хеширует каждое из 512 чисел секретного ключа, таким образом получая 512 хешей по 256 бит каждый. Эти 512 хешей составляют открытый ключ Алисы, который она публикует.

Подпись сообщения

Теперь Алиса хочет подписать сообщение. Для начала она хеширует сообщение и получает 256-битный хеш. Затем, для каждого бита в этом хеше, она берёт соответствующее число из секретного ключа. Если, например, первый бит в хеше сообщения равен нулю, она берёт *первое* число из первой пары секретного ключа. Если же первый бит равен единице, она использует *второе* число из первой пары. И так далее. В итоге получается 256 случайных чисел, размер которых составляет 256×256 бит = 8 КиБ. Эти числа и составляют подпись, которую Алиса отправляет вместе с сообщением.

Стоит обратить внимание, что после того как Алиса использовала свой секретный ключ, он никогда не должен быть использован снова. Остальные 256 чисел, которые она не использовала в подписи, Алиса никогда не должна публиковать или использовать. Предпочтительно, чтобы она их удалила, так как иначе кто-то может получить к ним доступ и сгенерировать с их помощью поддельную подпись.

Проверка подписи

Боб хочет проверить подпись, которым Алиса подписала сообщение. Он также хеширует сообщение и получает 256-битный хеш. Затем для каждого бита в этом хеше он выбирает число из открытого ключа Алисы. Эти числа выбираются по такому же принципу, по которому Алиса выбирала числа для составления подписи. То есть, если первый бит хеша сообщения равен нулю, Боб выбирает первое число из первой пары в открытом ключе. И так далее.

Затем Боб хеширует каждое из 256 чисел из подписи Алисы и получает 256 хешей. Если эти 256 хешей в точности соответствуют 256 хешам, которые он только что получил из открытого ключа Алисы, Боб считает подпись подлинной. Если не соответствуют — то фальшивой.

Стоит обратить внимание, что до того, как Алиса опубликует подпись к сообщению, никто не знает 2×256 случайных чисел в секретном ключе. Таким образом, никто не может создать правильный набор из 256 чисел для подписи. После того, как Алиса опубликует подпись, никто всё ещё не знает остальные 256 чисел, и, таким, образом, не может создать подпись для сообщений, имеющих иной хеш.^[2]

10.3. Криптосистема типа «рюкзак» №3.

Криптосистемы с открытым ключом неразрывно связаны с понятием *односторонней функции*. Для таких функций их значение легко вычисляется по заданному аргументу x . В тоже время определение x по значению функции $f(x)$ является трудно вычислимым.

Очень сложно теоретически и практически показать, что *трудновычислимость* функции $x=f^{-1}(x)$ (задачи, алгоритма) используемой нами для вычисления является неслучайной и не есть степень нашего невежества в понимании конкретной проблемы. Трудновычислимость это понятие, взятое из теории сложности, которое отражает достижения фундаментальных исследований и в какой то мере свидетельствует о достижениях науки в целом. Теория сложности дает оценку сложности решения той или иной задачи с учетом имеющихся на текущий момент знаний (методов решения этой задачи). Трудновычислимость свидетельствует о том, что на текущий момент не существует известных методов решения задачи за полиномиальное время. Но это не значит, что такие методы не существуют и не будут открыты в будущем. Поэтому в криптографии используется достаточно осторожное определение односторонней функции.

Таким образом, *криптографической односторонней функцией* называется функция, отвечающая двум условиям:

1. Значение $f(x)$ легко вычисляется на основании значения x .
2. Определение x из $f(x)$, вероятно, будет трудновычислимым.

Одним из первых примеров односторонних функций является задача о рюкзаке (*knapsack problem*), которая может быть сформулирована следующим образом.

Пусть задан набор $K=\{k_1, k_2, k_3, \dots, k_n\}$, из n различных положительных чисел, а также еще одно положительное число C . Задачей является нахождение таких k_i , если это возможно, сумма которых равна C .

В простейшем случае C указывает размер рюкзака, а каждое из чисел k_i указывает размер предмета, который может быть упакован в рюкзак. Задачей является нахождение такого набора предметов, чтобы рюкзак был полностью заполнен. В качестве иллюстрации рассмотрим число $C=1524$ и набор из 10 чисел

$$K=(123, 763, 37, 1451, 830, 333, 621, 745, 971, 201).$$

Заметим, что

$$1524=123+37+830+333+201.$$

Таким образом, мы знаем решение. В принципе решение всегда может быть найдено полным перебором подмножеств K и проверкой, какая из их сумм равна C . В случае нашего примера это означает перебор $2^{10}=1024$ подмножеств и это вполне осуществимо. В реальных условиях набор $K=\{k_1, k_2, k_3, \dots, k_n\}$, будет состоять из $n \sim 200 \div 300$ различных чисел, для которых полный перебор не является реальным методом решения. Суть здесь в том, что неизвестны алгоритмы, имеющие существенно меньшую сложность по

сравнению с полным перебором. Поиск среди 2^{500} подмножеств не поддается обработке за реальное время. На сегодняшний день задача о рюкзаке известна как NP полная, имеющая экспоненциальную сложность.

Задача о рюкзаке является ярким примером односторонней функции $f(x)$, которую определим следующим образом.

Набор $K=\{k_1, k_2, k_3, \dots, k_n\}$, из n положительных чисел k_i определяет функцию $f(x)$ как двоичное число $0 \leq f(x) \leq 2^n - 1$ полученное из номеров индексов i представленных векторами из $n-1$ нулевого символа и одного единичного, таким образом, что позиция единичного символа соответствует позиции числа k_i в множестве K . Число k_1 имеет индекс $000\dots 01$, k_2 индекс $000\dots 10, \dots$ и k_n имеет индекс $100\dots 00$. Тогда $f(1) = f(000\dots 01) = k_1$, $f(2) = f(000\dots 10) = k_2, \dots, f(n) = f(100\dots 00) = k_n$.

Наш предыдущий результат можно записать как $f(565) = f(1000110101) = 1524$.

Используя определение односторонней функции, сформулируем основные компоненты криптографической системы.

Криптографическим ключом будет являться $K=\{k_1, k_2, \dots, k_n\}$, где k_i целые для $i=1, 2, \dots, n$, а исходным текстом n -битный код $M=\{x_1, x_2, \dots, x_n\}$, где $x_i \in \{0, 1\}$. Затем криптосистема типа рюкзак шифрует n -битный открытый текст по следующей формуле

$$C = K \times M = k_1 x_1 + k_2 x_2 + \dots + k_n x_n.$$

Вычисление C – это, как видно из приведенного выражения, достаточно простая процедура, но восстановление M из C и K как уже отмечалось, является трудноразрешимой задачей. Таким образом, в подобной интерпретации имеем криптографическую систему, когда даже при наличии криптографического ключа проблема дешифрования является в принципе неразрешимой задачей. Для того, что бы алгоритм рюкзака можно было использовать для целей криптографии, однонаправленная функция, описывающая его, должна иметь *лазейку (trap door)*

Если криптографический ключ K выбран так, что каждый его элемент k_i превосходит сумму предшествующих элементов, то решение задачи о рюкзаке становится очень простым. То есть если компоненты ключа удовлетворяют условию $k_i > k_1 + k_2 + k_3 + \dots + k_{i-1}$, то проблема дешифрования реализуется достаточно просто.

При выполнении процедуры шифрования последовательно будут формироваться следующие значения.

$$\begin{aligned} c_1 &= k_1 x_1; \\ c_2 &= k_1 x_1 + k_2 x_2; \\ &\dots \\ c_{n-1} &= k_1 x_1 + k_2 x_2 + \dots + k_{n-2} x_{n-2} + k_{n-1} x_{n-1}; \\ C = c_n &= k_1 x_1 + k_2 x_2 + \dots + k_{n-2} x_{n-2} + k_{n-1} x_{n-1} + k_n x_n. \end{aligned}$$

где c_n представляет собой закодированный текст C . Теперь, открытый текст M может быть восстановлен из c_n для $i=1,2,...,n$ и ключа K следуя следующей процедуре.

Если $c_n < k_n$, то значение $x_n=0$, это следует из того факта, что в противном случае если $x_n=1$ то $c_n \geq k_n$. Это означает, что если $c_n < k_n$, то коэффициент k_n не был вложен в рюкзак. Тогда соответственно $c_{n-1}=c_n$. Если $c_n \geq k_n$, то набор $x_n=1$ и $c_{n-1}=c_n-k_n$.

Используя ранее полученное значение c_{n-1} можно таким же образом найти x_{n-1} и c_{n-2} . Процедура декодирования продолжается до тех пор, пока $M=\{x_1, x_2, ..., x_n\}$, не будет полностью восстановлен.

Отметим, что в подобной интерпретации криптосистема типа рюкзак может быть использована только как симметричная криптосистема, когда только отправитель и получатель знают секретный ключ $K=\{k_1, k_2, ..., k_n\}$.

Пример 5.3. Допустим открытый текст, принимает следующий вид $M=\{11001\}$, а криптографический ключ $K=\{151, 187, 426, 1091, 2412\}$. Здесь для компонент ключа выполняется условие $k_i > k_1+k_2+k_3+...+k_{i-1}$.

Шифротекст C будет вычисляться согласно следующей зависимости.
 $C=K \times M = 1 \times 151 + 1 \times 187 + 0 \times 426 + 0 \times 1091 + 1 \times 2412 = 2750$.

Зашифрованный текст $C=c_5=2750$ высылается в адрес получателя информации. На первом этапе дешифрования получатель восстанавливает x_5 из c_5 и k_5 . Получим что $x_5=1$, так как $c_5=2750 > k_5=2412$. Полная процедура дешифрования x_i для $i=5,4,...,1$ имеет вид.

$$\begin{aligned} c_5 &= 2750 > k_5 = 2412, x_5 = 1; \\ c_4 &= c_5 - k_5 = 338 < k_4 = 1091, x_4 = 0; \\ c_3 &= 338 < k_3 = 426, x_3 = 0; \\ c_2 &= 338 > k_2 = 187, x_2 = 1; \\ c_1 &= c_2 - k_2 = 151 = k_1, x_1 = 1. \end{aligned}$$

Таким образом, как и ожидалось, восстановленный открытый текст $M=\{11001\}$.

Однако, как уже отмечалось, ключ K не может использоваться, как открытый ключ для шифрования, потому что любой пользователь, имеющий информацию о ключе, может легко восстановить M из C .

Для использования алгоритма рюкзака для целей создания криптосистем с открытым ключом, в рассмотренный выше алгоритм необходимо внести ряд модификаций. Процедура построения криптосистемы с открытым ключом будет состоять из следующих этапов.

1. Первоначально генерируется секретный ключ $K_p^*=\{k_1^*, k_2^*, ..., k_n^*\}$ для которого выполняется условию $k_i^* > k_1^*+k_2^*+k_3^*+...+k_{i-1}^*$. Этот ключ должен быть известен только одному пользователю.

2. Выбирается значение модуля m , которое должно быть больше, чем $k_1^*+k_2^*+...+k_n^*$, что необходимо для однозначного декодирования шифротекста. Затем определяется величина w , где $w < m$, $(w, m)=1$ и вычисляется еще один секретный параметр $v=w^{-1}$, в соответствии с $wv=1 \bmod m$.

m или что тоже самое $ww^{-1}=1 \pmod{m}$. Параметры m , w и v являются секретными.

3. Генерируется открытый ключ как вектор $K_p = wK_p^* \pmod{m}$, где его компоненты вычисляются как $k_i = wk_i^* \pmod{m}$, $i=1,2,\dots,n$.

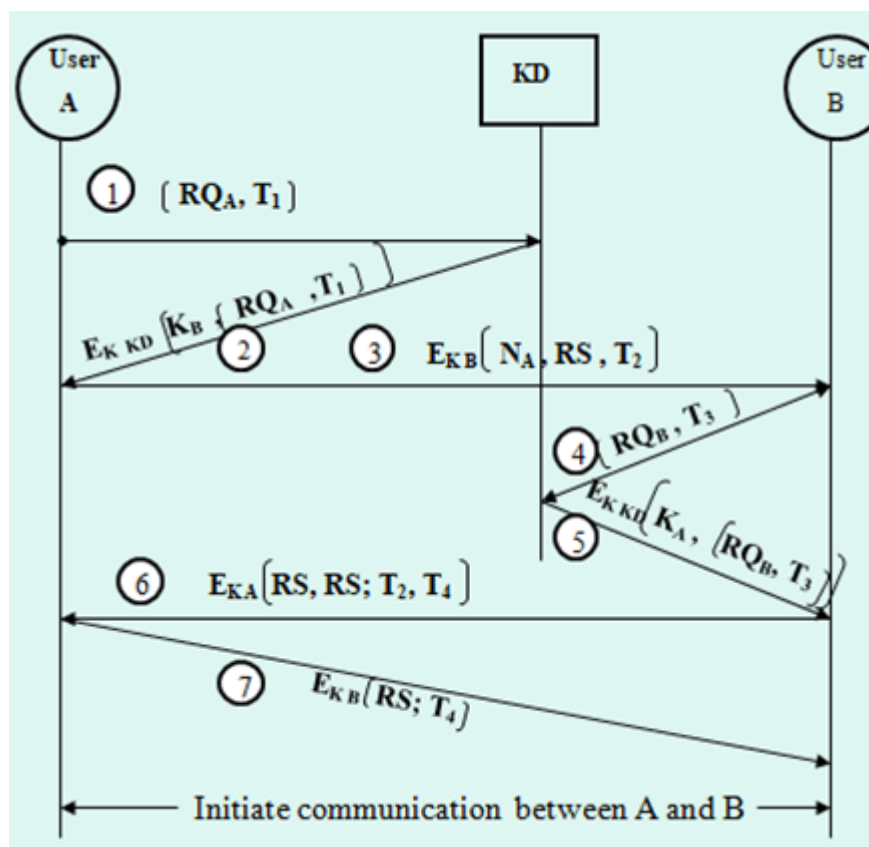
4. Открытый ключ $K_p = \{k_1, k_2, \dots, k_n\}$ является доступным для всех пользователей.

10.4. Протокол распределения ключей для асимметричных алгоритмов.

Протокол распределения ключей для асимметричных алгоритмов

В асимметричных криптосистемах открытый ключ может быть отправлен через небезопасный канал, не подвергая опасности секретный ключ. Но любой пользователь, получающий ключ, должен проверить подлинность ключа.

Используя технику временного тиснения, давайте разработаем протокол распределения ключей между двумя пользователями А и В, принадлежащими к одному и тому же ключевому каталогу. Пусть K_A - открытый ключ пользователя А и K_B , который для пользователя В. Они зарегистрированы в Key Directory (KD). Предположим также, что открытый ключ K_{KD} KD известен всем его пользователям. Протокол для установления распределения ключей между А и В следующий:



1. Пользователь А инициирует процедуру получения открытого ключа В, K_B , отправив запрос RQ_A вместе с текущим временем T_1 в ясной форме в KD.

2. Затем KD находит KB на основе запроса RQA и передает KB вместе с копией (RQA , $T1$) в A в виде криптограммы $E_{KD} [KB, (RQA, T1)]$, зашифрованной под открытым ключом KD .

3. Знание KD A может расшифровать криптограмму и извлечь аутентичный открытый ключ KB , копию запроса RQA и время $T1$. Пара (RQA , $T1$) затем сравнивается с оригиналом. Теперь, имея открытый ключ KB , A посылает B криптограмму, где NA является именем A , RS - случайная последовательность, выбранная A , а $T2$ - текущее время.

4. Получив криптограмму, B воссоздает четкую форму сообщений с использованием секретного ключа KS' . Поскольку B знает имя NA , B может запросить аутентичный открытый ключ A из KD , отправив соответствующий запрос RQB и текущее время $T3$.

5. В ответ KD передает на B криптограмму $E_{KD} [KA, (RQB, T3)]$, зашифрованную под открытым ключом KD .

6. После расшифровки криптограммы B получает аутентичный открытый ключ A , но пару (RQB , $T3$) следует сравнить с оригиналом. Затем B создает криптограмму $E_A (RS, RS', T2, T4)$, зашифрованную под открытым ключом A , где случайная последовательность RS получается от пользователя A , случайная последовательность RS' является самогенерируемой, а $T4$ является текущей время. Впоследствии эта криптограмма отправляется A .

7. Дешифрует криптограмму и сравнивает (RS , $T2$) с оригиналом. Если эти последовательности совпадают, A уверен, что пользователь B является аутентичным. Наконец, A ретранслирует случайную последовательность RS' в виде криптограммы $E_{KB} (RS', T4)$ в B .

8. В сравнить оригинал RS' для подтверждения подлинности A . Затем запустите

БИЛЕТ 11

1. DES: получение итерационных ключей
2. Модификация криптосистемы типа рюкзак до криптосистемы с открытым ключом
3. Количество информации. Энтропия
4. Гомофонные шифраторы (Омофонные шифраторы)

DES: получение итерационных ключей

Модификация криптосистемы типа рюкзак до криптосистемы с открытым ключом

Количество информации. Энтропия

Гомофонные шифраторы (Омофонные шифраторы)

Каждому символу s ставится в соответствие множество значений (символов), количество которых определяется частотой встречаемости символа s .

Определяющим недостатком является кратное увеличение шифротекстов.

Из-за случайного выбора падает качество шифратора (из-за выбора из подмножества) Из презентации

Шифр гомофонической подстановки преобразует каждый символ a алфавита открытого текста в набор элементов зашифрованного текста (fa), называемых омофонами. Открытое текстовое сообщение $M=m_1 m_2 \dots$ шифруется как $C=c_1 c_2 \dots$, где каждое c выбирается случайным образом из набора омофонов $f(m_3)$.

Пример. Пусть английские буквы зашифрованы как целые числа от 00 до 99, где количество целых чисел, присвоенных букве, пропорционально относительной частоте употребления буквы, и ни одно целое число не присваивается более чем одной букве. Показано возможное присвоение целых чисел буквам в криптографии сообщения ниже (целочисленные присвоения для остальных букв алфавита не приведены)

letter	Homophones
A	23, 25, 97, 95, 81, 33, 12, 11
C	44, 77, 34, 51
G	87, 41
H	59, 90, 00, 26
O	66, 02, 15, 22, 09, 83, 54
P	04, 58
R	38, 07, 94, 30, 56, 67
T	55, 71, 72, 80, 01, 12, 29, 50, 68
Y	88
One possible encipherment of the message is:	
$C = 77\ 07\ 88\ 58\ 72\ 54\ 41\ 30\ 97\ 04\ 00\ 88$	

1. Single letter frequency distributions.

A	0.0804	H	0.0549	O	0.0760	V	0.0099
B	0.0154	I	0.0726	P	0.0200	W	0.0192
C	0.0306	J	0.0016	Q	0.0011	X	0.0019
D	0.0399	K	0.0067	R	0.0612	Y	0.0173
E	0.1251	L	0.0414	S	0.0654	Z	0.0009
F	0.0230	M	0.0253	T	0.0925		
G	0.0196	N	0.0709	U	0.0271		

БИЛЕТ 12

1. Шифратор Билла
2. Международный алгоритм шифрования данных (IDEA). Основные характеристики и используемые операции
3. Электронная цифровая подпись
4. Сравнения

Шифратор Билла

Шифратор Била является гомофонным шифратором.

Пример. Ключам является Декларация Независимости США, записывается по 100 строк, в каждой по 10 слов.

<i>Declaration of Independence (first 100 words)</i>	
01	When, in the course of human events, it becomes necessary
11	for one people to dissolve the political bands which have
21	connected them with another, and to assume among the Powers
31	of the earth the separate and equal station to which
41	the Laws of Nature and of Nature's God entitle them,
51	a decent respect to the opinions of mankind requires that
61	they should declare the causes which impel them to the
71	separation. We hold these truths to be self-evident; that
81	all men are created equal, that they are endowed by
91	their Creator with certain unalienable rights; that among

Шифратор Била шифрует каждую букву в текстовом сообщении, заменив номер какого-либо слова, начинающегося с этой буквы. Буква W, например, была зашифрована номером 1, 19, 40, 66, 72, В качестве примера рассмотрим слово простой. Результат шифрования показан ниже:

<i>M</i> =	P	L	A	I	N
<i>C</i> =	13	42	81	08	44

Первой допущение: Декларация является среднестатистическим англоязычным текстом

Второе допущение: Вероятность буквы в начале текста примерно равна вероятности этой же буквы в тексте в принципе.

Международный алгоритм шифрования данных (IDEA). Основные характеристики и используемые операции

Постулаты Моссея:

1. Оптимальный размер блока равен 64 бита
2. Размер ключа – 128 бит
3. Функция, определенная Шенноном, которая оценивает зависимость С от М и к, сложная и неочевидная

4. Каждый бит исходного текста и каждый бит ключа должен быть учтен или использован для каждого бита шифротекста. (Диффузия)

Для достижения соответствующего уровня, представленном в 3 постулате, было предложено 3 операции над 16-битными операндами:

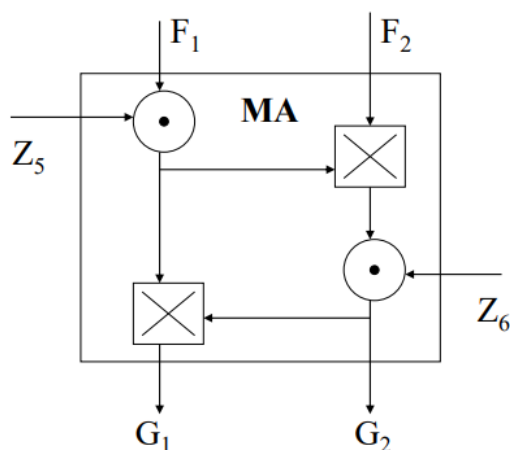
1. Поразрядный XOR
2. Обычное сложение целых чисел с игнорированием переноса
3. Умножение целых чисел (по модулю $2^{16} + 1$) код из 16 нулей понимается как 2^{16}

При использовании неравенств нужно соблюдать законы:

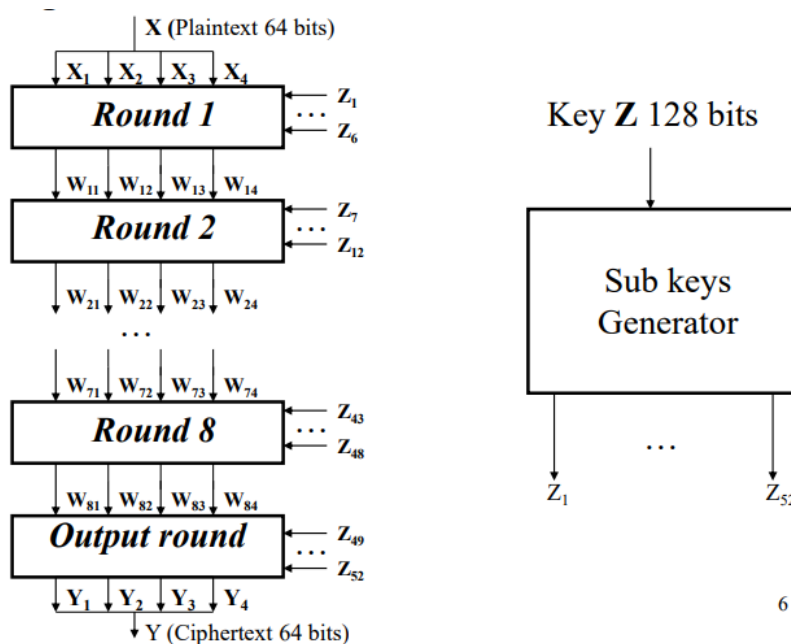
$$a \odot (b \boxtimes c) \neq (a \odot b) \boxtimes (a \odot c),$$

$$a \boxtimes (b \oplus c) \neq (a \boxtimes b) \oplus c.$$

Для достижения высокого уровня диффузии (постулат 4) строится мультипликативная аддитивная структура



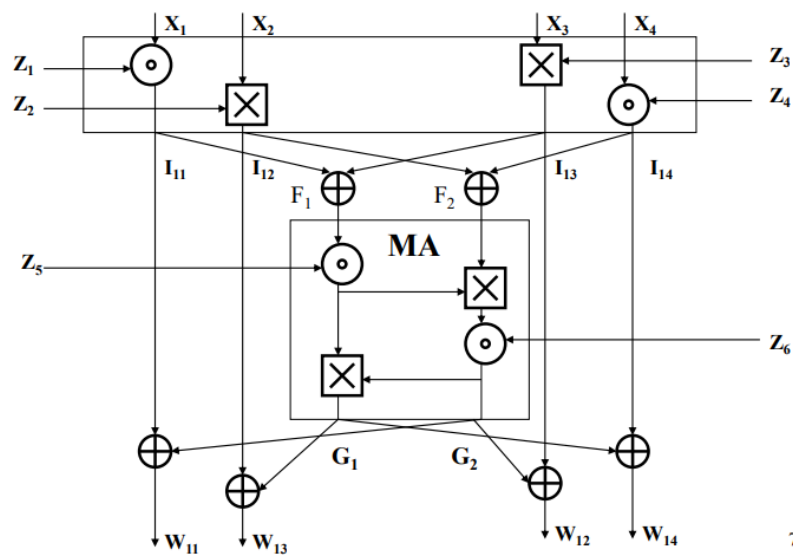
Общий алгоритм шифрования IDEA представляет собой:



Где $z_1 - z_{42}$ – идентификационные

$z_{43} - z_{52}$ – отличные

One round for ciphering (first)



7

Sub keys for ciphering

Round	Notation	Equivalent
#1	$Z_1 Z_2 Z_3 Z_4 Z_5 Z_6$	$Z[1 \dots 96]$
#2	$Z_7 Z_8 Z_9 Z_{10} Z_{11} Z_{12}$	$Z[97 \dots 128; 26 \dots 89]$
#3	$Z_{13} Z_{14} Z_{15} Z_{16} Z_{17} Z_{18}$	$Z[90 \dots 128; 1 \dots 25; 51 \dots 82]$
#4	$Z_{19} Z_{20} Z_{21} Z_{22} Z_{23} Z_{24}$	$Z[83 \dots 128; 1 \dots 50]$
#5	$Z_{25} Z_{26} Z_{27} Z_{28} Z_{29} Z_{30}$	$Z[76 \dots 128; 1 \dots 43]$
#6	$Z_{31} Z_{32} Z_{33} Z_{34} Z_{35} Z_{36}$	$Z[44 \dots 75; 101 \dots 128; 1 \dots 36]$
#7	$Z_{37} Z_{38} Z_{39} Z_{40} Z_{41} Z_{42}$	$Z[37 \dots 100; 126 \dots 128; 1 \dots 29]$
#8	$Z_{43} Z_{44} Z_{45} Z_{46} Z_{47} Z_{48}$	$Z[30 \dots 125]$
Output round	$Z_{49} Z_{50} Z_{51} Z_{52}$	$Z[23 \dots 86]$

Электронная цифровая подпись

Сравнения

Билет 13.

13.1. Шифратор Плейфер.

Шифр Плейфейра (Playfair Cipher), предложенный в 1854 году, является наиболее известным биграммным шифром замены. Данный шифр применялся в Великобритании во время первой мировой войны.

В отличие от всех ранее рассмотренных шифров, шифр Плайфейра основан на шифровании одновременно не одного, а двух символов, что также позволяет видоизменить частотные зависимости характерные для исходных текстов.

Основой данного шифра является шифрующая таблица со случайно расположенными буквами алфавита исходных текстов. Такая таблица представляет собой сеансовый ключ. Для удобства запоминания шифрующей таблицы подобно, как и для подстановочных таблиц используется ключевое слово или фраза, записываемая при заполнении начальных строк таблицы.

Для случая английского языка шифрующая таблица задается матрицей

Ключ шифра Плэйфэйра				
Y	A	R	M	O
L	I	K	B	C
D	E	F	G	H
N	P	Q	S	T
U	V	W	X	Z

5×5 , состоящей из 25 позиций. Позиция для символа J соответствует позиции для символа I .

Процедура шифрования включает следующие этапы.

1. Исходный текст M разбивается на пары символов $M = m_1 m_2 \dots$ (биграммы).

2. Последовательность биграмм $m_1 m_2 \dots$ открытого текста M преобразуется с помощью шифрующей таблицы в последовательность биграмм $c_1 c_2 \dots$ шифротекста C по следующим правилам.

2.1. Если буквы биграммы исходного текста m_1 и m_2 находятся в одной и той же строке шифрующей матрицы, то c_1 и c_2 представляют собой два символа справа от m_1 и m_2 , соответственно. Здесь первый столбец матрицы, является столбцом справа по отношению к последнему столбцу. Например, если $m_1 m_2 = EH$ то $c_1 c_2 = FD$.

2.2. Если m_1 и m_2 находятся в одном и том же столбце, то c_1 и c_2 принимают значения, соответствующие символам ниже m_1 и m_2 , соответственно. Первая строка считается строкой ниже последней. Например, если $m_1 m_2 = FW$ то $c_1 c_2 = QR$.

2.3. Если $m1$ и $m2$ находятся в различных строках и столбцах, то $c1$ и $c2$ соответствуют двум другим углам прямоугольника, имеющего $m1$ и $m2$, в качестве двух исходных углов, при этом $c1$ находится в той же строке что и $m1$, а $c2$ находится в той же строке что и $m2$. Например, если $m1m2=KT$, то $c1c2=CQ$ как видно из следующей диаграммы

Y	A	R	M	O
L	I	K	B	<u>C</u>
D	E	F	G	H
N	P	<u>Q</u>	S	T
U	V	W	X	Z

2.4. Если $m1=m2$ тогда пустой (нулевой) символ (например, X) вставляется в исходный текст между $m1$ и $m2$, чтобы устранить равенство $m1=m2$. Например, если $m1m2=KK$, тогда $m1m2m3=KXK$ и соответственно $c1c2=DW$.

13.2. Алгоритм BlowFish. Процесс инициализации P-массивов и S-матриц.
Структура алгоритма.

Блочный *шифратор* **BLOWFISH** принадлежит к классу стандартных симметричных шифраторов. Основные принципы BLOWFISH были опубликованы в 1994 Брюсом Шнейером (*Bruce Schneier*), как очередная альтернатива стандарту DES. При проектировании криптосистемы BLOWFISH было достигнуто выполнение следующих требований.

1. **Производительность.** Процедуры шифрования и дешифрирования требуют минимального процессорного времени. BLOWFISH кодирует 64 битные блоки данных на 32 разрядном микропроцессоре в течение 18 циклов на байт.

2. **Объем памяти.** BLOWFISH требует для своей реализации менее чем 5 Кбайт памяти.

3. **Простота реализации.** Структура BLOWFISH очень проста для реализации и в то же время обладает высокой криптостойкостью.

4. **Размерность ключа.** Длина ключа должна быть достаточной, чтобы противостоять так называемым прямым атакам путем перебора всех

возможных значений ключей. В то же время он должен обеспечивать высокую скорость работы процедур шифрования и дешифрирования. Вот почему BLOWFISH предлагает использование ключей различной длины. Длина колеблется от 32 до 448 бит

5 Длина блока данных. BLOWFISH работает с 64-битными блоками открытого текста и получает 64-битные зашифрованные блоки. В настоящее время BLOWFISH используется в огромном количестве программных продуктов и имеет прекрасные оценки специалистов.

Рассмотрим основные компоненты шифратора BLOWFISH.

Ключ. BLOWFISH позволяет использовать ключи длиной от одного 32 битного слова K_1 до четырнадцати 32 битных слов.

$$K_1, K_2, \dots, K_j, 1 \leq j \leq 14.$$

Подключи. Подключи генерируются на основании исходного ключа и хранятся в P -массиве состоящем из восемнадцати 32-битных слов

$$P_1, P_2, \dots, P_{18}.$$

S -матрицы. В шифраторе BLOWFISH определяются четыре S -матрицы. Каждая матрица состоит из 256 32-битных слов.

$$\begin{aligned} S_{1,0}, S_{1,1}, \dots, S_{1,255}, \\ S_{2,0}, S_{2,1}, \dots, S_{2,255}, \\ S_{3,0}, S_{3,1}, \dots, S_{3,255}, \\ S_{4,0}, S_{4,1}, \dots, S_{4,255}. \end{aligned}$$

Перед использованием шифратора BLOWFISH первоначально вычисляются значения подключей P и S -матриц. Алгоритм вычисления итерационных ключей и S -матриц состоит из следующих этапов.

1. Для инициализации P -массива и S -матриц используется число π таким образом, что P_1 равняется 32 битам числа π (дробной его части), следующие 32 бита присваиваются P_2 и так далее. Вначале инициализируется P -массив, затем последовательно четыре S -матрицы. В шестнадцатеричной системе счисления это выглядит следующим образом: $P_1=243F6A88$, $P_2=85A308D3$, ..., $S_{4,254}=57FDFE3$, $S_{4,255}=3AC372E6$.

2. Выполняется побитовая XOR операция между P -массивом и K -массивом, повторяя значение ключа, если это необходимо. Например, для максимальной длины ключа имеем $P_1=P_1 \oplus K_1$, $P_2=P_2 \oplus K_2$, ..., $P_{14}=P_{14} \oplus K_{14}$, $P_{15}=P_{15} \oplus K_1$, ..., $P_{18}=P_{18} \oplus K_4$.

3. Основываясь на текущих значениях массива P и S -матриц, шифруется 64 битный блок данных, состоящий из 64 нулей. Затем 64 бита зашифрованных данных используются в качестве новых значений P_1 и P_2 , а также как незашифрованный блок данных для следующей итерации шифрования. Новые значения P и массивов S используются в последующих итерациях шифрования для получения новых значений для $P_3, P_4, \dots, P_{18}, S_{1,0}, S_{1,1}, S_{1,2}, \dots, S_{4,254}, S_{4,255}$. Эта процедура может быть представлена как $P_1, P_2 = EP, S[0]$, $P_3, P_4 = EP, S[P_1 || P_2]$, $P_5, P_6 = EP, S[P_3 || P_4]$, ...,

$P_{17}, P_{18} = EP, S [P_{15} || P_{16}], S_{1,0}, S_{1,1} = EP, S [P_{17} || P_{18}], S_{1,2}, S_{1,3} = EP, S [S_{1,0} || S_{1,1}], \dots, S_{4,254}, S_{4,255} = EP, S [S_{4,252} || S_{4,253}].$

Структура шифратора BLOWFISH во многом повторяет структуру стандарта DES. Он также состоит из последовательно выполняемых итераций, на каждой из которых используются значения итерационных ключей.

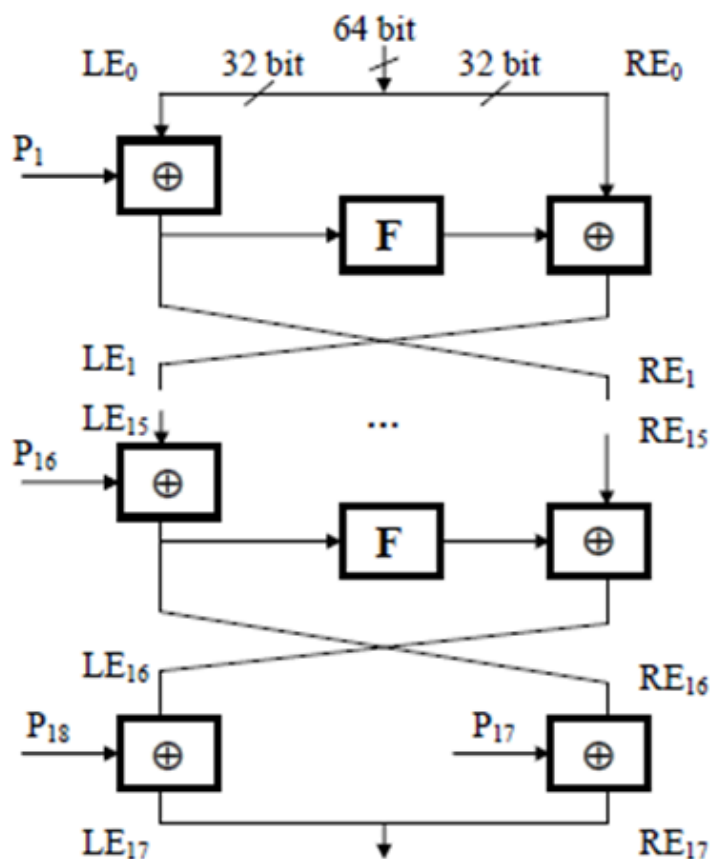


Рис.4.11. Процедуры шифрования BLOWFISH

Подобно, как и в DES исходный блок шифруемых данных разбивается на две части: правую RE_0 и левую LE_0 части по 32 бита. Затем левая часть LE_0 поразрядно складывается по модулю два со значением первого подключа P_1 . Результат операции сложения является аргументом функции F , значение которой суммируется с правой частью RE_0 . Подобная последовательность операций повторяется для последующих итераций. Аналитически шифрование можно описать в виде следующего алгоритма

```

For i=1 to 16 do
     $RE_i = LE_{i-1} \oplus P_i;$ 
     $LE_i = F[RE_i] \oplus RE_{i-1};$ 
 $LE_{17} = RE_{16} \oplus P_{18};$ 
 $RE_{17} = LE_{16} \oplus P_{17}.$ 

```

При дешифрировании подключи P используются в обратной последовательности в соответствии со следующей блок схемой.

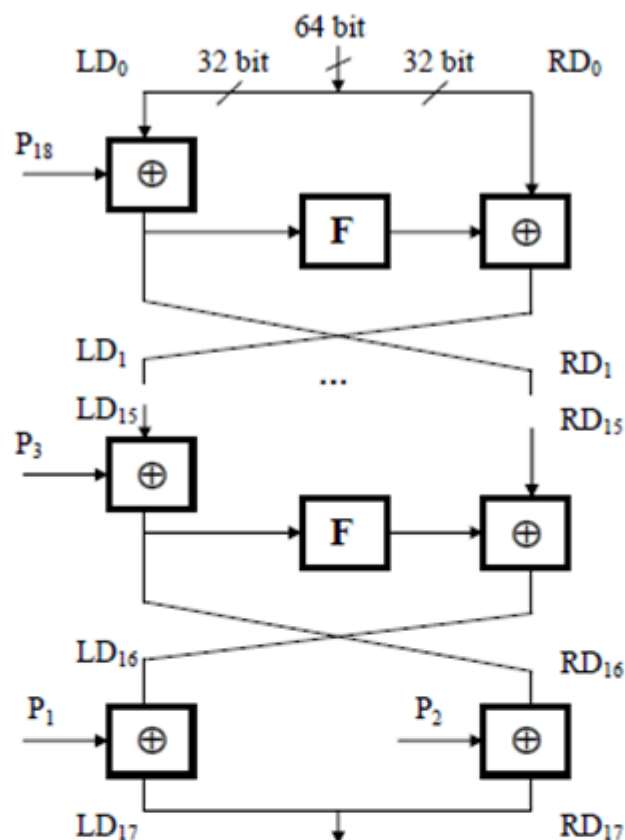


Рис.4.12. Процедуры дешифрирования BLOWFISH

Аналитически шифрование можно описать в виде следующего алгоритма.

```

For i=1 to 16 do
     $RD_i = LD_{i-1} \oplus P_{19-i};$ 
     $LD_i = F[RD_i] \oplus RD_{i-1};$ 
 $LD_{17} = RD_{16} \oplus P_1;$ 
 $RD_{17} = LD_{16} \oplus P_2;$ 

```

Детальная структурная схема реализации одной итерации BLOWFISH приведена на рис.4.13.

Штрих-пунктиром на приведенном рисунке показана функция F , которая является основным элементом каждой итерации. Четыре S -матрицы используются как таблицы подстановки.

Для реализации BLOWFISH необходимо использовать 2 операции: операцию сложение (+) по модулю 2^{16} ; побитовое исключающее или ($\oplus$ -сложение по модулю два).

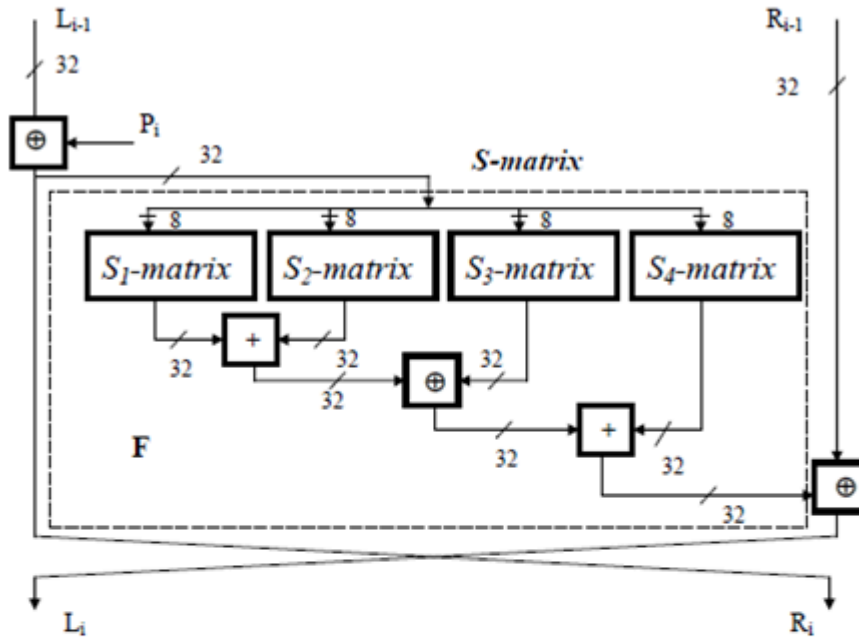


Рис.4.13. Структурная схема реализации одной итерации BLOWFISH

Все итерации в процессе шифрования и дешифрования включают составную функцию F , основным элементом которой является операция подстановки, определенная S -матрицами.

Первоначально 32 битные входные значения делятся на 4 части a, b, c, d по 8 бит каждая. В соответствии с 8-битным кодом, являющимся индексом элемента S -матрицы, получается новое 32-битное значение. Четыре S -матрицы являются генераторами операндов для реализации функции F . Общее выражение для этой функции имеет вид.

$$F(a,b,c,d)=(S1,a+S2,b) \oplus S3,c+S4,d.$$

BLOWFISH имеет следующие достоинства.

1. В сравнении с DES алгоритмом S -матрицы для BLOWFISH зависят от сеансового ключа и поэтому всегда их элементы имеют новые значения.
2. В каждой итерации подвергаются преобразованию данные как левой, так и правой частей блока.

3. Скорость шифрования и дешифрования для BLOWFISH самая высокая в сравнении с классическими симметричными алгоритмами.

В таблице 4.12 приведена сравнительная характеристика эффективности алгоритмов шифрования рассмотренных в предыдущих разделах данной главы.

Таблица 4.10.

Сравнительная характеристика алгоритмов шифрования

Алгоритм	Количество циклов на одну итерацию	Количество итераций	Количество циклов для шифрования блока данных
BLOWFISH	9	16	144
DES	18	16	288
IDEA	50	8	400
Triple DES	18	48	856

Анализ приведенной таблицы показывает высокую эффективность рассмотренных алгоритмов, которые являются основой подавляющего большинства реальных приложений, использующих криптографические алгоритмы.

13.3. Потокковые системы шифрования.

Характерной особенностью потоковых шифраторов является использование при их реализации криптографических ключей большой размерности часто равной длине исходного текста.

Различают два типа потоковых шифров *синхронные* (*synchronous*) и *самосинхронизирующиеся* (*self-synchronizing*) шифраторы. В первом случае последовательность значений ключа является независимой от открытого текста и шифротекста. Понятие синхронный шифратор, связано с тем, что для успешного дешифрования требуется синхронизация между последовательностью значений ключа и зашифрованным текстом. В то время как для самосинхронизирующихся шифраторов значения ключа зависят либо от исходного текста, либо от шифротекста. Обобщенная структура синхронного потокового шифратора приведена на рис.5.1.

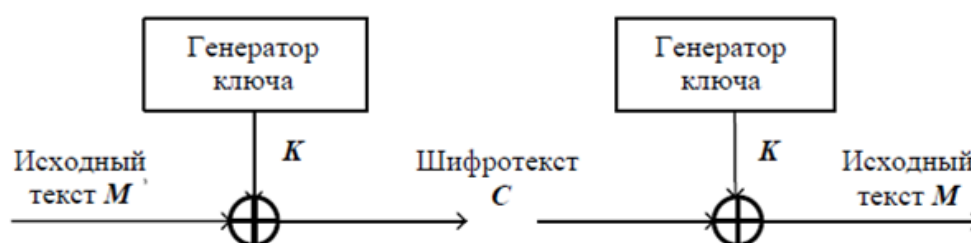


Рис.5.1. Структурная схема синхронного потокового шифратора

Подобная структура шифратора позволяет достичь высокой помехозащищенности и надежности функционирования. Действительно искажение одного символа в зашифрованном тексте приведет к искажению только одного символа исходного текста при дешифрировании. В тоже время нарушение синхронизации является причиной искажения всех последующих символов исходного текста с момента потери синхронного формирования ключа с шифротекстом. Подобная ситуация может возникнуть при удалении либо добавлении символов шифротекста.

С целью восстановления синхронизации используются самосинхронизирующиеся потоковые шифраторы. При их использовании ошибочно добавленный или удаленный бит вызывает только ограниченное количество ошибочных символов в дешифрированном тексте, после чего правильный текст восстанавливается.

Различают принципиальные отличия между симметричными блочными шифраторами (DES, IDEA, BLOWFISH, ...) и потоковыми системами шифрования.

96

1.Блочный шифр одновременно шифрует целый блок данных, как правило, 64 бита, а в потоковых шифраторах процедура шифрования и дешифрирования выполняется поразрядно.

2.В блочном шифре, каждый бит зашифрованного текста является сложной функцией открытого текста, и криптографического ключа. В потоковом шифраторе (см. рис.5.1) каждый бит шифротекста получается в результате поразрядной операции сложения по модулю два очередного бита открытого текста и бита ключа.

3.Блочный шифр не требует задания начального вектора для генератора последовательности символов ключа, а в потоковом шифраторе такой вектор необходим|

4. Блочные шифры подобно как DES, используется в коммерческом секторе, а потоковые шифры используется для специальных приложений и, как правило, широко не обсуждаются.

13.4. Криптосистема типа рюкзак №2.

Видоизменён вид ключа: ограничение на элементы ключа. I-тый элемент ключа должен быть > суммы предыдущих: . Тогда полученный шифротекст состоит из:.

$$c_1 = k_1 x_1,$$

$$c_2 = k_1 x_1 + k_2 x_2,$$

...

$$C = c_n = k_1 x_1 + k_2 x_2 + \dots + k_n x_n$$

На первом этапе $C = c_n$ сравнивается с ключом k_n . Если $c_n < k_n$, $c_{n-1} = c_n$ и $x_n = 0$. Если $c_n \geq k_n$, $c_{n-1} = c_n - k_n$ и $x_n = 1$. Получив c_{n-1} , сравниваем c_{n-1} с k_{n-1} и всё повторяем.

Пример: $M = \{11001\}$, $K = \{151, 187, 426, 1091, 2412\}$,

$C = K \times M = 1 \times 151 + 1 \times 187 + 0 \times 426 + 0 \times 1091 + 1 \times 2412 = 2750$ - посылается получателю, у которого такой же ключ.

Поскольку $C = c_5 = 2750$ является зашифрованным текстом, восстановление x_5 из y_5 и k_5 равно $x_5 = 1$, поскольку $c_5 = 2750 > k_5 = 2412$.

Следовательно, процедура восстановления x_i для $i = 1, 2, \dots, 5$ будет продолжаться, как показано ниже.

$$y_5 = 2750 > k_5 = 2412, x_5 = 1;$$

$$y_4 = y_5 - k_5 = 338 < k_4 = 1091, x_4 = 0;$$

$$y_3 = 338 < k_3 = 426, x_3 = 0;$$

$$y_2 = 338 > k_2 = 187, x_2 = 1;$$

$$y_1 = y_2 - k_2 = 151 = k_1, x_1 = 1.$$

Таким образом, восстановленный текст имеет вид $M = \{11001\}$, как и ожидалось.

Однако этот простой ранцевый вектор K нельзя использовать в качестве открытого ключа шифрования, потому что кто-то может легко восстановить M из C .

БИЛЕТ 15

1. Шифры, использующие коды переменной длины на примере VIC-шифратора

2. Цифровая подпись, основанная на симметричных алгоритмах
3. PUF типа Кольцевой осциллятор (Ring Oscillator)
4. Алгоритм децимаций. Определение криптографического ключа для процедуры дешифрования

Шифры, использующие коды переменной длины на примере VIC-шифратора

Шифр VIC – это сложный шифр, выданный Советским Союзом, по крайней мере, одному из своих шпионов. Он представляет интерес, поскольку, несмотря на то, что он написан карандашом и бумагой, он кажется очень надежным.

Это был шифр, которым было написано сообщение, найденное на кусочке микрофильма внутри выдолбленной пятицентовой монеты разносчиком газет в 1953 году. Шифр VIC, который мы продемонстрируем здесь, адаптированный для отправки сообщений на английском языке, начинается с процедуры получения десяти псевдослучайных цифр. Агент должен запомнить:

1. Шесть цифр (которые были в форме даты);
2. Первые 20 букв ключевой фразы (которая была началом популярной песни или крылатой фразы);
3. Пять случайных цифр для использования в качестве индикатора сообщения.

Пусть датой будет 4 июля 1776 года (04.07.1776), чтобы получить цифры 741776. (На самом деле, русские использовали свою обычную форму дат, с указанием месяца на втором месте.)

И пусть случайно выбранным идентификатором будет 77651.

Алгоритм

1. Вычитаем из идентификатора значение даты (без переноса единицы десятков в случае вычитания из меньшего большего)

7	7	6	5	1
7	4	1	7	7
--	--	--	--	--
0	3	5	8	4

2. Ключевую фразу сокращаем до 20 символов без учета знаков препинания и пробелов. Делим полученную фразу на 2 равные части по 10 символов. Каждой букве присваивается числовое значение от 0 до 9 (0 после 9) в соответствии с положением буквы в алфавите

ссылка:

I	D	R	E	A	M	O	F	J	E		A	N	N	I	E	W	I	T	H	T
6	2	0	3	1	8	9	5	7	4		1	6	7	4	2	0	5	8	3	9

3. Расширяем полученное на шаге 1 значение (03584) до 10-значного числа путем цепного сложения. (первые значения остаются, а после них записываются

значения по т. Фибоначчи этих же 5 цифр: $0+3=3$, $3+5=8$, $8+4=12$ (десятку откидываем, получаем 2), $4+3=7$). Полученное значение имеет вид
03584 38327

4. Полученное на шаге 3 значение прибавляем к числу, которое получено на шаге 2 под левой частью фразы (6203189574)

$$\begin{array}{r} 6203189574+ \\ 0358438327= \\ \hline 6551517891. \end{array}$$

5. Складываем

Index	<i>1 2 3 4 5 6 7 8 9 0</i>
Second half	<i>1 6 7 4 2 0 5 8 3 9</i>
Ten-digits result	<i>6 5 5 1 5 1 7 8 9 1</i>
Result	0 2 2 1 2 1 5 8 3 1

6. На базе полученного числа на шаге 5 формируем 50 псевдо случайных значений по такому же принципу

pseudorandom digits for use in encipherment: 0 2 2 1 2 1 5 8 3 1 * 2 4
3 3 3 6 3 1 4 3 * 6 7 6 6 9 9 4 5 7 9 * 3 3 2 5 8 3 9 2 6 2 * 6 5 7 3 1 2 1
8 8 8 * 1 2 0 4 3 3 9 6 6 9

7. последние 10 значений из полученного ключа (1204339669) используются как буквы в ключевом слове для транспозиции значений от 0 до 9 (0 поле 9):

1204339669 => 1205348679

(формируется второе значение так же как и значения на шаге 2, вместо букв используются числа из первого значения)

Данное значение (1205348679) используется как обозначение колонок и строк шахматной доски.

	1 2 0 5 3 4 8 6 7 9
	A T O N E S I R
0	B C D F G H J K L M
8	P Q U V W X Y Z . /

8. Получаем таблицу подстановки.

{ [Норм сайтик почитать этот алгос еще раз если непонятно описано тут](#) }

Цифровая подпись, основанная на симметричных алгоритмах

PUF типа Кольцевой осциллятор (Ring Oscillator)

Алгоритм децимаций. Определение криптографического ключа для процедуры дешифрования

БИЛЕТ 16

5. Современная роторная машина
6. Регистры сдвига с линейной обратной связью двух типов
7. Однонаправленные хэш-функции
8. Аутентификация ключей

Современная роторная машина

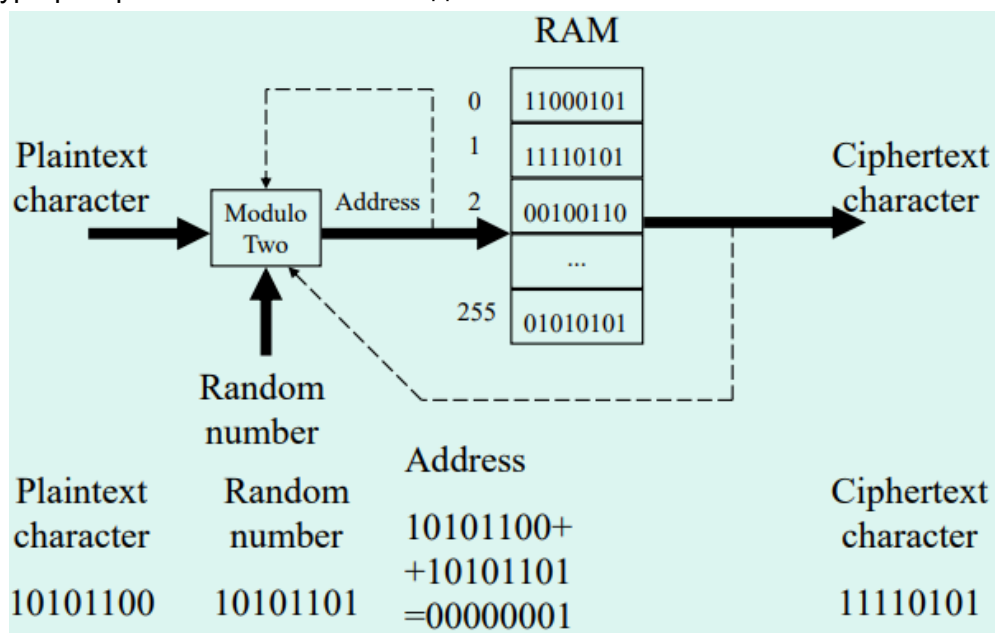
Основа на ОЗУ

Адреса	ОЗУ (Таблица подстановки)
000	101
001	111
010	001
011	010
100	011
101	000
110	100
111	110

В качестве ротера ОЗУ, хранящего 8-битные значения – ???

Для имитации сдвига на роторной машинах – добавление к шифросимволу заранее predetermined (псевдослучайных, известных и получателю, и отправителю) значений.

Структура роторной машины имеет вид.

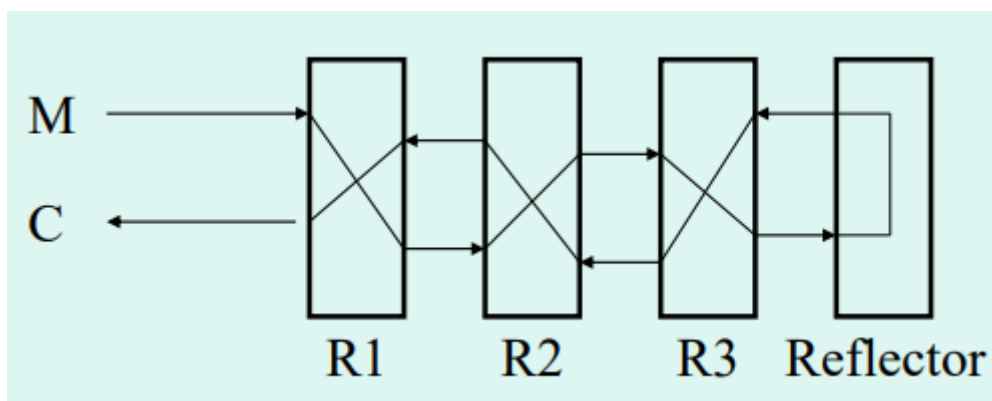


+ В билетах нет просто роторной машины (на примере Энигмы), есть мнение, что ее работу надо описывать в этом билете, так что на всякий случай:

Основа в использовании множества таблиц подстановки (в идеале каждой букве своя таблица).

Представляет собой электрический ротор: два вращающихся относительно друг друга цилиндра, связанного контактами.

На практике: несколько последовательно связанных роторов (пример – Энигма).



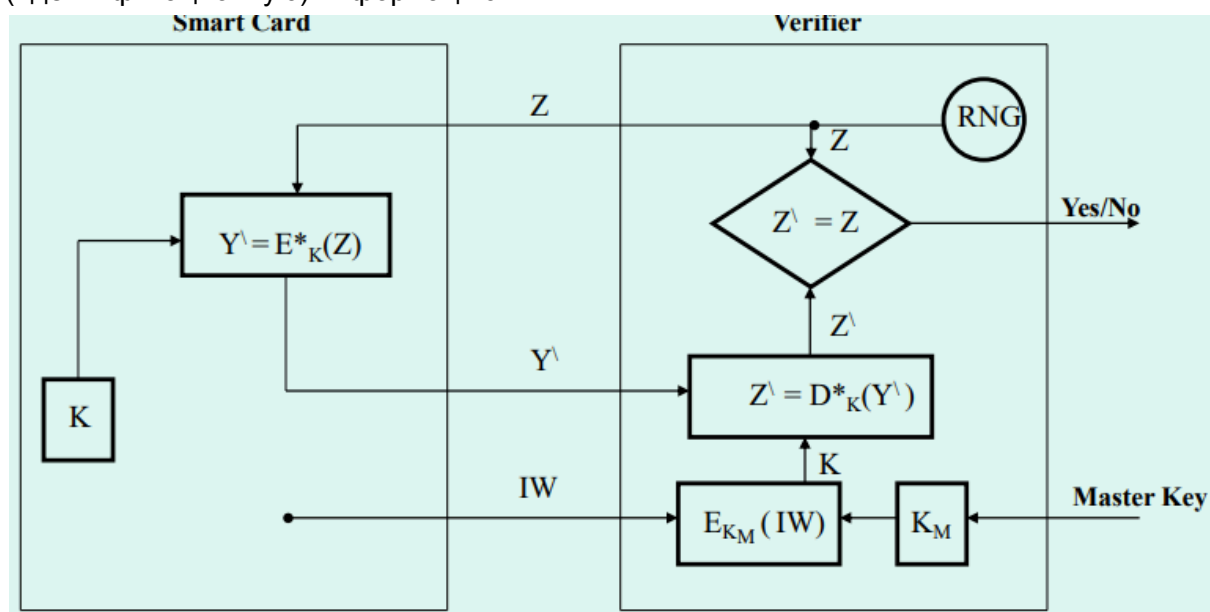
При таком шифровании число комбинаций для английского алфавита составит 26^n , при этом новых уникальных таблиц подстановки будет только 26! Оптимальным значением для n считается примерно 6, тк $26^6 \approx 26!$

Регистры сдвига с линейной обратной связью двух типов

Однонаправленные хэш-функции

Аутентификация ключей

При выдаче банковской карты каждый владелец предоставляет личную (идентификационную) информацию



Z — псевдослучайное число

$K = E_{K_M}^*(IW)$, где K_M — главный ключ (master key)

Алгоритм:

1. Процессор высчитывает Z (вычисление определяется мощностью процессора карты)
2. На основе секретного ключа (K) получает шифротекст $Y' = E_K^*(Z)$ (оно же зашифрованное Z). Параллельно посылает IW (идентификационные данные) от карты в банкомат

3. Верификатор на основе IW и K_M генерирует E_{K_M}
4. Дешифрование $Z' \Rightarrow$ Если $Z' = Z$, то высылает ответ да/нет (действительна ли карта или нет соответственно)

БИЛЕТ 17

1. Криптосистема RSA
2. Свойства M-последовательностей
3. Шифратор Playfair. Модификация шифратора Playfair.
4. Иерархия криптографических ключей. Методы генерирования ключей.

Криптосистема RSA

Алгоритм RSA (по первым буквам фамилий его создателей Rivest-Shamir-Adleman) основан на свойствах простых чисел, причем очень больших простых чисел. В RSA, модуль n представляет собой составное число как произведение двух больших простых чисел p и q , то есть $n=pq$.

n - открыт, а p и q - закрытые большие числа.

Шифрование: для того, чтобы зашифровать сообщение с использованием открытого ключа (e, n) :

- Вычисляется шифротекст: $C = M^e \bmod n$.

Дешифрирование: для того, чтобы дешифровать сообщение с использованием закрытого ключа (d, n) :

- Вычисляется исходный текст: $M = C^d \bmod n$.

d - случайное число (но вроде как оно взаимно простое с $\Psi(n)$)

$$\Psi(n) = (p - 1)(q - 1)$$

$$e = \text{inv}(d, \Psi(n)).$$

e выбирается таким образом, что:

1. оно взаимно простое с $\Psi(n)$
2. маленького размера
3. в RSA часто используются такие значения как 3, 17 и 65537

Пример:

$$p = 5, q = 7, n = 35;$$

$$\Psi(n) = 4 \cdot 6 = 24;$$

$$d \cdot e \bmod \Psi(n) = 1;$$

$$\text{или } e = \text{inv}(d, \Psi(n))$$

Свойства M-последовательностей

1. Существует $\Psi(2^m - 1)/m$ примитивных полиномов для заданного значения его старшей степени m , где Ψ есть функция Эйлера. Так, например, для $m = 3$
 $\Psi(2^m - 1)/m = \Psi(2^3 - 1)/3 = 6/3 = 2$. Таким образом, для $m = 3$ существует два примитивных полинома $\varphi(x) = 1 + x + x^3$ и $\varphi(x) = 1 + x^2 + x^3$

2. Для каждого заданного полинома $\varphi(x)$ существует инверсный полином $\varphi(x)^{-1}$, который может быть изучен благодаря следующему отношению:

$$\varphi(x)^{-1} = x^m * \varphi(x^{-1})$$

3. Период M-последовательности зависит от степени m примитивного порождающего полинома и равняется:

$$L = 2^m - 1$$

4. Для заданного полинома $\varphi(x)$ существует L различных M последовательностей, каждый из которых отличается фазовым сдвигом.
5. M последовательность представляет собой псевдослучайную последовательность, в которой вероятность появления нулей и единиц определяется соотношением:

$$p(a_k=1) = \frac{2^{m-1}}{2^m-1} = \frac{1}{2} + \frac{1}{2^{m+1}-2};$$

$$p(a_k=0) = \frac{2^{m-1}-1}{2^m-1} = \frac{1}{2} - \frac{1}{2^{m+1}-2};$$

Шифратор Playfair. Модификация шифратора Playfair.

Шифр Плейфейра (Playfair Cipher), предложенный в 1854 году, является наиболее известным биграммным шифром замены. Данный шифр применялся в Великобритании во время первой мировой войны. В отличие от всех ранее рассмотренных шифров, шифр Плейфейра основан на шифровании одновременно не одного, а двух символов, что также позволяет видоизменить частотные зависимости характерные для исходных текстов.

Основой данного шифра является шифрующая таблица со случайно расположенными буквами алфавита исходных текстов. Такая таблица представляет собой сеансовый ключ. Для удобства запоминания шифрующей таблицы подобно, как и для подстановочных таблиц используется ключевое слово или фраза, записываемая при заполнении начальных строк таблицы.

Для случая английского языка шифрующая таблица задается матрицей

Ключ шифра Плэйфэйра				
Y	A	R	M	O
L	I	K	B	C
D	E	F	G	H
N	P	Q	S	T
U	V	W	X	Z

55, состоящей из 25 позиций. Позиция для символа J соответствует позиции для символа I.

Процедура шифрования включает следующие этапы:

1. Исходный текст M разбивается на пары символов $M=m_1m_2...$ (биграммы).
2. Последовательность биграмм $m_1m_2...$ открытого текста M преобразуется с помощью шифрующей таблицы в последовательность биграмм $c_1c_2...$ шифротекста C по следующим правилам.

3. 2.1. Если буквы биграммы исходного текста m_1 и m_2 находятся в одной и той же строке шифрующей матрицы, то c_1 и c_2 представляют собой два символа справа от m_1 и m_2 , соответственно. Здесь первый столбец матрицы, является столбцом справа по отношению к последнему столбцу. Например, если $m_1m_2=EH$ то $c_1c_2=FD$.
4. 2.2. Если m_1 и m_2 находятся в одном и том же столбце, то c_1 и c_2 принимают значения, соответствующие символам ниже m_1 и m_2 , соответственно. Первая строка считается строкой ниже последней. Например, если $m_1m_2=FW$ то $c_1c_2=QR$.
5. 2.3. Если m_1 и m_2 находятся в различных строках и столбцах, то c_1 и c_2 соответствуют двум другим углам прямоугольника, имеющего m_1 и m_2 , в качестве двух исходных углов, при этом c_1 находится в той же строке что и m_1 , а c_2 находится в той же строке что и m_2 . Например, если $m_1m_2=KT$, то $c_1c_2=CQ$ как видно из следующей диаграммы

Y	A	R	M	O
L	I	K	B	<u>C</u>
D	E	F	G	H
N	P	<u>Q</u>	S	T
U	V	W	X	Z

6. 2.4. Если $m_1=m_2$ тогда пустой (нулевой) символ (например, X) вставляется в исходный текст между m_1 и m_2 , чтобы устранить равенство $m_1=m_2$. Например, если $m_1m_2=KK$, тогда $m_1m_2m_3=KXK$ и соответственно $c_1c_2=DW$.
7. 2.5. Если исходный текст имеет нечетное число знаков, пустой символ добавляется в конец текста для получения четного числа символов исходного текста.

Иерархия криптографических ключей. Методы генерирования ключей

Иерархия криптографических ключей:

1. главный ключ (как правило один)
2. ключ для шифрования ключей
3. сеансовые ключи

БИЛЕТ 18

1. Симметричные криптосистемы. Lucifer.
2. Пример потоковой криптосистемы на базе M последовательностей.
3. Скорость языка.
4. Слепая подпись.

Симметричные криптосистемы. Lucifer

Симметричные криптосистемы — это методы шифрования, в которых один и тот же ключ используется для шифрования и расшифрования данных (или они легко получаются друг из друга). Эти системы отличаются от асимметричных криптосистем, где используются два различных ключа: открытый и закрытый. Симметричные криптосистемы известны своей скоростью и эффективностью и широко применяются в различных областях, требующих быстрой обработки данных.

Идея комбинированного использования операций подстановки и перестановки принадлежит К.Шеннону и является средством для достижения высокого качества криптосистем, хорошей аппроксимации абсолютной секретности. Совместное применение подстановок и перестановок позволяет получить "комбинированное преобразование", которое произвольно распределяет исходные сообщения равномерно по набору всех возможных зашифрованных сообщений. Смешанные преобразования могут быть созданы, например, путём применения преобразования состоящего в последовательном применении перестановок и подстановок. Хорошее качество достигается путем итерационного применения идентичных подстановочно-перестановочных преобразований.

Впервые этот подход реализован в виде шифра LUCIFER, созданного на фирме IBM Фейстлом (Feistl). LUCIFER использует преобразование исходного сообщения, попеременно применяя подстановки и перестановки.

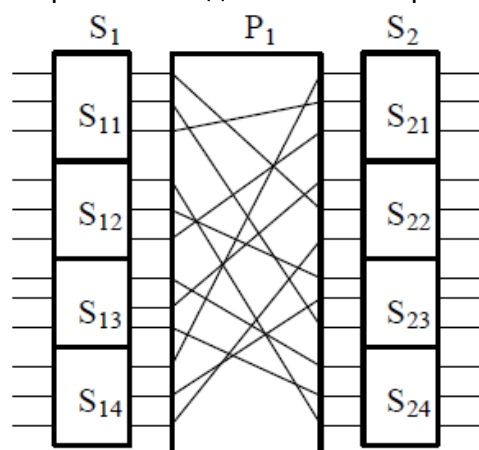


Рис.4.1. Структурная схема шифратора LUCIFER

Процедура подстановки шифруемого блока S_i из 12 бит состоит из 4 операций подстановки $S_{i1}, \dots, S_{i4}$, меньшей размерности, каждая из которых управляет трехбитным блоком данных. Это необходимо чтобы уменьшить сложность операций подстановки, которая реализуется с помощью таблиц подстановки. Перестановка P_i переставляет (перемешивает) все двенадцать

бит, полученные как результат подстановки, в некоторой случайной последовательности. Например, для подстановки P1 первый символ переставляется на пятую позицию, второй, на девятую, третий на вторую и так далее. Таким образом, последовательно выполняется несколько итераций для достижения хорошего качества шифрования.

Пример потоковой криптосистемы на базе M последовательностей

Скорость языка

Скорость (частота) языка - значение энтропии на один символ.

Пусть для заданного языка определено множество сообщений длиной N символов, тогда **частота (скорость) языка** для сообщений X длиной N определяется как

$$r = H(X)/N,$$

где величина r определяет среднее число бит информации в каждом символе сообщения.

Простейший способ определения частоты языка (**абсолютной частоты R**), основывается на предположении, что все буквы алфавита языка имеют одинаковую вероятность появления во всех возможных сообщениях так же, как и всевозможные последовательности букв алфавита в сообщениях равновероятны. Если алфавит языка содержит L букв, тогда **абсолютная частота** может быть получена так:

$$R = \log_2 L,$$

Для английского языка $L=26$, тогда $R = \log_2 L = \log_2 26 = 4,7$.

Абсолютная частота языка $R=4,7$ определяет максимальное число бит информации, которое может быть получено при получении одной буквы сообщения. Или, максимальное количество бит, необходимых для кодирования сообщения, представленного на английском языке.

Частота букв русского языка

А	0.062	Л	0.053	Ц	0.004
Б	0.014	М	0.026	Ч	0.012
В	0.038	Н	0.053	Ш	0.006
Г	0.013	О	0.090	Щ	0.003
Д	0.025	П	0.023	Ы	0.016
Е	0.072	Р	0.040	Ь, Ь	0.014
Ж	0.007	С	0.045	Э	0.003
З	0.016	Т	0.053	Ю	0.006
И	0.062	У	0.021	Я	0.018
Й	0.010	Ф	0.002		
К	0.028	Х	0.009		

Частота букв английского языка

A	0.0804	B	0.0154	C	0.0306
D	0.0399	E	0.1251	F	0.0230
G	0.0196	H	0.0554	I	0.0726
J	0.0016	K	0.0067	L	0.0414
M	0.0253	N	0.0709	O	0.0760
P	0.0200	Q	0.0011	R	0.0612
S	0.0654	T	0.0925	U	0.0271
V	0.0099	W	0.0192	X	0.0019
Y	0.0173	Z	0.0009		

Слепая подпись

Основой является затемняющий фактор. Перед отправкой документа автор затемняет текст. Получатель подписывает затемненные текст, а автор, получив документ обратно, снимает затемнение.

1. Подготовительная фаза

Нотариус N формирует открытый ключ (D, n) и секретный ключ (E, n), где $n = pq$

2. Формирование подписи

Автор генерирует r ; $(r, n) = 1$

Вычисляет затемняющий фактор: $r^D \bmod n$

3. Автор выполняет затемнение текста: $m^* = m r^D \bmod n$

4. Автор посылает m^* к нотариусу

5. Нотариус ставит подпись под m^* :

$$S^* = (m^*)^E \bmod n$$

6. Подписанный текст отправляется к автору

7. Автор удаляет затемняющий фактор, чтобы получить реальную подпись S под своим документом:

$$S = S^* r^{-1} \bmod n = m^E \bmod n$$

БИЛЕТ 19

1. Введение в теорию чисел. Теорема евклида. Теорема 2. Теорема 3.
2. Недостаток синхронных потоковых криптосистем. Процедура их взлома.
3. SHA хэш алгоритм.
4. Аутентификация ключей.

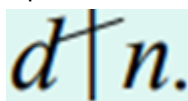
Введение в теорию чисел. Теорема евклида. Теорема 2. Теорема 3

Все множество натуральных чисел состоит из двух подмножеств: действительных чисел и целых чисел. Основным подмножеством чисел, используемых в области защиты данных, являются целые числа.

Число P является простым, если $P > 1$ и делится только на 1 и самого себя

Целое число d является делителем n тогда и только тогда, когда $n = kd$.

$d \mid n$ – d есть делитель числа n



– d не является делителем числа n

Теорема 1. (Эвклида). Существует бесконечное множество простых чисел.

Доказательство: Предположим, что это множество конечно и состоит из простых чисел $p_1, p_2, p_3, \dots, p_k$, тогда получили противоречие, заключающееся в том, что число не делится ни на одно простое число $p_1, p_2, p_3, \dots, p_k$, тогда как оно делится на 1 и на самого себя.

Теорема 2. Сколько большим не было бы k , ($k > 1$), оказывается возможным на числовой оси найти k последовательно идущих друг за другом составных чисел.

Доказательство: $(k + 1)! = 2 * 3 * 4 \dots * (k + 1)$ делится на любое число 2, 3... $k + 1$.

Рассмотрим последовательность $(k + 1)! + 2, (k + 1)! + 3, \dots (k + 1)! + (k + 1)$, все числа являются составными из-за того факта, что первое число кратно 2, второе трем и т.д

Теорема 3. Если $n > 1$ не делится ни на одно простое число, то оно является простым.

Доказательство: Предположим, что $n = a * b$. Можно утверждать следующее, что a и b не могут быть одновременно больше .

Недостаток синхронных потоковых криптосистем. Процедура их взлома

Синхронные потоковые криптосистемы на базе LFSR (Linear feedback shift register) имеют 2 недостатка:

- Легко взламываются
- Ненадежны в плане получения корректного текста у получателя из-за нарушения синхронизации процесса

Взламываются с помощью линейных уравнений вида:

$$\begin{aligned}
k_{m+1} &= \alpha_1 k_m \oplus \alpha_2 k_{m-1} \oplus \alpha_3 k_{m-2} \oplus \dots \oplus \alpha_m k_1; \\
k_{m+2} &= \alpha_1 k_{m+1} \oplus \alpha_2 k_m \oplus \alpha_3 k_{m-1} \oplus \dots \oplus \alpha_m k_2; \\
k_{m+3} &= \alpha_1 k_{m+2} \oplus \alpha_2 k_{m+1} \oplus \alpha_3 k_m \oplus \dots \oplus \alpha_m k_3; \\
&\dots \\
k_{2m} &= \alpha_1 k_{2m-1} \oplus \alpha_2 k_{2m-2} \oplus \alpha_3 k_{2m-3} \oplus \dots \oplus \alpha_m k_m;
\end{aligned}$$

Основой построения линейных уравнений является нахождение 2m бит исходного текста, который соответствует 2m битам шифротекста. Здесь коэффициент α_m , так же, как и α_0 для любого полинома равен единице

Пример взлома: $M = 10101111$; $C = 11010101$. K – поток ключа

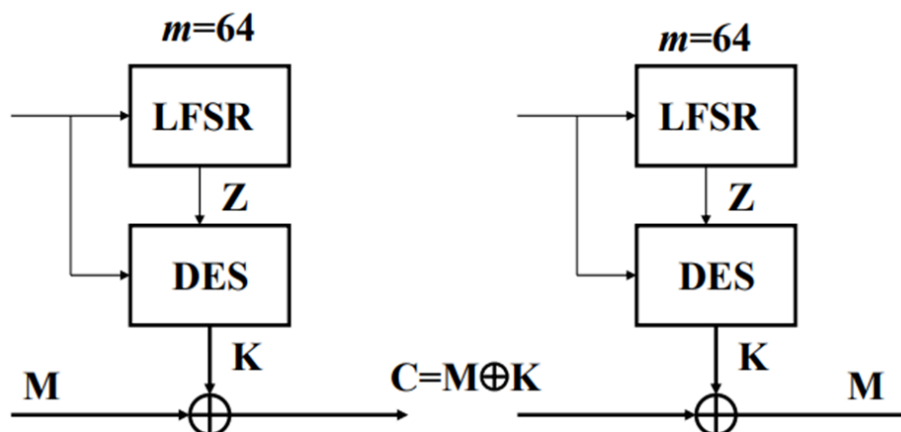
$M =$	1	1	0	1	0	1	0	1		$k_5 = \alpha_1 k_4 \oplus \alpha_2 k_3 \oplus \alpha_3 k_2 \oplus \alpha_4 k_1;$
$C =$	1	0	1	0	1	1	1	1	$\Rightarrow$	$k_6 = \alpha_1 k_5 \oplus \alpha_2 k_4 \oplus \alpha_3 k_3 \oplus \alpha_4 k_2;$
$K =$	0	1	1	1	1	0	1	0		$k_7 = \alpha_1 k_6 \oplus \alpha_2 k_5 \oplus \alpha_3 k_4 \oplus \alpha_4 k_3;$
	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8		$k_8 = \alpha_1 k_7 \oplus \alpha_2 k_6 \oplus \alpha_3 k_5 \oplus \alpha_4 k_4;$

$$\begin{aligned}
1 &= \alpha_1 \oplus \alpha_2 \oplus \alpha_3; \\
\Rightarrow 0 &= \alpha_1 \oplus \alpha_2 \oplus \alpha_3 \oplus \alpha_4; \\
1 &= \alpha_2 \oplus \alpha_3 \oplus \alpha_4; \\
0 &= \alpha_1 \oplus \alpha_3 \oplus \alpha_4;
\end{aligned}$$

$$\alpha_4 = 1; \alpha_1 = 1; \alpha_2 = 0; \alpha_3 = 0;$$

33

Для повышения эффективности криптосистемы можно использовать комбинированный подход (комбинация потокового шифра и алгоритма DES).



В данном случае поток бит ключа формируется на выходе системы шифрования DES. Начальное значение ключа K_0 (Первая стрелка слева в LFSR и DES) определяет исходное состояние LFSR и может использоваться как криптографический ключ для DES. Его значение удобно рассматривать как сеансовый ключ комбинированной криптосистемы, в которой очередные 64 бита ключа получаются при шифровании, в соответствии с алгоритмом DES, очередного состояния LFSR

SHA хэш алгоритм

В общем случае хэш-функция от сообщения m произвольной длины позволяет получить фиксированный образ, как правило небольшой.

SHA (Secure Hash Algorithm) был построен на базе MD5 (Message Digest)

Размер сообщения для шифрования должен быть меньше 2^{64} бит

Хэш-функция $H(M)$ должна быть размером в 160 бит

Определение $H(M)$ в соответствии с алгоритмом Secure Hash выполняет следующие шаги:

1. Сообщение M делится на блоки M_i . Длина каждого по 512 бит. Последний блок формируется стандартным образом
2. Пять 32-битных переменных принимают первоначальные значения:
 $\cdot A=0X67452301,$
 $\cdot B=0XEFCDA89$
 $\cdot C=0X98BADCFE,$
 $\cdot D=0X10325476,$
 $\cdot E=0XC3D2E1F0.$
3. Основной цикл рассчитан на выполнение 512-битного блока M_i от M_1 , затем M_2 и до M_j . Для каждого блока должна выполняться стандартная процедура. Пять 32-битных переменных A, B, C, D и E копируются во внутренние переменные a, b, c, d и e . Основной код:

For $i = 0$ to 79

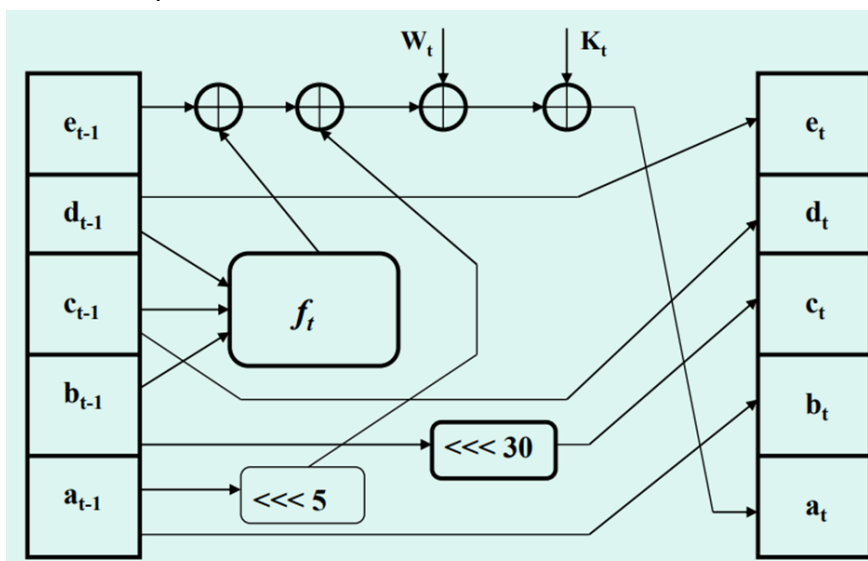
$Temp = (a \lll 5) \dot{-} f_t(b, c, d) \dot{-} e \dot{-} W_t \dot{-} K_t$

$e = d$

$d = c$

$c = (b \lll 30)$

$a = Temp$



Нелинейные булевы функции $f_i(a, b, c)$:

$$f_i(a, b, c) = (a \cdot b) + (\overline{a} \cdot c); \quad t = \overline{0, 19};$$

$$f_i(a, b, c) = a \oplus b \oplus c; \quad t = \overline{20, 39};$$

$$f_i(a, b, c) = (a \cdot b) + (a \cdot c) + (b \cdot c); \quad t = \overline{40, 59};$$

$$f_i(a, b, c) = a \oplus b \oplus c; \quad t = \overline{60, 79};$$

Существует четыре постоянных значения

- $Kt = 0X5A827999$,
- $Kt = 0X6ED9EBA1$,
- $Kt = 0X8F1BBCDC$,
- $Kt = 0XCA62C1D6$

для тех же значений t , что и в случае булевых функций

Блок M_i , состоящий из шестнадцати 32-битных слов ($M_0, M_1, \dots, M_{15}$), преобразуется в восемьдесят 32-битных слов ($W_0, W_1, \dots, W_{79}$) в соответствии с отношениями:

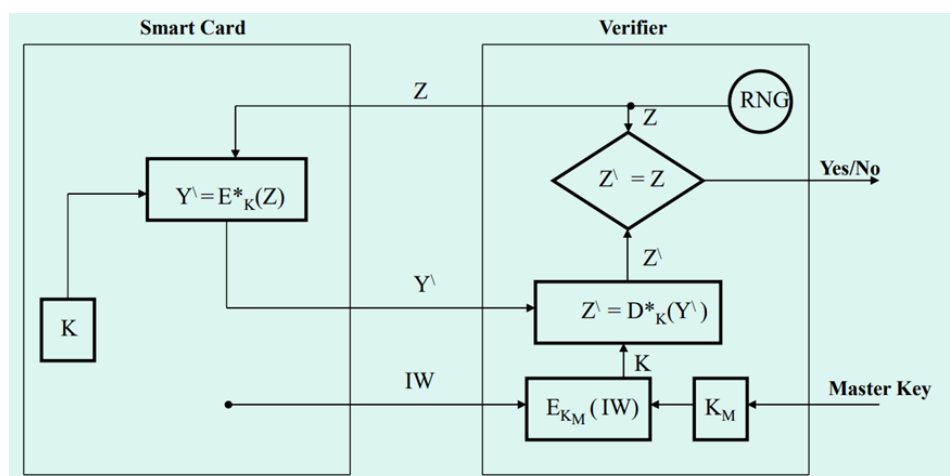
$$W_t = M_t; \quad t = \overline{0, 15};$$

$$W_t = (W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) \lll 1; \quad t = \overline{16, 79};$$

Где $\lll s$ – циклический сдвиг влево на S позиций.

1. В конце основного цикла значения переменных a, b, c, d, e добавляются к значениям соответствующих переменных A, B, C, D, E . Итоговое значение формируется как конкатенация переменных A, B, C, D и E .

Аутентификация ключей



При выдаче банковских(смарт) карт, каждый владелец в банке предоставляет собственную идентификационную информацию.

Есть идентификационное слово, оно шифруется по хостовому ключу K_M через криптографический алгоритм E : $K = E_{K_M}(IW)$. Верификатором является, к примеру банкомат. IW – Identification Ward (идентификационные данные)

Алгоритм аутентификации смарт карты (SC) верификатором (V)

1. На первом шаге аутентификации верификатор (RNG) формирует случайное число Z .
2. Число Z передается SC (смарт карте)
3. Получив число, смарт-карта использует секретный ключ K и алгоритм шифрования E^* . Это число Z шифруется и получается $Y' = E_K^*(Z)$. Y' посылается обратно верификатору
4. Параллельно карта высылает идентификационные данные IW .
5. Верификатор выполняет процедуру шифрования по главному (хостовому) ключу K_M и получает ключ K .
6. Далее верификатор выполняет дешифрование по полученному ключу K и шифротексту Y' , получая Z' . $Z' = D_K^*(Y')$.
7. Верификатор сравнивает два значения Z и Z' . В случае одинакового значения верификатор делает вывод, что карта является валидной.

БИЛЕТ 20

1. Алгоритм Евклида.
2. Текстовая стеганография.
3. PUF на базе статических запоминающих устройств.
4. Генераторы криптографического ключа для потоковых криптосистем.
Генераторы М-последовательностей. Достоинства и недостатки.

Алгоритм Евклида

Проблема нахождения наибольшего общего делителя является одной из часто используемых процедур в криптографии, которая в частности позволяет определить взаимную простоту целых чисел. Одним из исторически первых инструментов определения взаимной простоты является хорошо известный алгоритм Евклида. Методологической основой указанного алгоритма является следующая теорема:

Если $a=bq+r$, тогда наибольший общий делитель чисел a, b равен наибольшему общему делителю чисел b, r , то есть $(a, b) = (b, r)$. Доказательство: Пусть $d = (a, b)$, тогда из утверждения da и db можно сделать вывод, что d является делителем bq , а также делителем разности чисел a и bq таким образом d является делителем $a-bq=r$.

$$\text{НОД}(a, b) = d \Rightarrow$$

$$d/a \text{ и } d/b \Rightarrow$$

$$d/(a+b) \text{ и } d/(a-b)$$

$$d/b \Rightarrow d/bq$$

Алгоритм Евклида) Для любых целых чисел $a > 0$ и $b > 0$ таких, что $a > b$, и b не является делителем a , для некоторого s , существуют целые числа $q_0, q_1, q_2, \dots, q_s$ и $r_1, r_2, \dots, r_s$ такие, что $b > r_1 > r_2 > \dots > r_s > 0$ и $a = bq_0 + r_1$, $b = r_1q_1 + r_2$, $r_1 = r_2q_2 + r_3, \dots, r_{s-2} = r_{s-1}q_{s-1} + r_s$, $r_{s-1} = r_sq_s$, и соответственно $(a, b) = r_s$

Текстовая стеганография

Текстовая стеганография:

1. Кодирование за счёт изменения межстрочных интервалов
2. Внедрение информации за счёт изменений интервалов между словами
3. Кодирование за счёт изменения кодировки букв

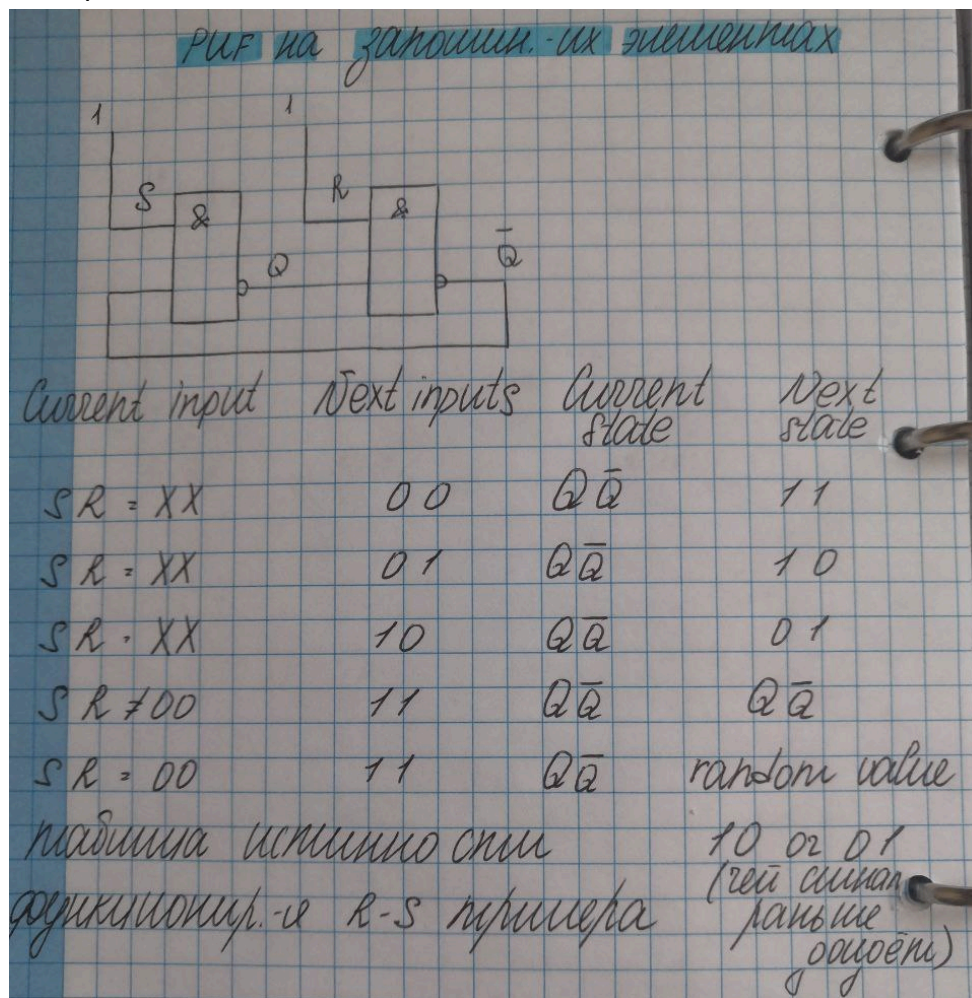
PUF на базе статических запоминающих устройств

Физически не клонированные функции (PUF) - сложные физические неупорядоченные системы со сверхбольшим объёмом структурной информации, которая удовлетворяет следующим условиям:

1. информационное содержание системы может быть найденной многократно извлечено путем измерения C различными запросами C_i и получения результирующего ответа K_i
2. количество возможных запросов s должно быть настолько большим, чтобы не было возможным за реальное время сгенерировать все ответы на эти запросы

3. должно быть возможным моделирование подобной системы и предсказания возможных пар на основании уже известных
4. должно быть невозможно физическое повторение или же клонирование PUF как физических систем

Изначально PUF использовался для защиты аппаратного обеспечения от клонирования.



Генераторы криптографического ключа для потоковых криптосистем.
Генераторы М-последовательностей. Достоинства и недостатки

БИЛЕТ 21

1. Последовательность Форда.
2. Генерирование простых чисел. Решето Эратосфена.
3. Потокосые криптосистемы. Общие сведения. Принцип потокового шифрования.
4. Вычисление мультипликативной инверсной величины.

Последовательность Форда

Частным случаем последовательностей де Брейна являются так называемые последовательности Форда, которые описываются более простым алгоритмом формирования. Последний носит рекуррентный характер и заключается в

формировании очередного m -разрядного кода $X_k = (b_2, b_3, b_4, \dots, b_{m+1})$ на основе

предыдущего кода $X_{k-1} = (b_1, b_2, b_3, \dots, b_m)$, где значение $b_{m+1} \in \{0, 1\}$ определено следующим образом. Формируется код вида

$X_{k-1}^* = (b_2, b_3, b_4, \dots, b_m, 1)$ и строятся всевозможные его циклические сдвиги,

среди них выбирается код $X_{k-1}^{**} = (b_i, b_{i+1}, \dots, b_m, 1, b_2, \dots, b_{i-1})$,

представляющий собой наибольшее m -разрядное число. Если

$b_2 = \dots = b_{i-1} = 0$, то значение b_{m+1} равно

$b_{m+1} = b_1 \oplus 1$, а в противном случае $b_{m+1} = b_1$. Отметим, что начальным кодом

X_0 для формирования последовательности Форда может быть любой код, включая

$X_0 = (0, 0, 0, \dots, 0)$.

Диаграмма состояний генератора последовательности Форда

k	$X_k =$ $=(b_2, b_3, b_4, \dots, b_{m+1})$	$X^{**}_{k-1} =$ $=(b_i, b_{i+1}, \dots, b_m, 1, b_2, \dots, b_{i-1}),$	b_{m+1}
0	0 0 0 0	1 0 0 0	$b_1 \oplus 1 = 0 \oplus 1 = 1$
1	0 0 0 1	1 1 0 0	$b_1 \oplus 1 = 0 \oplus 1 = 1$
2	0 0 1 1	1 1 1 0	$b_1 \oplus 1 = 0 \oplus 1 = 1$
3	0 1 1 1	1 1 1 1	$b_1 \oplus 1 = 0 \oplus 1 = 1$
4	1 1 1 1	1 1 1 1	$b_1 \oplus 1 = 1 \oplus 1 = 0$
5	1 1 1 0	1 1 1 0	$b_1 = 1$
6	1 1 0 1	1 1 1 0	$b_1 = 1$
7	1 0 1 1	1 1 1 0	$b_1 \oplus 1 = 1 \oplus 1 = 0$
8	0 1 1 0	1 1 1 0	$b_1 = 0$
9	1 1 0 0	1 1 0 0	$b_1 = 1$
10	1 0 0 1	1 1 0 0	$b_1 \oplus 1 = 1 \oplus 1 = 0$
11	0 0 1 0	1 0 1 0	$b_1 \oplus 1 = 0 \oplus 1 = 1$
12	0 1 0 1	1 1 1 0	$b_1 = 0$
13	1 0 1 0	1 0 1 0	$b_1 \oplus 1 = 1 \oplus 1 = 0$
14	0 1 0 0	1 1 0 0	$b_1 = 1$
15	1 0 0 0	1 0 0 0	$b_1 \oplus 1 = 1 \oplus 1 = 0$
16	0 0 0 0	1 0 0 0	...
...	...		

Генерирование простых чисел. Решето Эратосфена

Решето Эратосфена — это один из древнейших и наиболее эффективных алгоритмов для нахождения всех простых чисел до заданного целого числа n .

1) Если в наборе целых чисел $2, 3, 4, \dots, N$, удалить все числа, которые делятся на первые r простых чисел $2, 3, 5, 7, \dots, p_r$, тогда первое (наименьшее) не удаленное число будет простым.

2) Если в наборе целых чисел $2, 3, 4, \dots, N$, удалить все числа, которые делятся на простые числа меньше или равные $(N)^{1/2}$, тогда все оставшиеся числа будут простыми числами p принадлежащими интервалу $(N)^{1/2} < p \leq N$.

Доказательство:

1) Любое составное число n делится, по крайней мере, на одно простое число меньше, чем n .

2) Каждое составное число n , такое что $(N)^{1/2} < n \leq N$ делится, по крайней мере, на одно простое $p_i \leq (n)^{1/2} \leq (N)^{1/2}$ то есть на одно из чисел $2, 3, \dots, p_r$ ($p_r < (N)^{1/2} \leq p_{r+1}$) и, следовательно, будет вычеркнуто. #

Пример 3.1. В качестве примера рассмотрим пятьдесят первых чисел числового ряда, то есть

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49											

В результате удаления всех чисел кратных 2, 3, 5 и 7 получим множество простых чисел: 11,13,17,19,23,29,31,37,41,43,47, которое удовлетворяет вышеприведенной теореме.

Решето Эратосфена основано на исключении составных чисел путем последовательного исключения всех их кратных, начиная с первого простого числа. Рассмотрим пошаговое описание алгоритма:

1. Создайте список чисел от 2 до n .
2. Начните с первого числа в списке (2), которое является простым.
3. Удалите все кратные этого числа из списка.
4. Перейдите к следующему числу в списке, которое не было удалено, и повторите шаг 3.
5. Повторяйте процесс до тех пор, пока не достигнете числа $\sqrt{n}$.

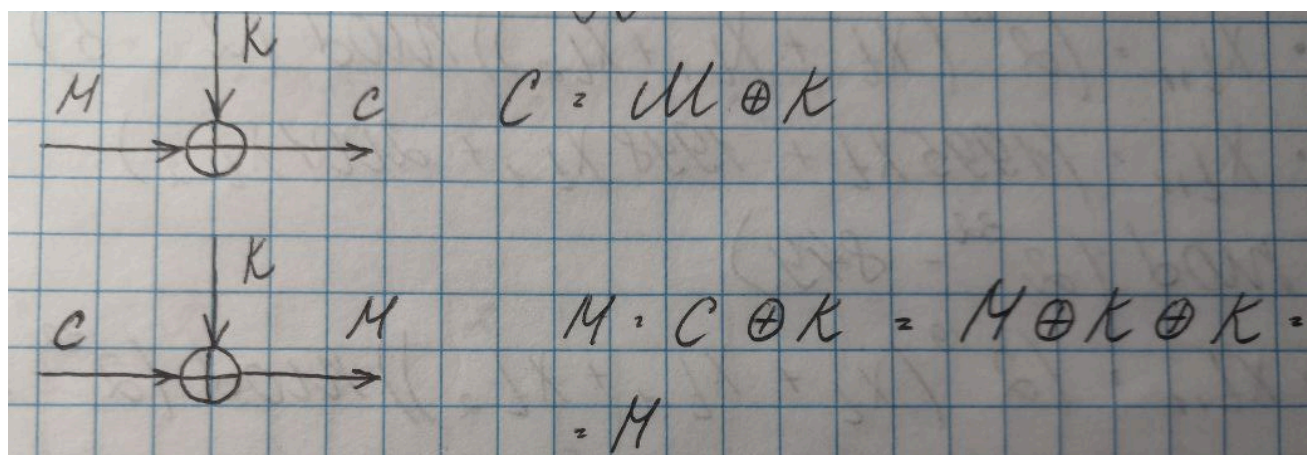
После выполнения этих шагов все оставшиеся числа в списке будут простыми.

Потоковые криптосистемы. Общие сведения. Принцип потокового шифрования

Потоковый принцип шифрования основан на применении простейшей операции сложения по модулю 2.

Различают два типа потоковых шифров синхронные (synchronous) и самосинхронизирующиеся (self-synchronizing) шифраторы. В первом случае последовательность значений ключа является независимой от открытого текста и шифротекста. Понятие синхронный шифратор, связано с тем, что для успешного дешифрирования требуется синхронизация между последовательностью значений ключа и зашифрованным текстом. В то время как для самосинхронизирующихся шифраторов значения ключа зависят либо от исходного текста, либо от шифротекста.

На изображении ниже приведён пример синхронных шифраторов.



Подобная структура шифратора позволяет достичь высокой помехозащищенности и надежности функционирования. Действительно искажение одного символа в зашифрованном тексте приведет к искажению только одного символа исходного текста при дешифрировании. В тоже время нарушение синхронизации является причиной искажения всех последующих символов исходного текста с момента потери синхронного формирования ключа с шифротекстом. Подобная ситуация может возникнуть при удалении либо добавлении символов шифротекста. С целью восстановления синхронизации используются самосинхронизирующиеся потоковые шифраторы. При их использовании ошибочно добавленный или удаленный бит вызывает только ограниченное количество ошибочных символов в дешифрованном тексте, после чего правильный текст восстанавливается.

Различают принципиальные отличия между симметричными блочными шифраторами (DES, IDEA, BLOWFISH, ...) и потоковыми системами шифрования.

1. Блочный шифр одновременно шифрует целый блок данных, как правило, 64 бита, а в потоковых шифраторах процедура шифрования и дешифрирования выполняется поразрядно.
2. В блочном шифре, каждый бит зашифрованного текста является сложной функцией открытого текста, и криптографического ключа. В потоковом шифраторе каждый бит шифротекста получается в результате поразрядной операции сложения по модулю два очередного бита открытого текста и бита ключа.
3. Блочный шифр не требует задания начального вектора для генератора последовательности символов ключа, а в потоковом шифраторе такой вектор необходим.
4. Блочные шифры подобно как DES, используется в коммерческом секторе, а потоковые шифры используется для специальных приложений и, как правило, широко не обсуждаются.

Вычисление мультипликативной инверсной величины

Теорема 3.17. Если наибольший общий делитель d чисел a и n равен единице $(a,n)=1$, то есть, a и n являются взаимно простыми числами, то линейное сравнение $ax=b \bmod n$ имеет одно решение.

Доказательство: Предположим, что существует полная система вычетов $\{0,1,2,\dots,n-1\}$ по модулю n . Тогда согласно тому, что a и n взаимно простые целые числа, множество чисел $\{0\cdot a, 1\cdot a, 2\cdot a, \dots, (n-1)\cdot a\}$ образует полную систему вычетов по модулю n . Тогда среди всех целых чисел есть одно ax_0 и только одно с вычетом равным b .#

Пример 3.18. Линейное сравнение $2x=1 \bmod 3$ имеет одно решение $x_0=2$, так как $a=2$ и $n=3$ являются взаимно простыми числами.

Теорема 3.17 позволяет сформулировать задачу нахождения решения линейного сравнения для случая, когда a и n являются взаимно простыми числами. Здесь возможны два случая в зависимости от величины b .

Для $b=1$ линейное сравнение принимает вид $ax=1 \bmod n$, где неизвестная величина $x=a^{-1}$ является **мультипликативной инверсной величиной** по отношению к a . Тогда $aa^{-1}=1 \bmod n$.

Для вычисления мультипликативной инверсной величины возьмём два линейных сравнения: исходное сравнение $ax=1 \bmod n$ в соответствии с условием теоремы 3.17 и сравнение $1=a^{\psi(n)} \bmod n$ соответствующее теореме Эйлера, затем, используя лемму 3.6, перемножим левую и правую части этих сравнений. Как результат получим $ax=a^{\psi(n)} \bmod n$. Используя лемму 3.2, разделим правую и левую части последнего равенства $ax=a^{\psi(n)} \bmod n$ на a , получим

$$x=a^{\psi(n)-1} \bmod n,$$

что и является основным расчетным соотношением для вычисления мультипликативной инверсной величины. Для случая, когда n является простым числом $x=a^{n-2} \bmod n$.

Мультипликативная обратная величина (или мультипликативный обратный элемент) числа a по модулю m — это такое число b , что произведение a и b по модулю m равно 1. Математически это выражается следующим образом:

$$a \cdot b \equiv 1 \pmod{m}$$

Это означает, что $a \cdot b - 1$ делится на m .

Условия существования: мультипликативная обратная величина a по модулю m существует тогда и только тогда, когда a и m взаимно просты, то есть их наибольший общий делитель (НОД) равен 1:

$$\gcd(a,m)=1$$

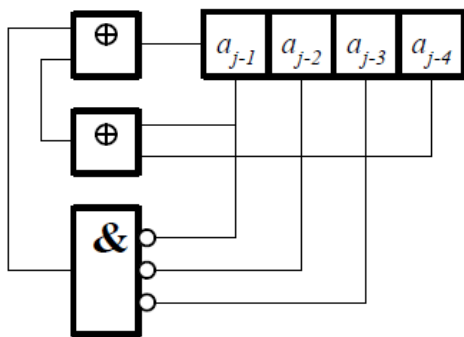
БИЛЕТ 22

1. Генератор последовательностей de Bruin.
2. Стеганография (Общая информация).
3. Абсолютная секретность.
4. Управление ключами. Деление ключей.

Генератор последовательностей de Bruin

Основан на следующих утверждениях:

1. Для любой M -последовательности существует только один код вида $00\dots 01$, сформированный на LFSR
2. Для любой M -последовательности всегда существует последовательность кодов формируемых на LFSR
3. Для любой M -последовательности существует только один код, формируемый на LFSR, состоящий из $m-1$ нулей, стоящих на $m-1$ регистрах LFSR

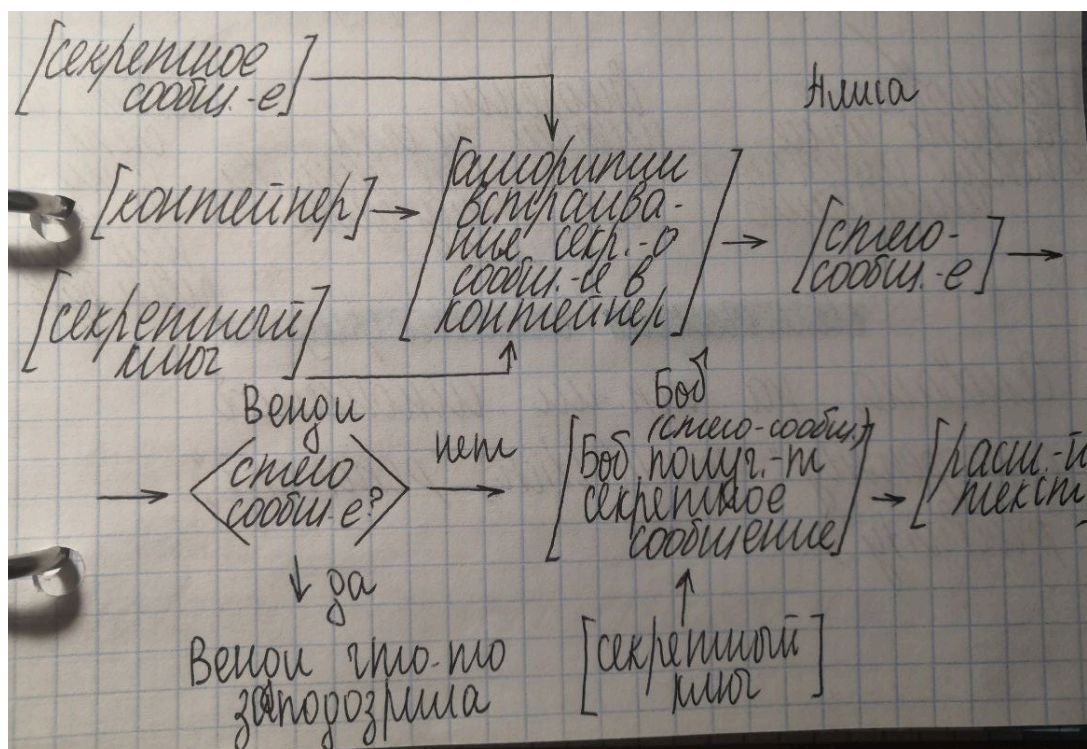


Стеганография (Общая информация)

Стеганография - двойная криптография, в которой факт того, что информация передаётся тоже засекречен.

Стеганография применяется как для секретной передачи информации, так и для реализации систем водяных знаков и отпечатков пальцев.

В общем случае стеганографическое преобразование информации описывается следующей диаграммой:



Стеганография:

1. цифровая
2. текстовая
3. лексическая
4. графическая

Абсолютная секретность

Как уже отмечалось ранее, основным вкладом К.Шеннона в теорию и практику защиты информация явилось его теоретическое обоснование возможности создания абсолютно секретных криптосистем. В своем анализе криптографических систем он выделял три вида информации:

1. Исходные сообщения M , описываемые вероятностями $p(M)$ появления сообщений M , где $\sum_M P(M) = 1$.

2. Зашифрованные сообщения C , описываемые вероятностями $p(C)$ появления шифротекстов C , где $\sum_C P(C) = 1$.

3. Ключи K , описываемые вероятностями $p(K)$ появления ключей K , где $\sum_K P(K) = 1$.

Если $P_C(M)$ есть вероятность того, что на основании исходного сообщения M было получено зашифрованное сообщение C , тогда согласно определению К.Шеннона **абсолютная секретность** будет достигнута тогда и только тогда, когда будет выполняться следующее равенство.

$$P_C(M) = P(M).$$

Здесь $P(M)$ есть вероятность того, что на основании исходного сообщения M может быть получено любое другое зашифрованное сообщение. Приведенное равенство свидетельствует о том, что перехват зашифрованного сообщения C не даёт криптоаналитику никакой дополнительной информации об исходном сообщении.

Необходимым и достаточным *условием абсолютной секретности* является выполнение для каждого C следующего равенства

$$P_M(C) = P(C),$$

которое для конкретного зашифрованного сообщения C должно выполняться для всех M .

Условие абсолютной секретности $P_M(C) = P(C)$ означает, что вероятность $P_M(C)$ получения зашифрованного сообщения C при шифровании исходного сообщения M такая же, что и вероятность $P(C)$ получения C при шифровании некоторого другого сообщения M' (шифрование M' выполнялось другим ключом).

Таким образом, абсолютная секретность возможна, в случае использования, по крайней мере, столько абсолютно случайных криптографических ключей, сколько существует возможных исходных сообщений, которые кодируются этими ключами.

Управление ключами. Деление ключей

БИЛЕТ 23

1. Свойства М-последовательностей (свойства 6-9).
2. Комбинированные генераторы ключа. Генератор Geffe.
3. Теорема Ферма.
4. DES: функция f , реализующая подстановку.

Свойства М-последовательностей (свойства 6-9)

1. Свойство сдвига и сложения:

8. Свойство **сдвига и сложения**. Для любого s ($1 \leq s < L$) существует $r \neq s$ ($1 \leq r < L$) такое что $\{a_k\} \oplus \{a_{k-s}\} = \{a_{k-r}\}$.

$$\begin{array}{ll} \{a_0\} & 000111101011001 \\ \{a_2\} & 011110101100100 \\ \{a_9\} & 011001000111101 \end{array}$$

№ такта	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$\{a_0\}$	0	1	1	1	1	0	1	0	1	1	0	0	1	0	0
$\{a_2\}$	1	1	1	0	1	0	1	1	0	0	1	0	0	0	1
$\{a_0\} \oplus \{a_2\} = \{a_9\}$	1	0	0	1	0	0	0	1	1	1	1	0	1	0	1

2. Свойство децимации: Если индекс децимации равен 2, то из последовательности выбирается каждый 2 элемент. В результате будет получено М-последовательность формируемая полиномом той же степени.
Пример: 000111101011001 $\Rightarrow$ 001111010110010

8. Автокорреляционная функция М-последовательности равна

$$R(\tau) = \begin{cases} 1, & \tau = 0 \bmod L; \\ \frac{1}{L}, & \tau \neq 0 \bmod L. \end{cases}$$

Другими словами, среди L фазовых сдвигов М-последовательности только в одном случае из L происходит совпадение, а в остальных $L - 1$ случаях они не совпадают.

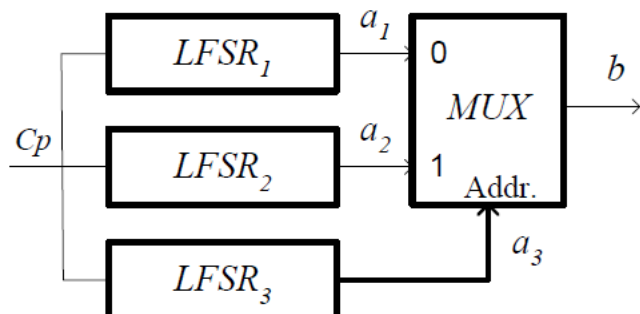
3. Среди всех М-последовательности можно найти одну такую которая при децимации по модулю 2 даст саму себя

7. Свойство серий. В М-последовательности, описываемой характеристическим полиномом $f(x)$ степени m , серия из одной единицы встретится 2^{m-1} раз, а серия из одного нуля – $2^{m-1} - 1$ раз. Серия из двух идущих подряд единиц встретится 2^{m-2} раза, а серия из двух идущих подряд нулей встретится $2^{m-2} - 1$ раз. Серия 10 (а также серия 01) встретится 2^{m-2} раза. В общем случае серия из r символов встретится в М-последовательности 2^{m-r} раз.

- 5.

Комбинированные генераторы ключа. Генератор Geffe

Одним из наиболее очевидных решений при построении комбинированных генераторов ключа является генератор Геффи (Geffe generator). В основе данного генератора лежит использование трех LFSR и двухвходового мультиплексора:



Два генератора M последовательностей $LFSR_1$ и $LFSR_2$ являются источниками последовательностей a_1 и a_2 подаваемыми на информационные входы мультиплексора, а последовательность a_3 формируемая $LFSR_3$, подается на адресный вход мультиплексора. Все три $LFSR$ функционируют синхронно при подаче синхронизирующих импульсов Cp . Выходная последовательность b формируется в соответствии со следующим выражением $b = (a_2 a_3) + (a_1 (1 \oplus a_3))$, а период равен наименьшему общему кратному периодов трех M последовательностей генерируемых $LFSR_1$, $LFSR_2$ и $LFSR_3$. Если их порождающие полиномы имеют степени m_1 , m_2 и m_3 то максимальный период последовательности b будет равен $(2^{m_1}-1) \times (2^{m_2}-1) \times (2^{m_3}-1)$. Обычно $m_1 \neq m_2 \neq m_3$, а их величины близки и, как правило, определяются размерностью сеансового ключа.

Возможны различные модификации генератора Геффи в части увеличения количества используемых $LFSR$, а также режимов их функционирования.

Теорема Ферма

Большое практическое значение в криптографии имеет известная теорема Ферма.

Теорема 3.11. (Fermat). Если p простое число и $(a, p) = 1$, где a целое, тогда

$$a^{p-1} = 1 \mod p.$$

Доказательство: Пусть для заданных взаимно простых величин p и a , $(a, p) = 1$ существует $p-1$ положительных произведений $a, 2a, 3a, \dots, (p-1)a$. Тогда любая пара ia, ja ($i \neq j$) этих произведений не сравнима по модулю p , то есть

$$ia \not\equiv ja \mod p,$$

что следует из леммы 3.2 для сравнений приведенной ранее. Следовательно, каждое произведение имеет свой уникальный ненулевой вычет r_i , ($1 \leq r_i \leq p-1$). Эти вычеты $\{1, 2, 3, \dots, (p-1)\}$ для множества чисел $\{a, 2a, 3a, \dots, (p-1)a\}$ расположены в произвольном порядке и формируют полную систему вычетов $\{r_\alpha, r_\beta, r_\gamma, \dots, r_\lambda\} = \{1, 2, 3, \dots, (p-1)\}$, где $a = r_\alpha \bmod p$, $2a = r_\beta \bmod p$, $3a = r_\gamma \bmod p, \dots, (p-1)a = r_\lambda \bmod p$.

На основе леммы 3.7 для сравнений $a = r_\alpha \bmod p$, $2a = r_\beta \bmod p$, $3a = r_\gamma \bmod p, \dots, (p-1)a = r_\lambda \bmod p$ получим $a \cdot 2a \cdot 3a \cdot \dots \cdot (p-1) \cdot a = r_\alpha \cdot r_\beta \cdot r_\gamma \cdot \dots \cdot r_\lambda \bmod p$. Учитывая, что $\{r_\alpha, r_\beta, r_\gamma, \dots, r_\lambda\} = \{1, 2, 3, \dots, (p-1)\}$ и выполнив перестановку, будем иметь $a \cdot 2a \cdot 3a \cdot \dots \cdot (p-1) \cdot a = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-1) \bmod p$. Окончательно $a^{p-1} \cdot (p-1)! = (p-1)! \bmod p$. Поскольку $(p-1)!$ и p взаимно простые целые числа, то есть $((p-1)!, p) = 1$, тогда окончательно получаем $a^{p-1} = 1 \bmod p$. #

Отметим, что теорема Ферма справедлива только для простых чисел p . Обобщением данной теоремы является теорема Эйлера, которая основана на использовании функции Эйлера. Определим данную функцию для целого числа n .

DES: функция f , реализующая подстановку

Стандарт шифрования данных (Data Encryption Standard - DES) в настоящее время является одним из наиболее популярных методов шифрования. Он часто используется на практике и широко обсуждается в литературе. Основой данного метода шифрования является совместное использование двух простейших операций, таких как подстановки и перестановки.

В 1977 Национальное Бюро Стандартов США представило стандарт шифрования данных (Data Encryption Standard - DES) для использования в несекретных приложениях правительства США, а также различными неправительственными организациями. Алгоритм шифрования был разработан на фирме IBM и был производным от криптосистемы LUCIFER. DES кодирует 64-битные блоки данных с использованием 56-битного ключа. Существует мнение, что 56-битный ключ являлся достаточно сильным на ранних стадиях использования стандарта, а в настоящее время его размерность недостаточна.

Алгоритм стандарта DES, который используется и для кодирования и для декодирования, в обобщенном виде представлен на рисунке 4.2. Входной блок $T = t_1 t_2, \dots, t_{64}$ состоящий из 64 бит, предварительно перемешивается согласно исходной перестановке IP , $T_0 = IP(T)$. После прохождения через 16 итераций функции F , он перемешивается согласно обратной перестановке IP^{-1} и формирует итоговый результат. Перестановки IP и IP^{-1} заданы в таблице 4.1.

Таблица 4.1.

Таблицы перестановки IP и IP^{-1}

перестановка IP	перестановка IP^{-1}
58 50 42 34 26 18 10 2	40 8 48 16 56 24 64 32
60 52 44 36 28 20 12 4	39 7 47 15 55 23 63 31
62 54 46 38 30 22 14 6	38 6 46 14 54 22 62 30
64 56 48 40 32 24 16 8	37 5 45 13 53 21 61 29
57 49 41 33 25 17 9 1	36 4 44 12 52 20 60 28
59 51 43 35 27 19 11 3	35 3 43 11 51 19 59 27
61 53 45 37 29 21 13 5	34 2 42 10 50 18 58 26
63 55 47 39 31 23 15 7	33 1 41 9 49 17 57 25

Эти таблицы читаются слева направо, сверху вниз. Например, IP перемещает $T=t_1t_2...t_{64}$ в $T_0=t_{58}t_{50}...t_7$. Обе таблицы фиксированы. Между начальной и последней перестановками алгоритм выполняет 16 итераций функции F , которая комбинирует подстановку и перестановку. Пусть T_i обозначает результат i -й итерации, а L_i и R_i обозначают левую и правую половины T_i соответственно, таким образом, $T_i = L_i R_i$, где $L_i = t_1t_2...t_{32}$, $R_i = t_{33}t_{34}...t_{64}$. Тогда $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$, где $\oplus$ есть знак операции поразрядного сложения по модулю два (XOR), а K_i - это 48-битный ключ для i -й итерации.

На следующем рисунке (Рис.4.3) представлена функция $F(R_{i-1}, K_i)$. Первый операнд данной функции R_{i-1} увеличивается до 48-битного блока $E(R_{i-1})$ используя таблицу 4.2 побитного выбора E . Эта таблица применяется для перестановок, таким образом, что некоторые биты R_{i-1} выбираются более одного раза.



В результате для $R_{i-1}=r_1r_2...r_{32}$, имеем $E(R_{i-1})=r_{32}r_1r_2...r_{32}r_1$. Далее полученный результат $E(R_{i-1})$ поразрядно суммируется по модулю два с 48 битным значением ключа K_i , а полученный результат разбивается на восемь 6-битных блоков $B_1B_2...B_8$, где $E(R_{i-1}) \oplus K_i = B_1B_2...B_8$. Каждый 6-битный блок B_j затем используется в качестве входных данных подстановочной функции F , реализованной с использованием восьми, так называемых, *S-боксов* (*S-box*). Данная функция *S-box*, используя 6-битный блок B_j , возвращает 4-битный блок $S_i(B_j)$ как результат подстановки. Результирующие 4-битные блоки данных объединяются вместе в окончательный 32-битный блок, который затем перемешивается с помощью перестановки P , показанной в таблице 4.2. Таким образом, 32-битный блок, возвращаемый функцией $F(R_{i-1}, K_i)$, будет равен $P(S_1(B_1)...S_8(B_8))$. Блок-диаграмма функции $F(R_{i-1}, K_i)$, (S_i) представлена на следующем рисунке.

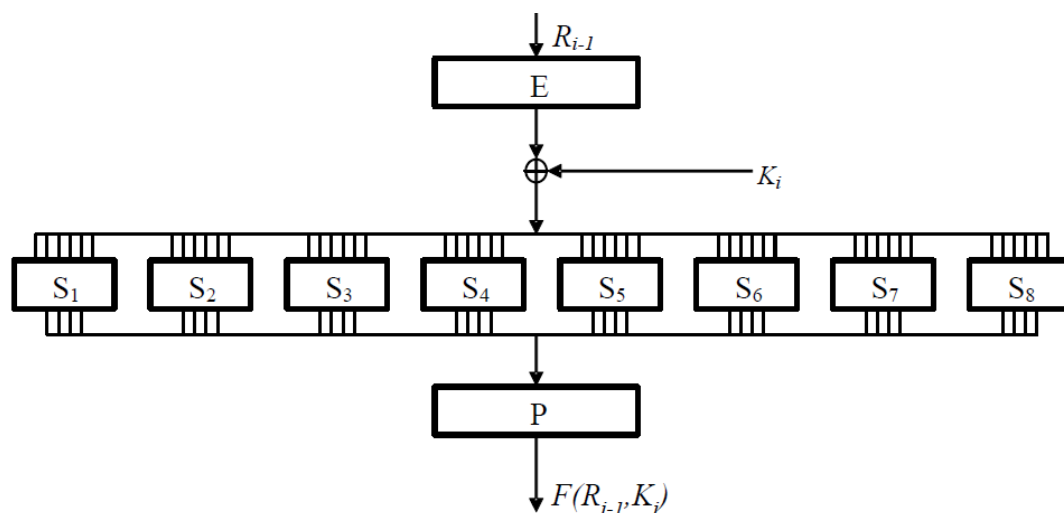


Рис. 4.3. Функция $F(R_{i-1}, K_i)$

Как отмечалось ранее, каждый *S-box* S_i отображает 6-битный блок $B_j=b_1b_2b_3b_4b_5b_6$ в 4-битный блок как показано в таблице 4.3. для случая S_1 .

Таблица 4.3.

Функция подстановки(S-box) S_1

Строка (b_1b_6)	Столбец ($b_2b_3b_4b_5$)															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Процесс подстановки осуществляется следующим образом. Четверичное число 0, 1, 2 или 3, соответствующее b_1b_6 , выделяет строку в таблице, в то время как шестнадцатеричное число 0,1,2, ... или 15, соответствующее $b_2b_3b_4b_5$ выделяет столбец. Тогда значение $S_i(B_j)$ это есть 4-битное шестнадцатеричное число, расположенное в строке и столбце определяемыми кодами b_1b_6 и $b_2b_3b_4b_5$.

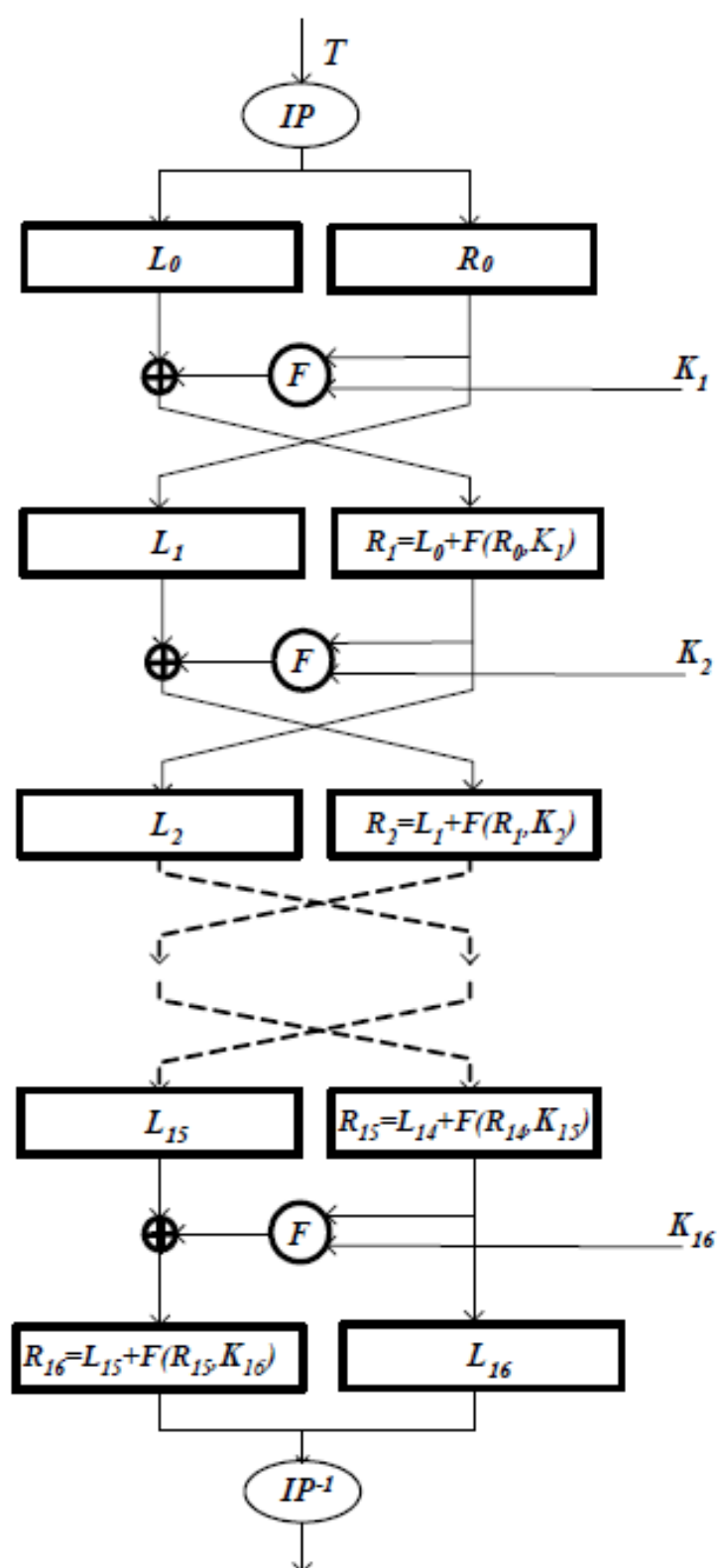


Рис. 4.2. Диаграмма DES

Функция f в алгоритме DES (Data Encryption Standard) выполняет основную часть преобразований, включающих подстановку и перестановку. Она является ключевым элементом шифрования, определяющим сложность и безопасность алгоритма. Вот пошаговое описание функции f :

Описание функции f

1. Входные данные:
 - 32-битный блок данных RRR (правая половина блока).
 - 48-битный ключ KKK (субключ, специфичный для каждого раунда).
2. Этапы функции f :

1. Расширение EEE-перестановка

32-битный вход RRR расширяется до 48 бит с помощью расширения EEE-перестановки. В результате этого расширения некоторые биты повторяются, чтобы получить 48-битный выход.

2. Смешивание с ключом

48-битный результат расширения складывается по модулю 2 (операция XOR) с 48-битным ключом KKK:

$$B = E(R) \oplus KB = E(R) \oplus KB = E(R) \oplus K$$

3. Подстановка через S-блоки

48-битный результат BBB разбивается на восемь 6-битных блоков $B_1, B_2, \dots, B_8$. Каждый из этих блоков обрабатывается через соответствующий S-блок, который выполняет нелинейное преобразование, сжимая 6-битный вход в 4-битный выход. S-блоки обеспечивают криптографическую стойкость, выполняя нелинейную подстановку.

4. P-перестановка

32-битный выход от S-блоков подвергается P-перестановке, которая переставляет биты в определенном порядке для дополнительного смешивания.